

Deep Learning for Computer Vision

114-1/Fall 2025

<https://cool.ntu.edu.tw/courses/53063> (NTU COOL)

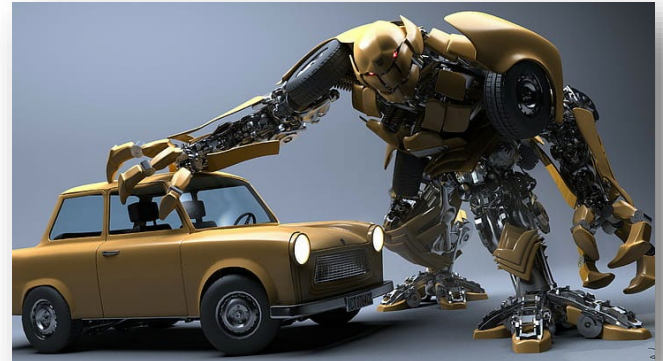
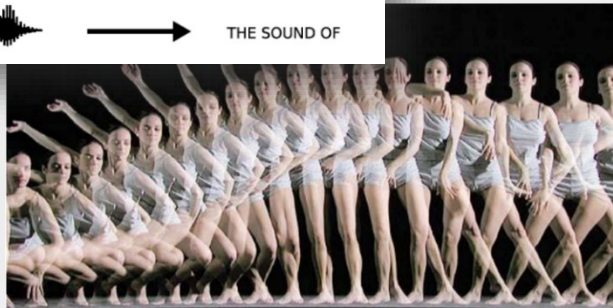
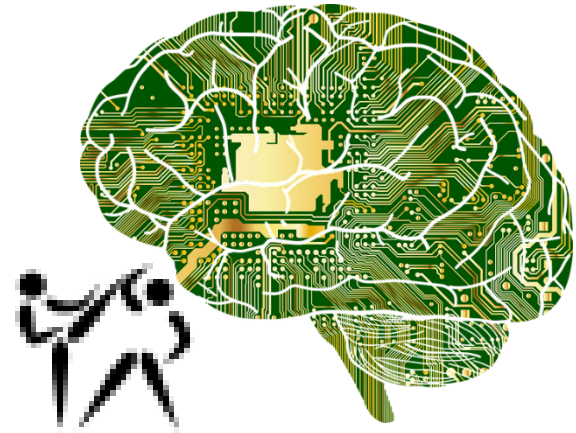
<http://vllab.ee.ntu.edu.tw/dlcv.html> (Public website)

Yu-Chiang Frank Wang 王鈺強, Professor

Dept. Electrical Engineering, National Taiwan University

What to Be Covered Today...

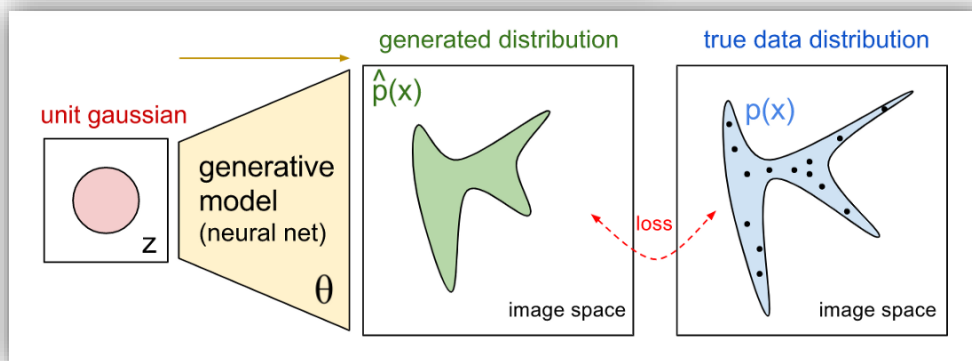
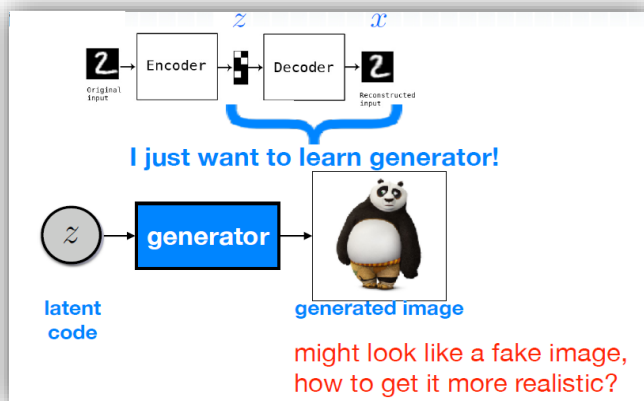
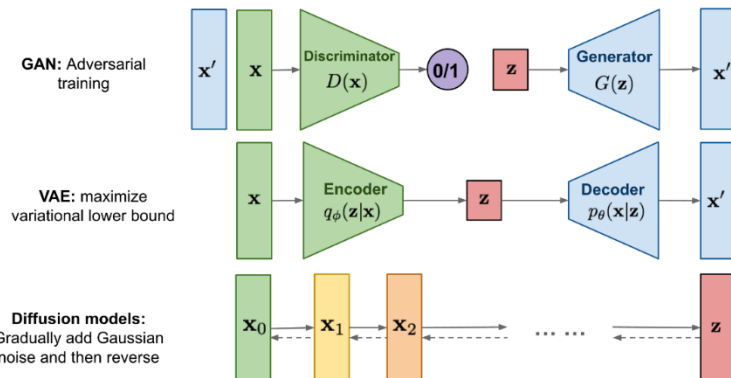
- Adversarial Learning for Transfer Learning
- Recurrent Neural Networks
 - From RNN to LSTM & GRU
 - Sequence-to-Sequence Learning
 - Attention in RNN
- Transformer



Recap: From VAE to GAN

- Remarks

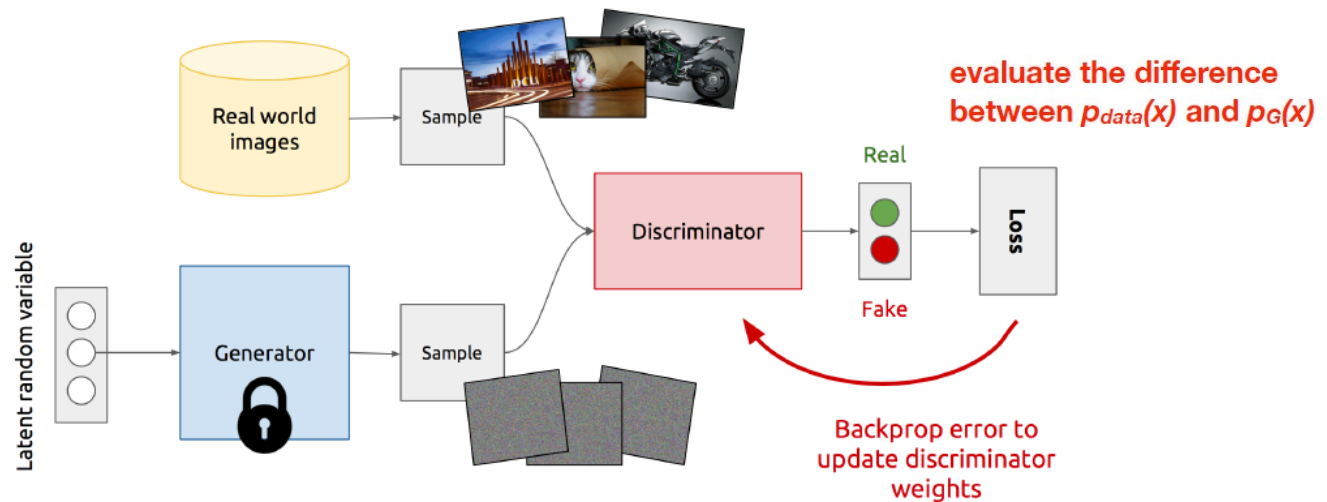
- We only need the decoder/generator in practice.
- We prefer fast generation.
- How do we know if the output images are sufficiently good?



GAN (cont'd)

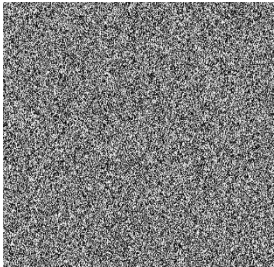
- Remarks
 - A function maps **normal distribution** $N(\mathbf{0}, I)$ to P_{data}
 - How good we are in mapping P_g to P_{data} ?
 - Train & ask the discriminator!
 - Conduct a two-player min-max game (see next slide for more details)

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

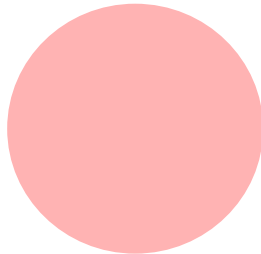


Recap: Problem #1 - Vanishing Gradients

- What Might Go Wrong?
 - GAN training is often unstable.
 - In other words, training might not converge properly.
 - The discriminator which we prefer is...



0



0.2



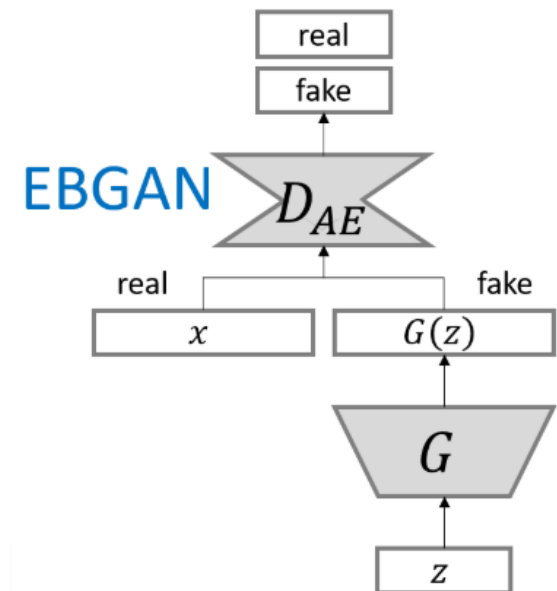
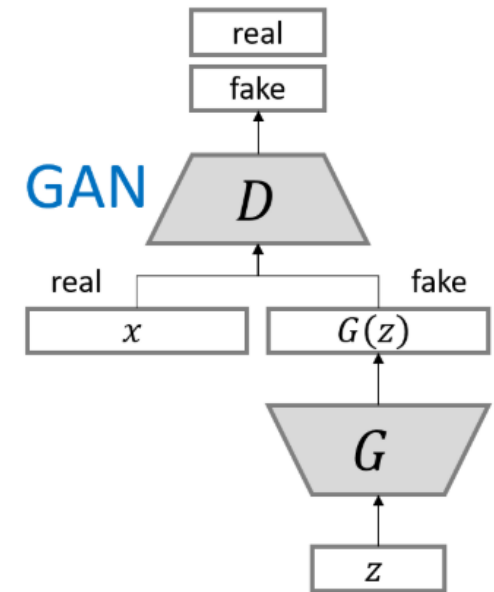
0.6



1

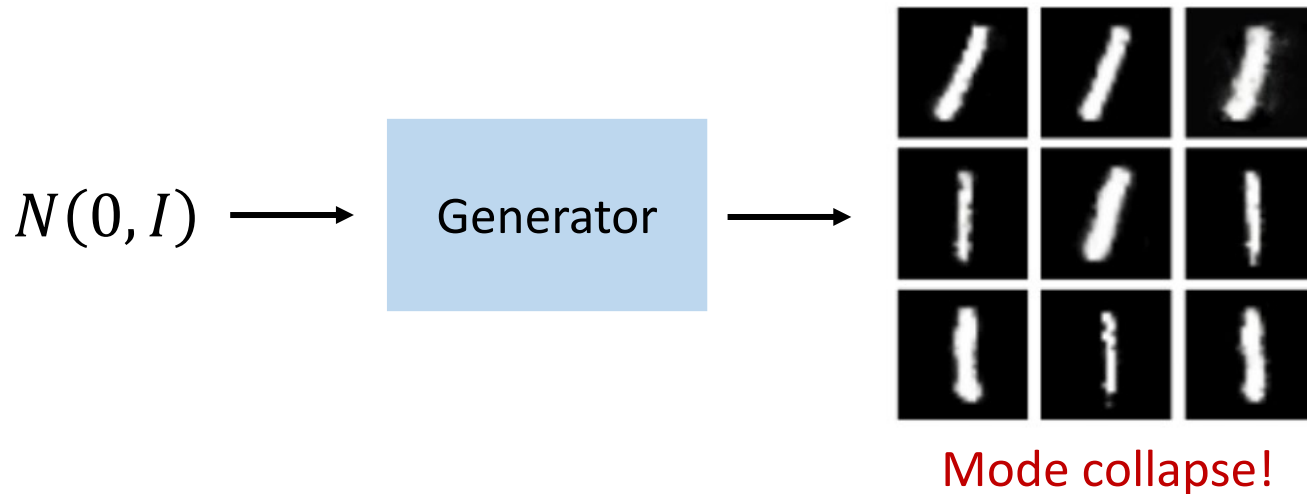
Energy-Based GAN

- Energy Function
 - Converting input data into scalar outputs, viewed as energy values
 - Desired configuration is expected to output low energy values & vice versa.
- Energy Function as Discriminator
 - Use of autoencoder; can be pre-trained!
 - Reconstruction loss outputs a range of values instead of binary logistic loss.
 - Empirically better convergence



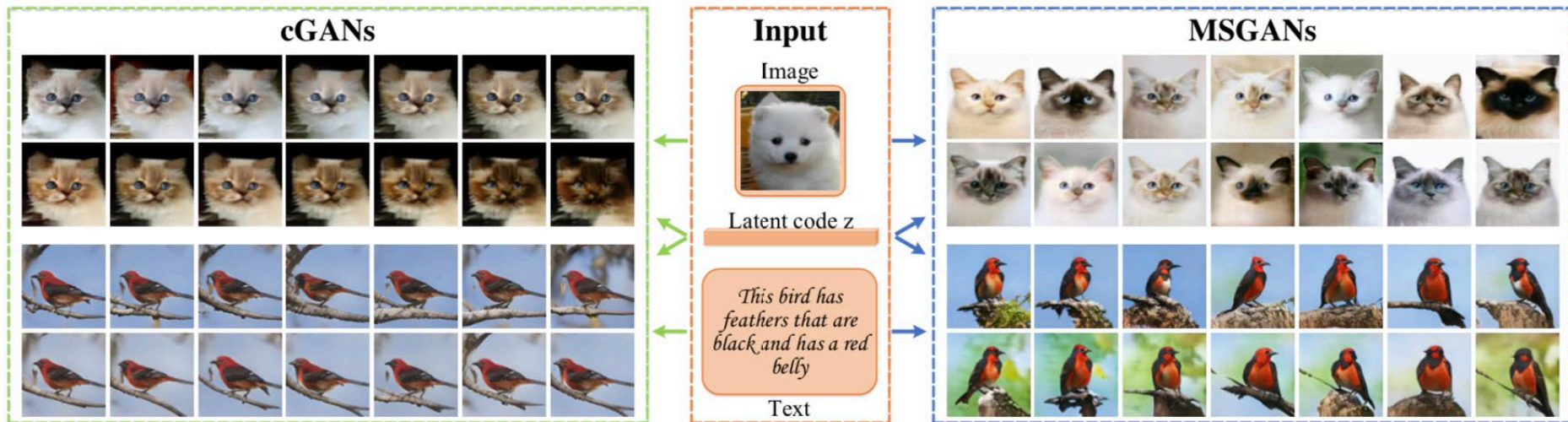
Recap: Problem #2 - Mode Collapse

- Remarks
 - The generator only outputs a limited number of image variants regardless of the inputs.



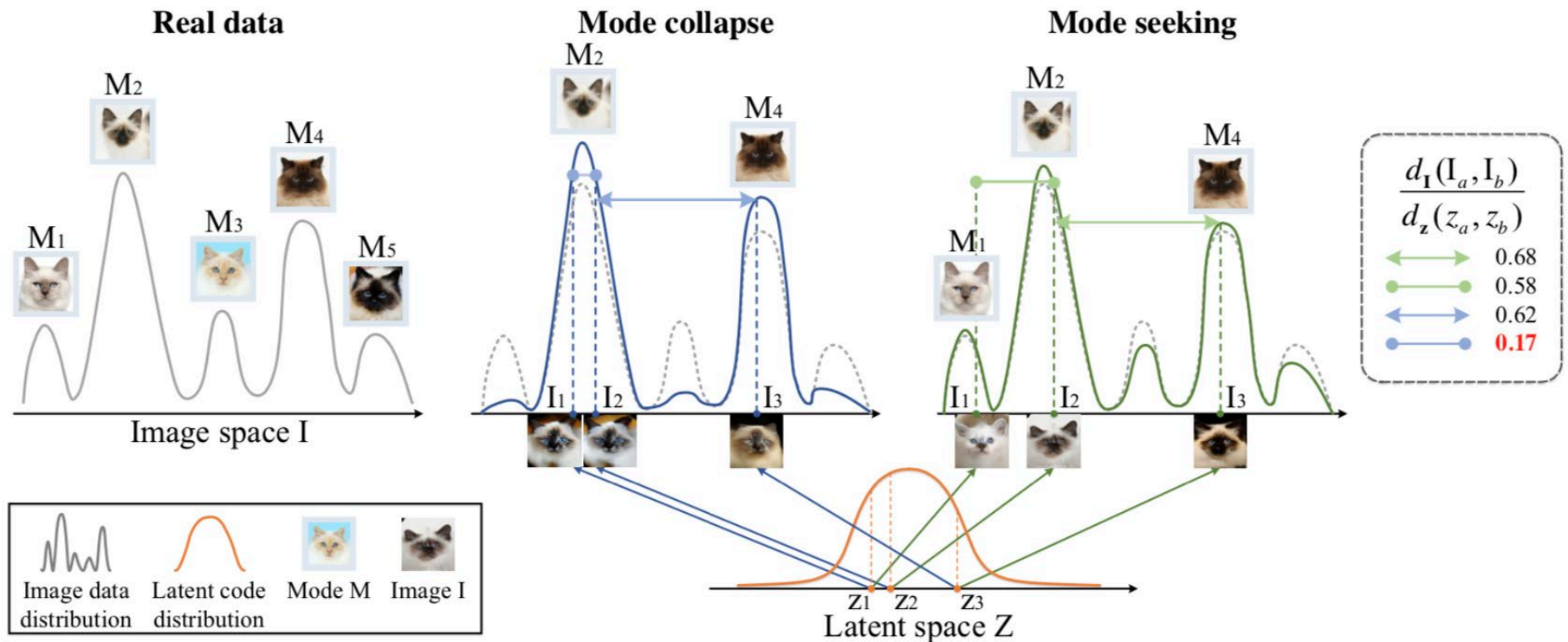
MSGAN

- To address the **mode collapse** issue by **conditional GANs**
- Mode Seeking Generative Adversarial Networks for Diverse Image Synthesis
- With the goal of producing **diverse** image outputs.



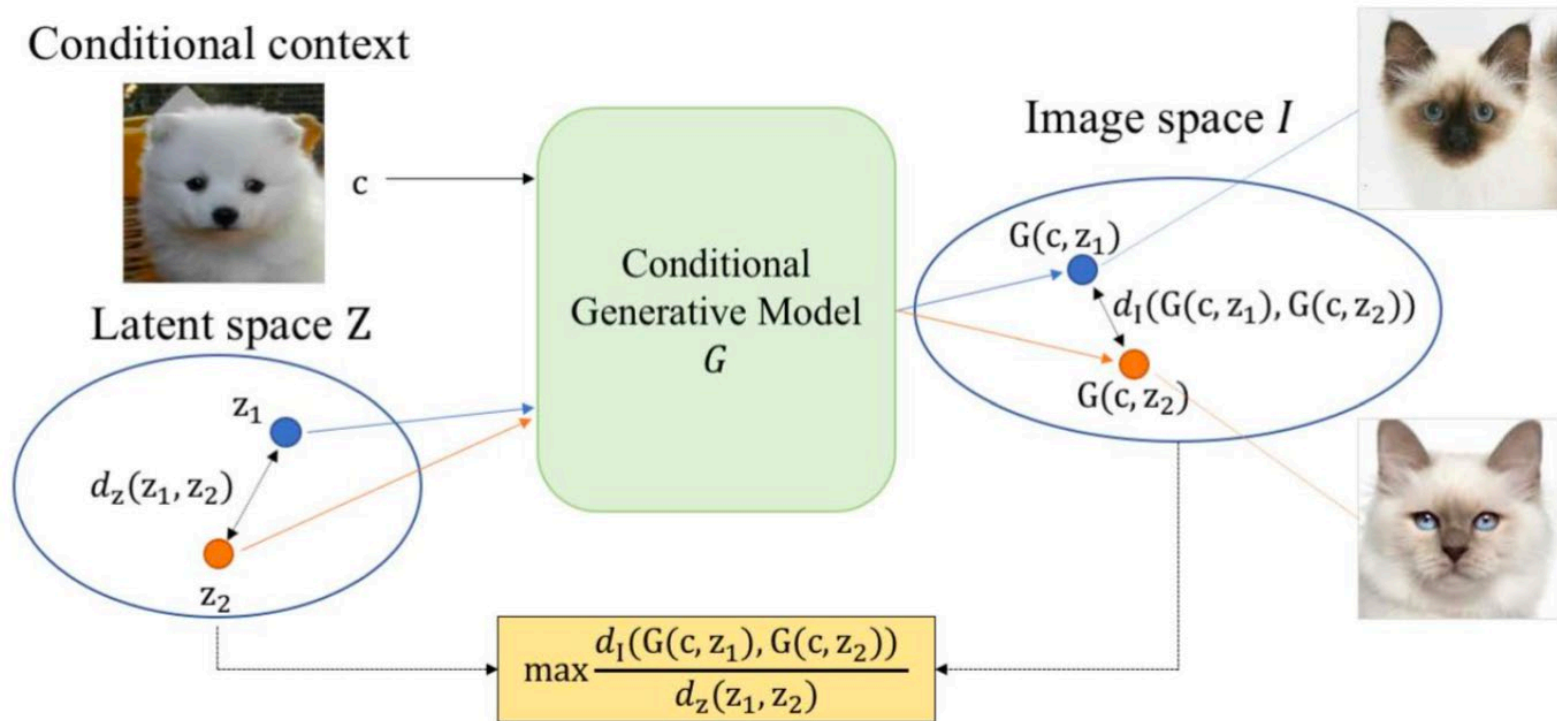
MSGAN (cont'd)

- Motivation (for unconditional GAN)



MSGAN (cont'd)

- Proposed Regularization (for conditional GAN)



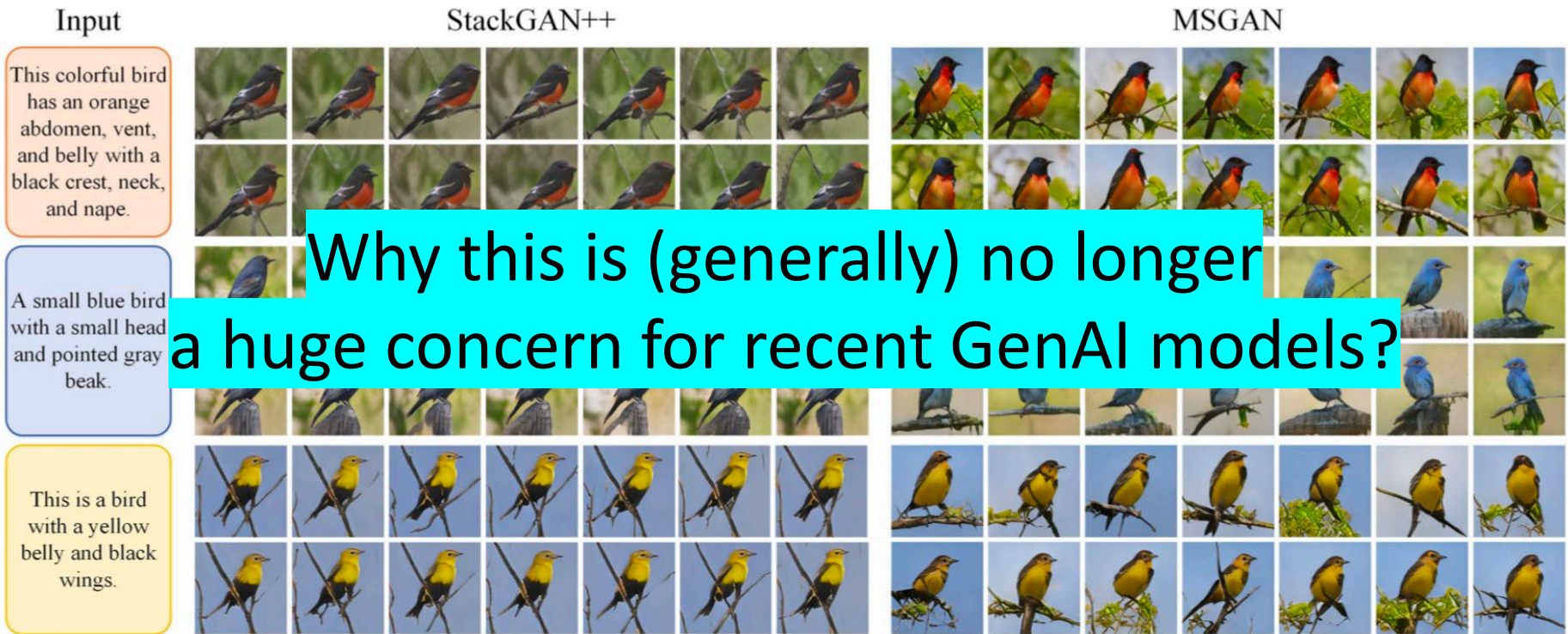
MSGAN (cont'd)

- Qualitative results
 - Conditioned on [paired images](#)



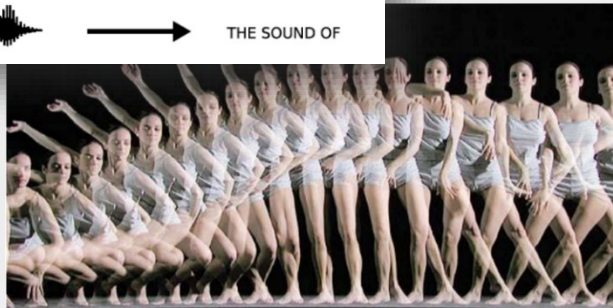
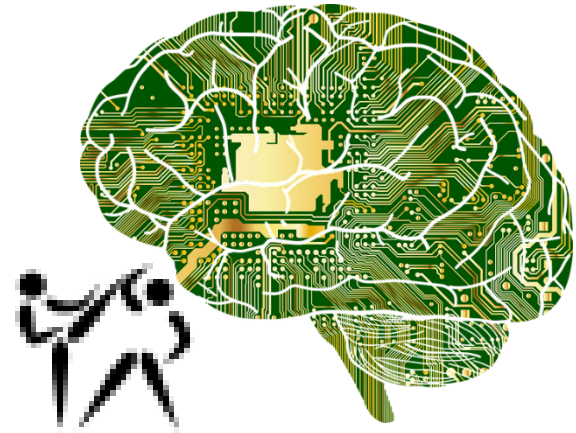
MSGAN (cont'd)

- Qualitative results
 - Conditioned on **text** (will talk about Vision & Language later this semester)



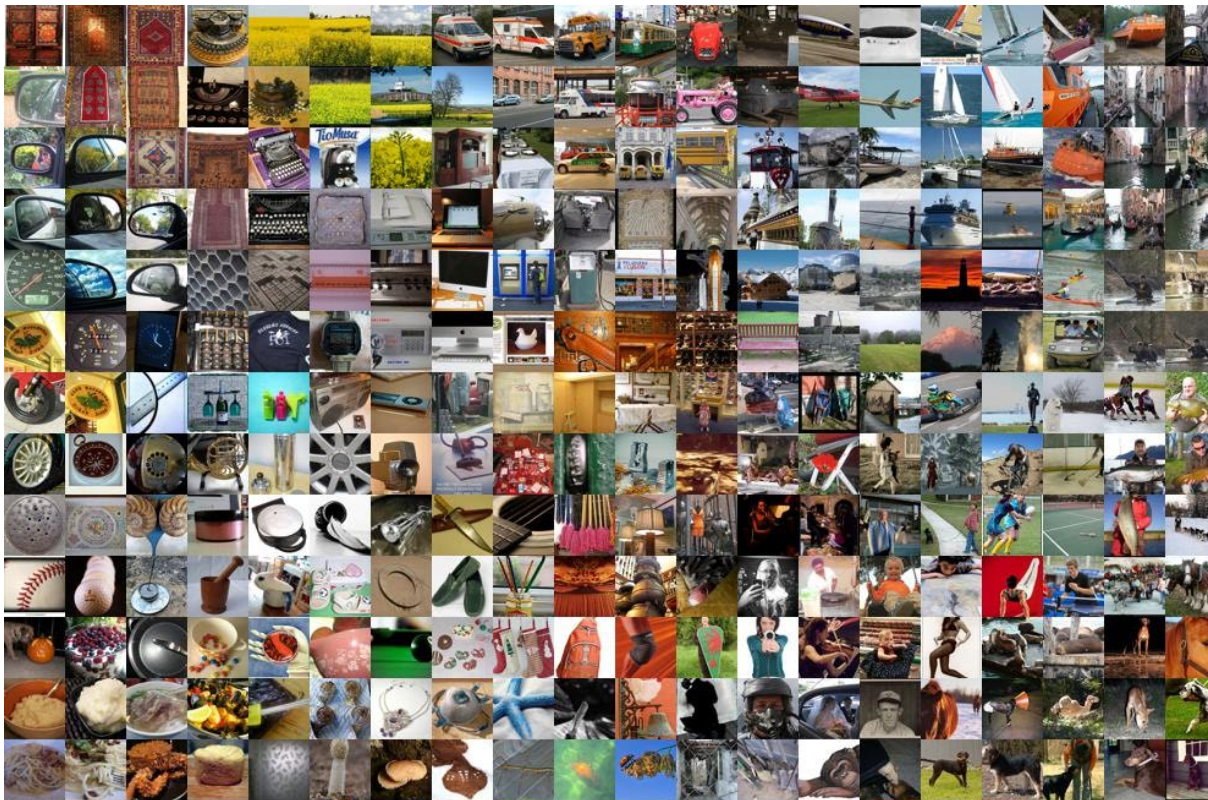
What to Be Covered Today...

- Adversarial Learning for Transfer Learning
- Recurrent Neural Networks
 - From RNN to LSTM & GRU
 - Sequence-to-Sequence Learning
 - Attention in RNN
- Transformer



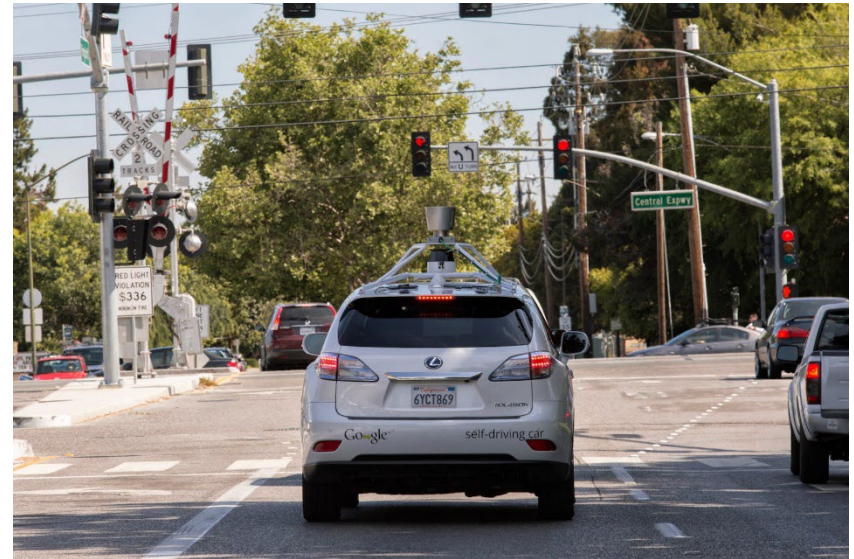
Transfer Learning?

- Machine Learning
 - Collecting/annotating data is typically **expensive**.



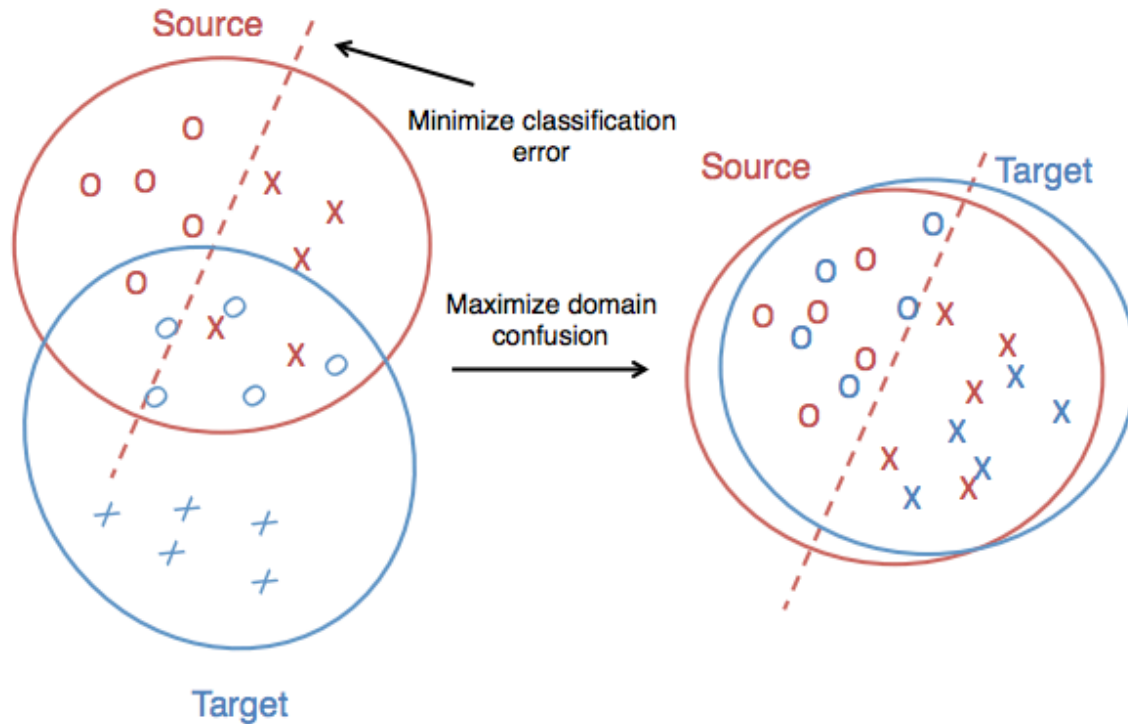
Transfer Learning? (cont'd)

- A practical example

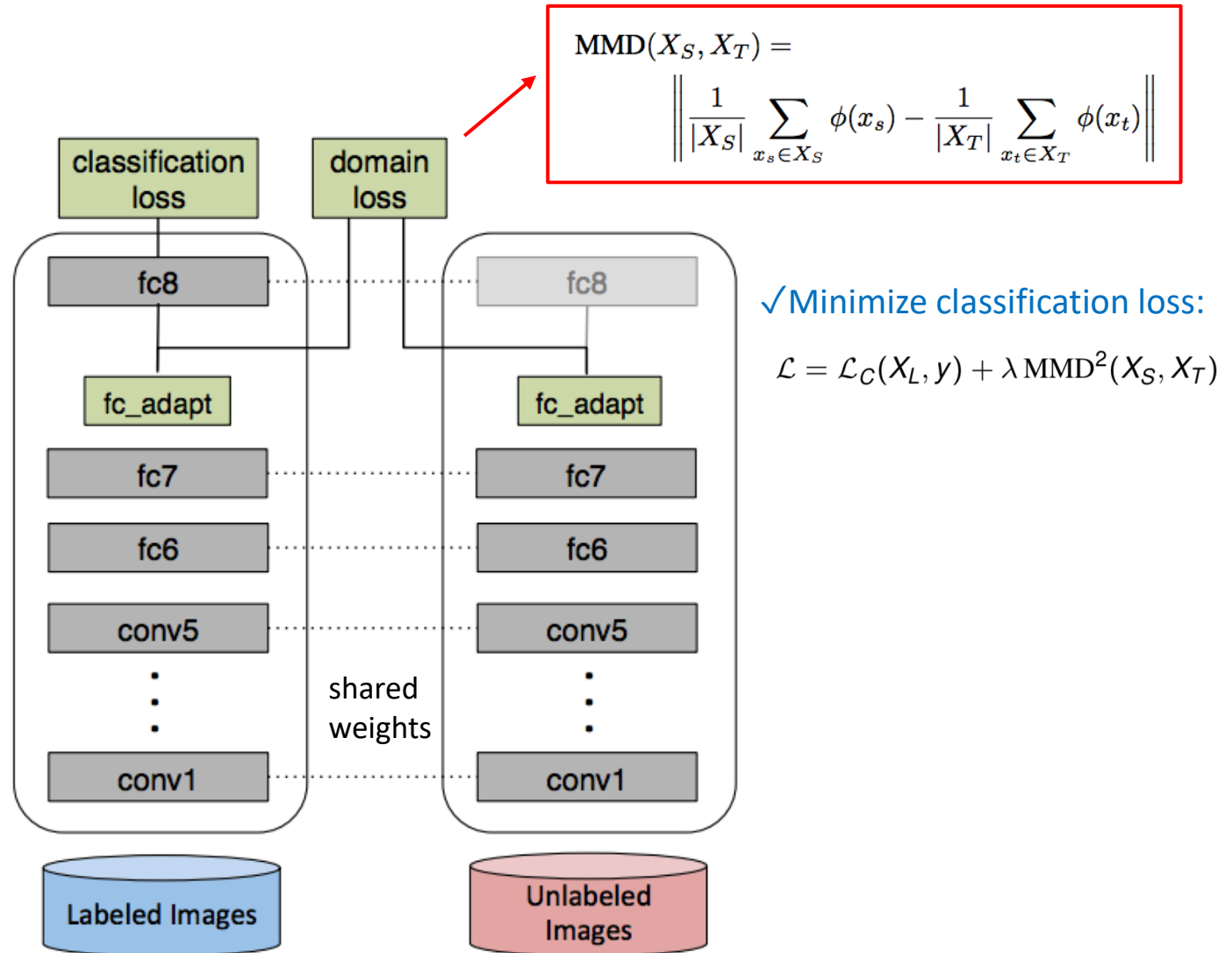


Deep Domain Confusion (DDC)

- Deep Domain Confusion: Maximizing for Domain Invariance
 - Tzeng et al., arXiv: 1412.3474, 2014

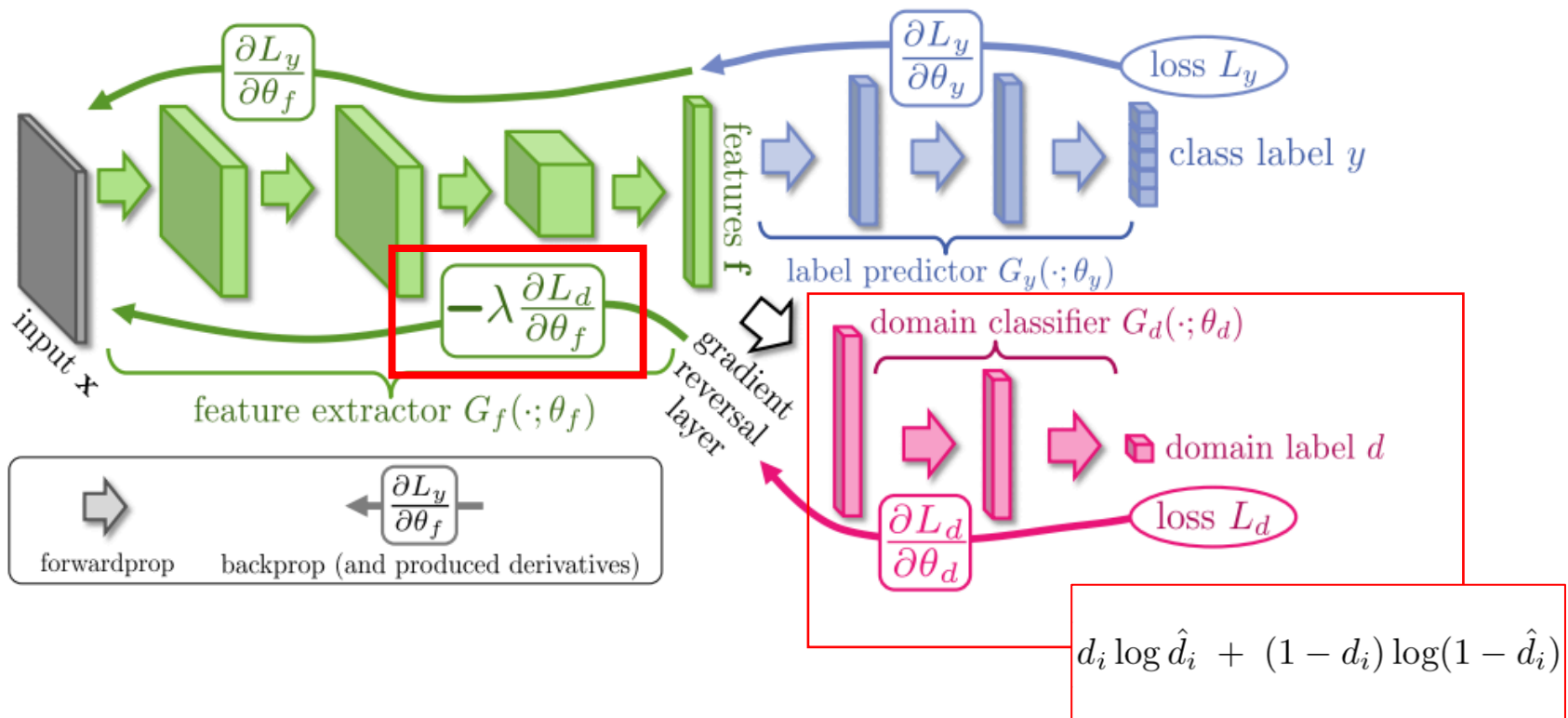


Deep Domain Confusion (DDC)



Domain Confusion by Domain-Adversarial Training

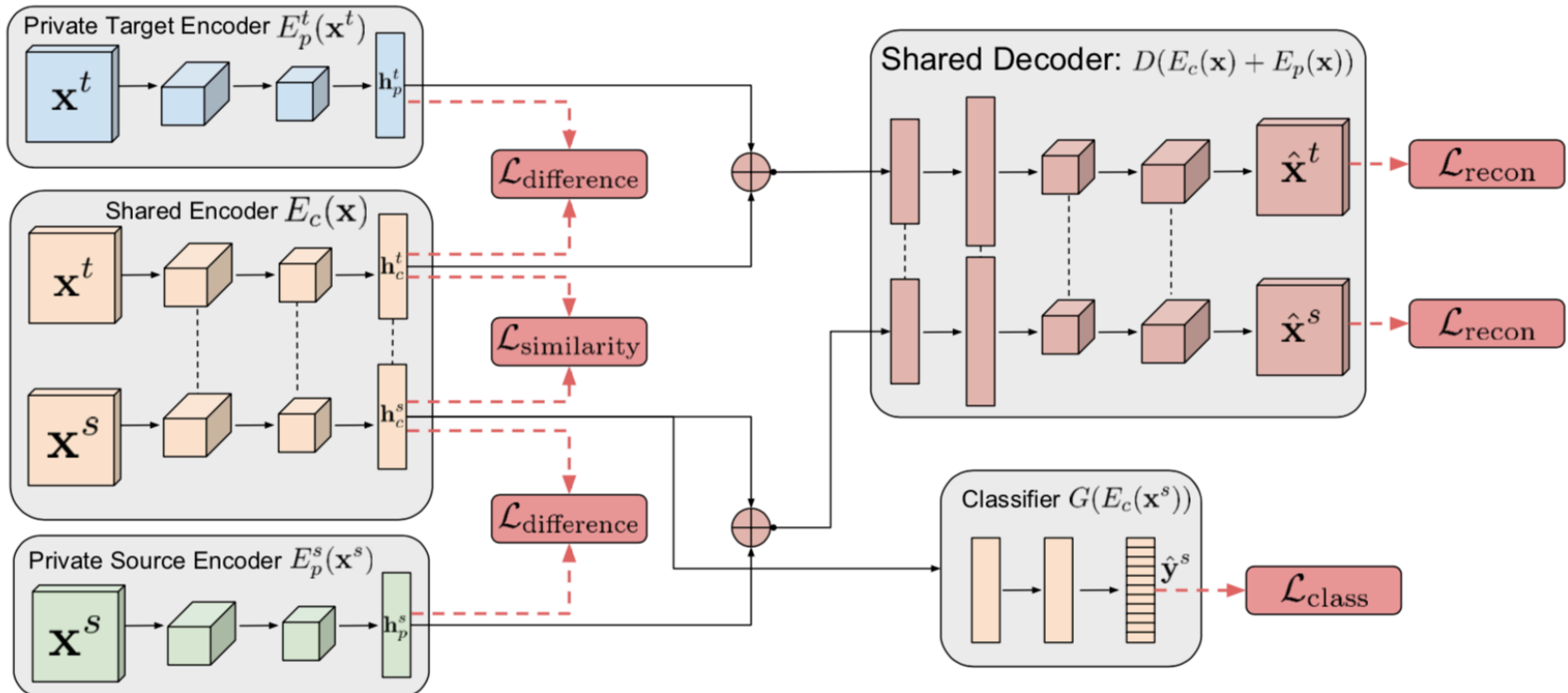
- Domain-Adversarial Training of Neural Networks (DANN)
 - Y. Ganin et al., ICML 2015
 - Maximize domain confusion = maximize domain classification loss
 - Minimize source-domain data classification loss
 - The derived **feature f** can be viewed as a disentangled & domain-invariant feature.



Beyond Domain Confusion

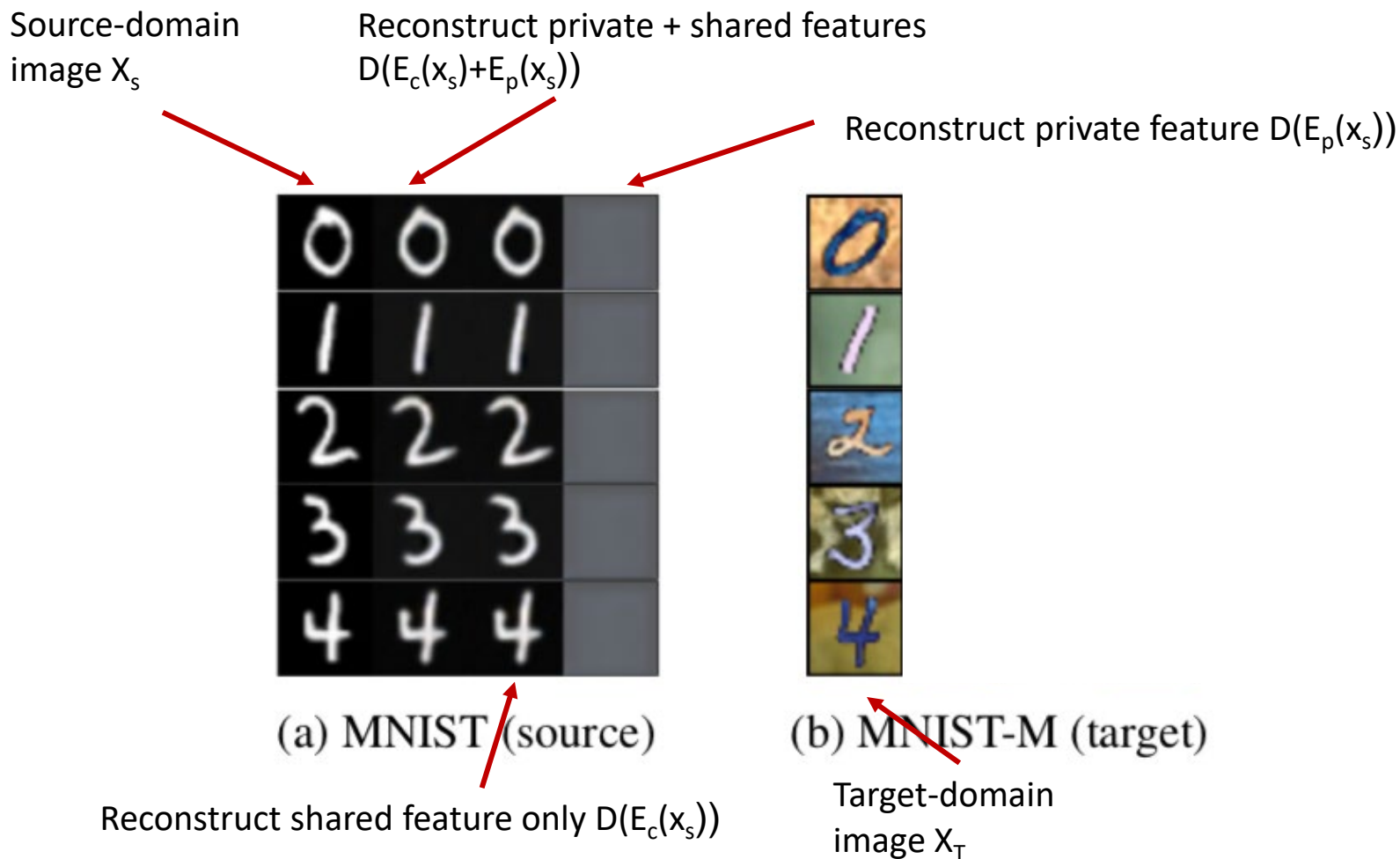
- **Domain Separation Network (DSN)**

- Bousmalis et al., NIPS 2016
- Separate encoders for domain-invariant and domain-specific features
- **Private**/**common** features are *disentangled* from each other.



Beyond Domain Confusion

- Domain Separation Network, NIPS 2016
 - Example results



Beyond Domain Confusion

- Domain Separation Network, NIPS 2016
 - Example results

Source-domain
image X_s



(a) MNIST (source)

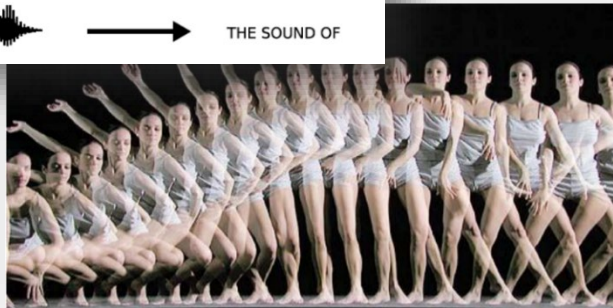
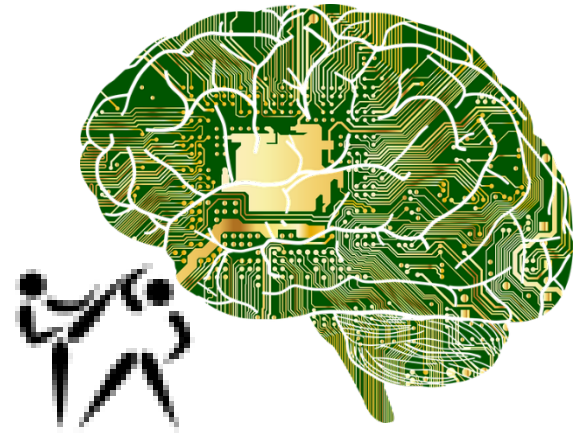


(b) MNIST-M (target)

Target-domain
image X_T

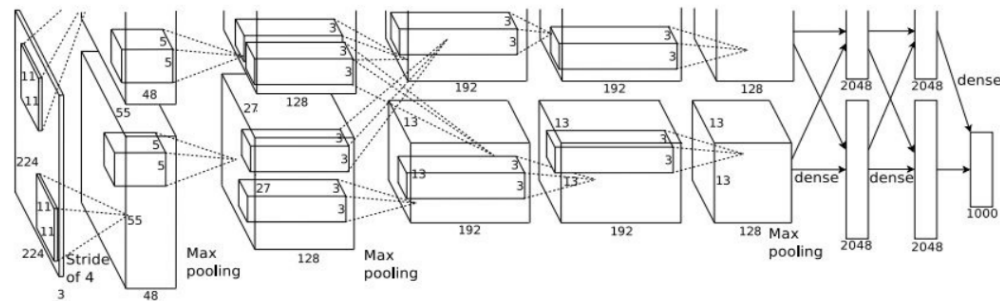
What to Be Covered Today...

- Adversarial Learning for Transfer Learning
- Recurrent Neural Networks
 - From RNN to LSTM & GRU
 - Sequence-to-Sequence Learning
 - Attention in RNN
- Transformer



What Are The Limitations of CNN?

- In terms of the input...



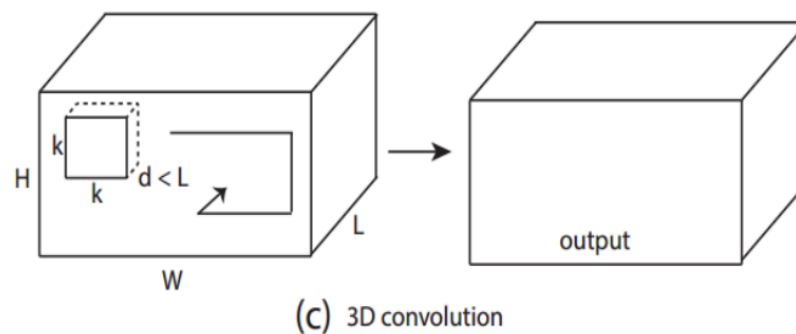
DOG

CAT

MONKEY

What Are The Limitations of CNN? (cont'd)

- In terms of the input...
- Any way to handle data beyond 2D via CNN?



3D convolution

What Are The Limitations of CNN? (cont'd)

- In terms of the output...
 - A simple feed-forward process
 - E.g., image captioning

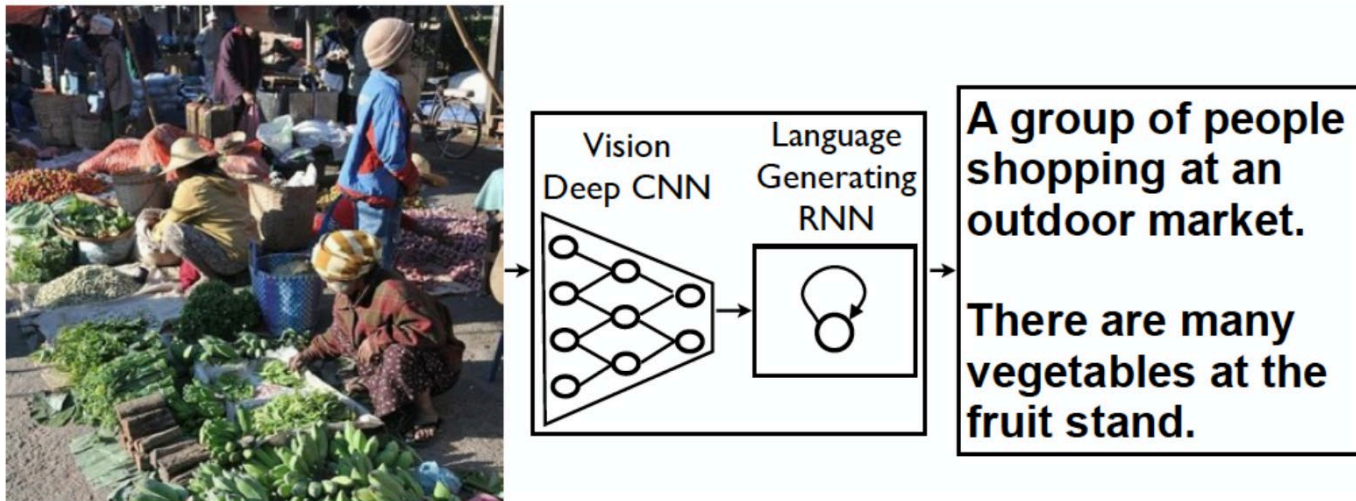
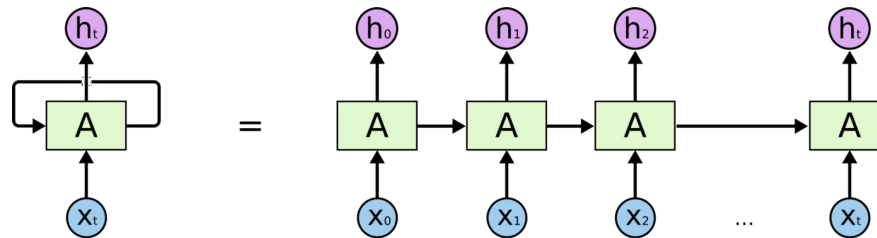


Figure from Vinyals et al, "Show and tell: A neural image caption generator", CVPR 2015

How to Model Sequential Data?

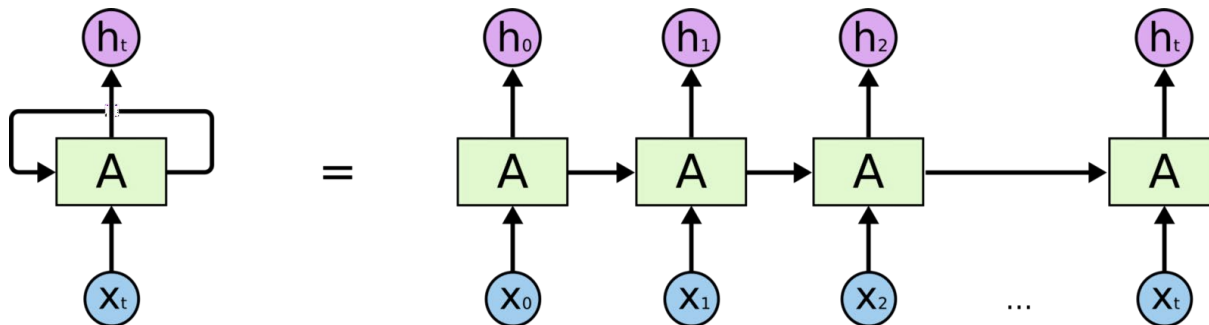
- Deep learning for sequential data
 - Possible solution: 3D convolution neural networks
 - Recurrent neural networks (RNN)



RNN

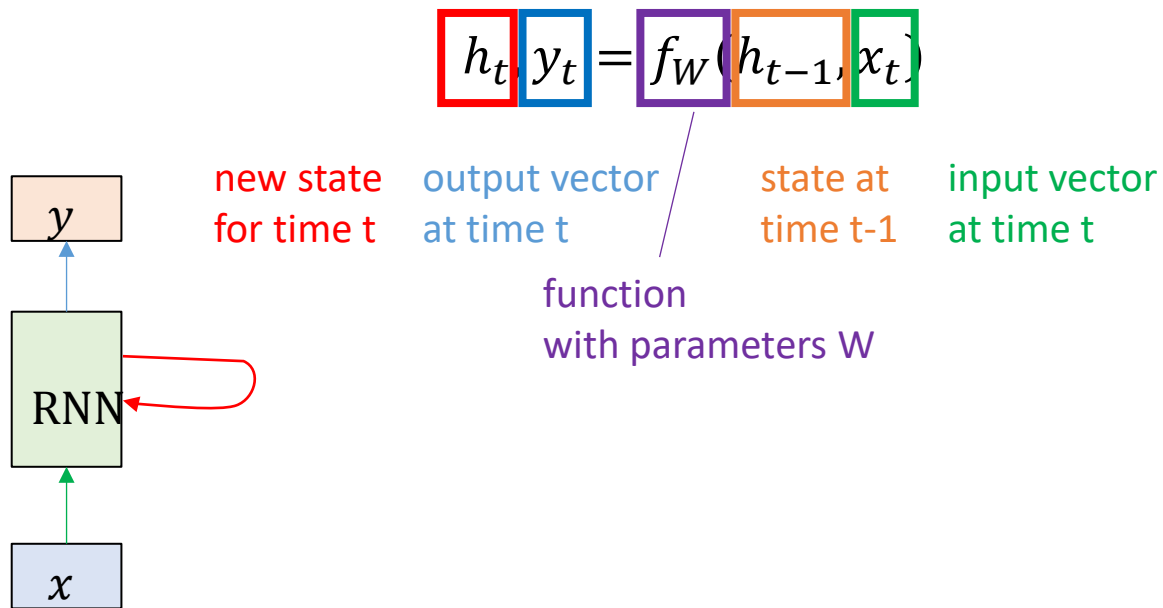
Recurrent Neural Networks

- Parameter sharing + unrolling
 - Keeps the number of parameters fixed
 - Allows sequential data with varying lengths
- Memory ability
 - Capture and preserve information which has been extracted/processed



Recurrence Formula

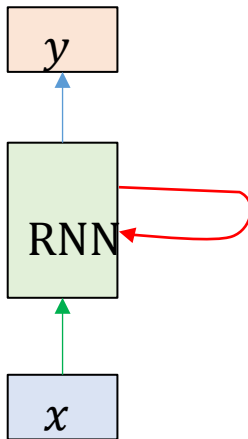
- Same function and parameters used at every time step:



Recurrence Formula

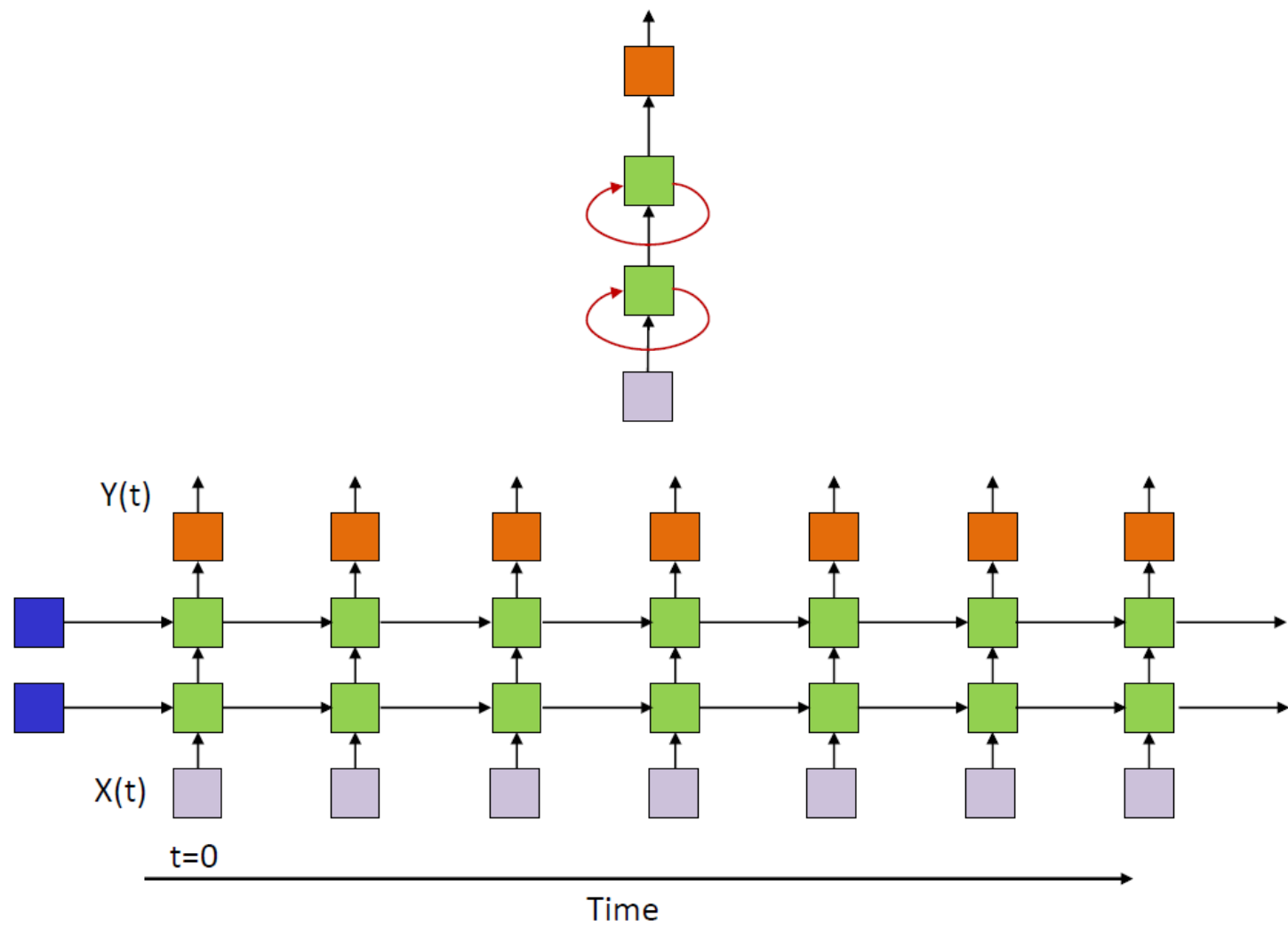
- Same function and parameters used at every time step:

$$h_t, y_t = f_W(h_{t-1}, x_t)$$

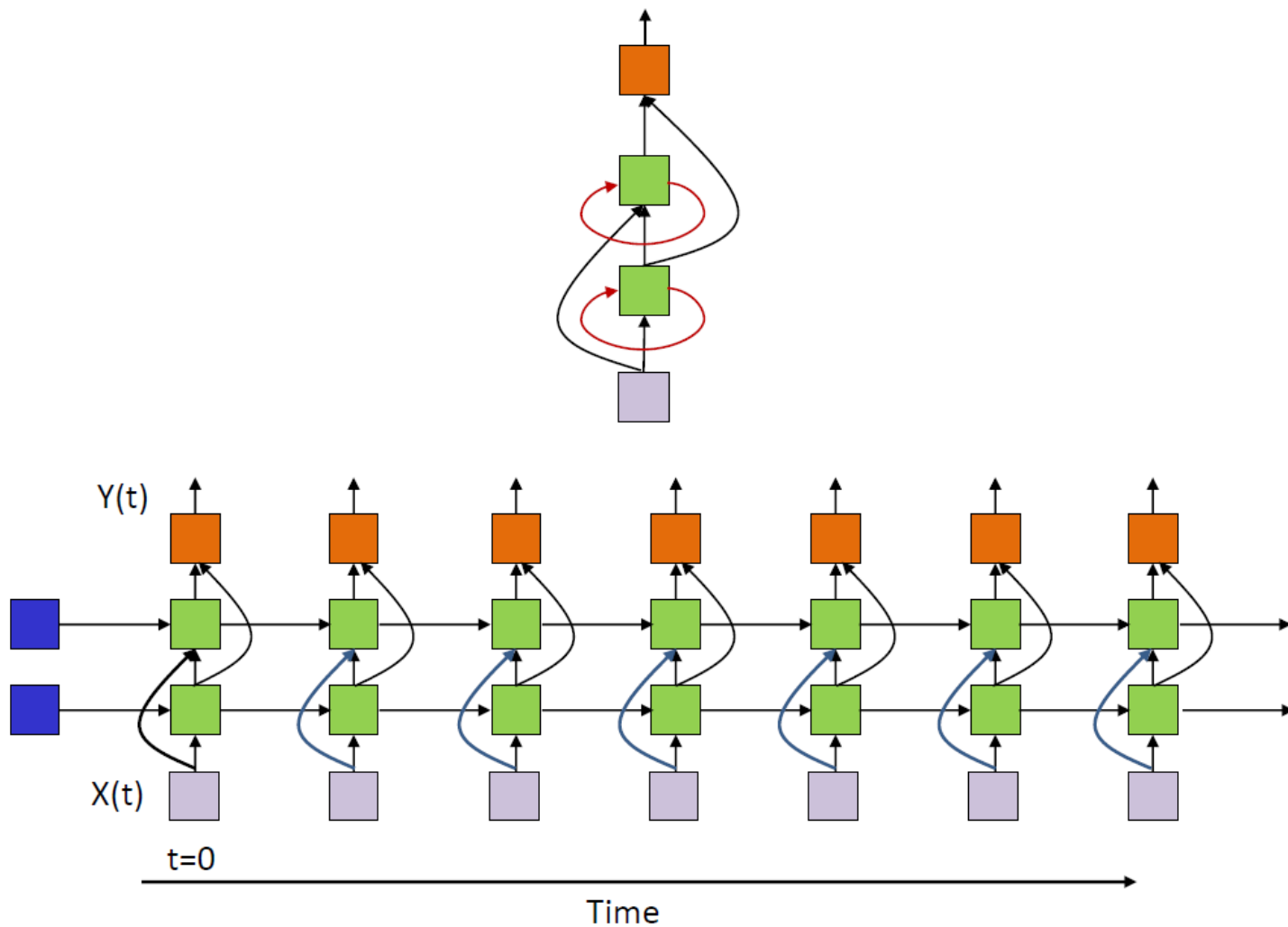


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$y_t = W_{hy}h_t$$

Multiple Recurrent Layers

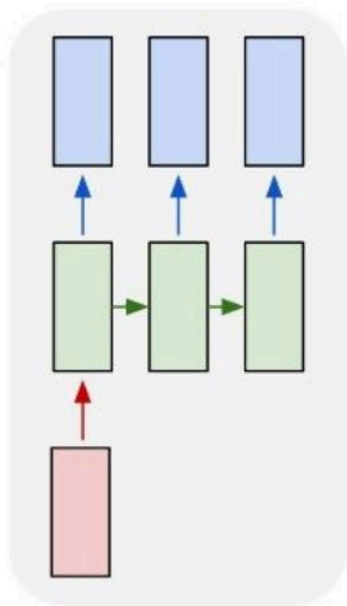


Multiple Recurrent Layers



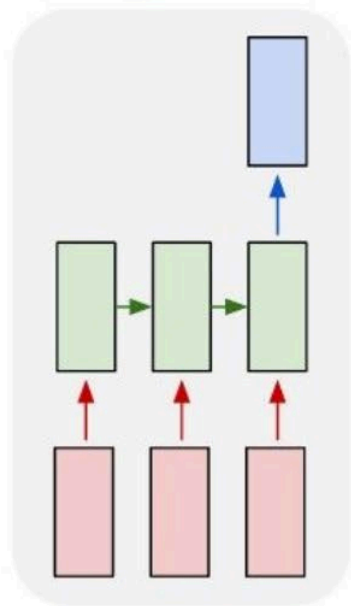
Sequence-to-Sequence Modeling

one to many



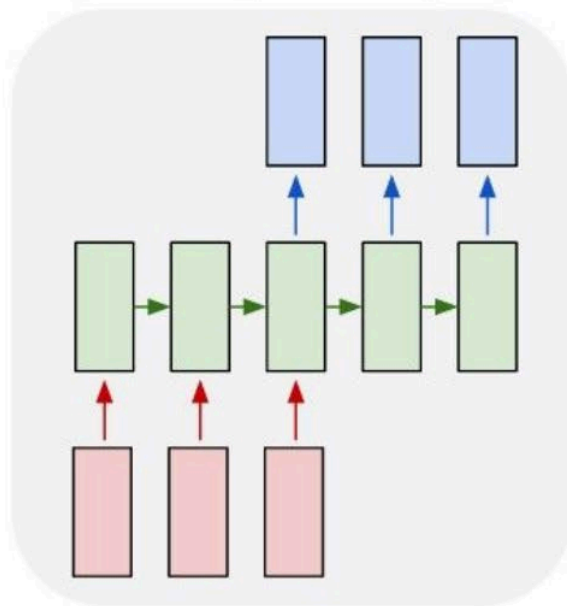
e.g., image caption

many to one



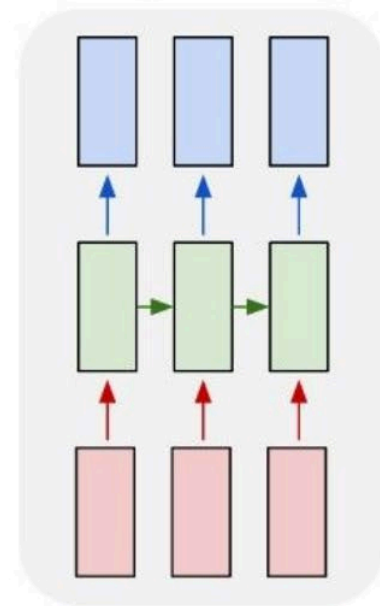
e.g., action recognition

many to many



e.g., video prediction

many to many

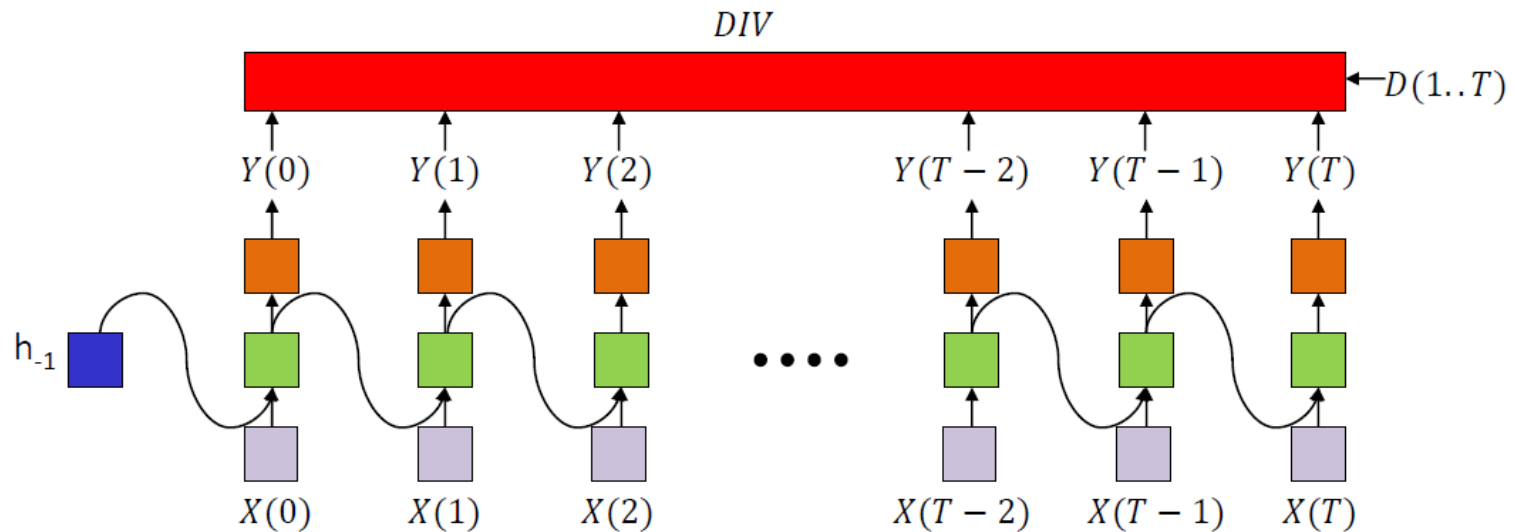


e.g., video indexing

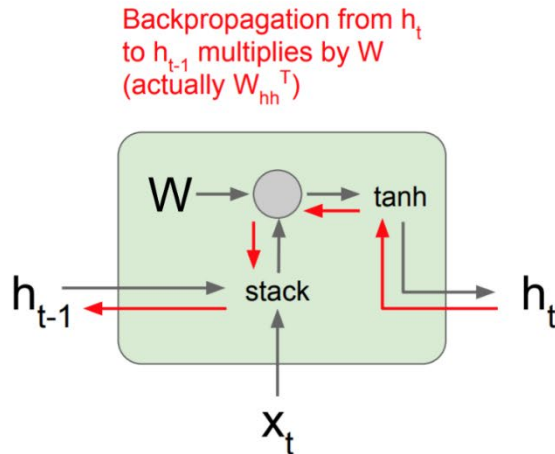
Training RNNs:

Back Propagation Through Time

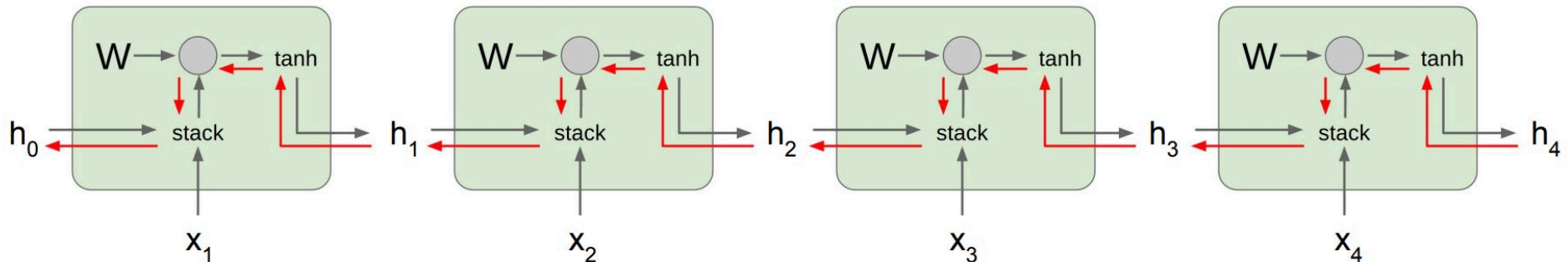
- Let's focus on one training instance.
- The divergence to be computed is between the sequence of outputs by the network and the desired output sequence.
- Generally, this is not just the sum of the divergences at individual times.



Back Propagation Through Time (BPTT)



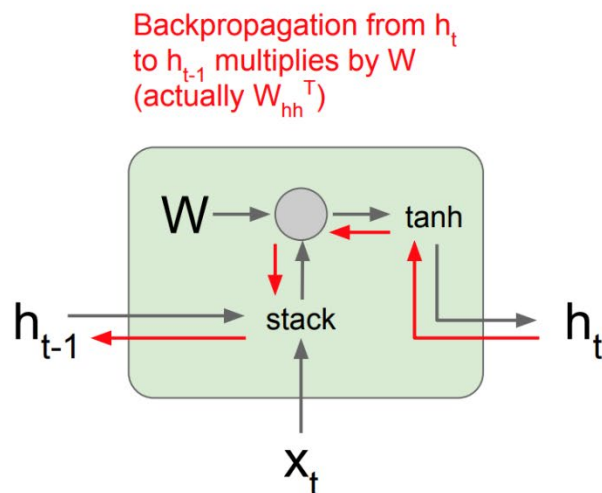
$$\begin{aligned}
 h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\
 &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\
 &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)
 \end{aligned}$$



Computing gradient of h_0 involves many factors of W (and repeated $\tanh$)

Gradient Vanishing & Exploding

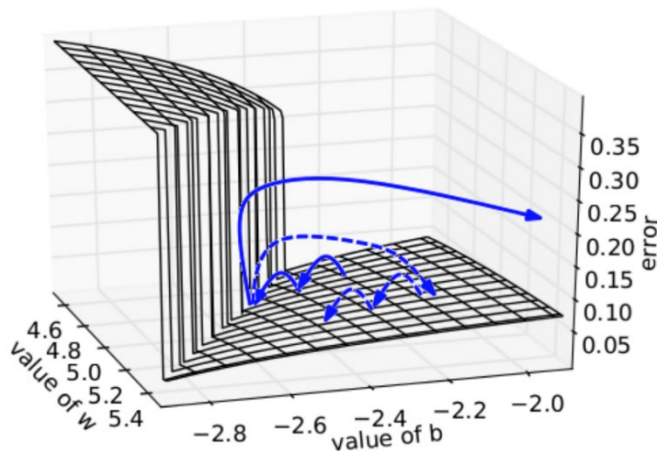
- Practical concerns...
 - Exploding gradients : Largest singular value > 1
 - Vanishing gradients : Largest singular value < 1



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Solutions...

- Gradients clipping: rescale gradients if too large



→ standard gradient descent trajectories

---→ gradient clipping to fix problem

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

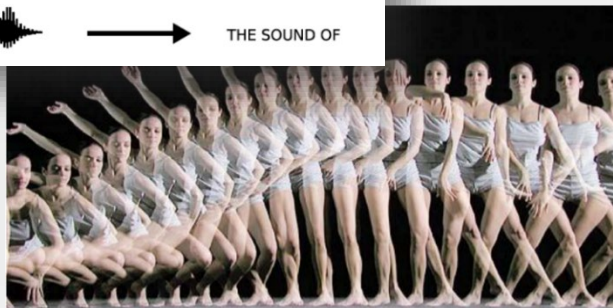
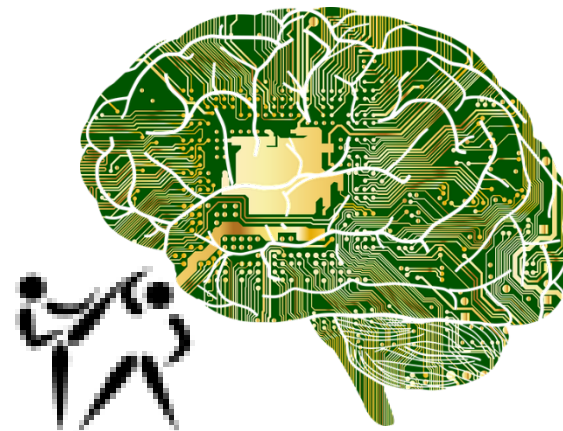
- How about vanishing gradients?
 - Change RNN architecture!

Variants of RNN (not covered)

- Long Short-term Memory (LSTM) [Hochreiter et al., 1997]
 - Additional memory cell
 - Input/Forget/Output Gates
 - Handle **gradient vanishing**
 - Learn long-term dependencies
- Gated Recurrent Unit (GRU) [Cho et al., EMNLP 2014]
 - Similar to LSTM
 - handle gradient vanishing & learn long-term dependencies
 - No additional memory cell
 - Reset / Update Gates
 - **Fewer parameters than LSTM**
 - Comparable performance to LSTM [Chung et al., NIPS Workshop 2014]

What to Be Covered Today...

- Adversarial Learning for Transfer Learning
- Recurrent Neural Networks
 - From RNN to LSTM & GRU
 - Sequence-to-Sequence Learning
 - Attention in RNN
- Transformer

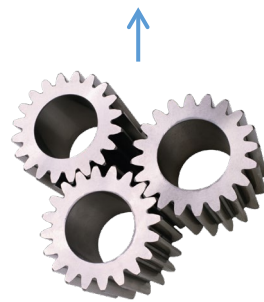


Sequence-to-Sequence Modeling

- Setting
 - An input sequence $\mathbf{X}_1, \dots, \mathbf{X}_N$
 - An output sequence $\mathbf{Y}_1, \dots, \mathbf{Y}_M$
 - Generally $N \neq M$, i.e., **no synchrony** between $\mathbf{X}$ and $\mathbf{Y}$
- Examples
 - *Speech recognition*: speech goes in, and a word sequence comes out
 - *Machine translation*: word sequence goes in, and another comes out
 - *Video captioning*: video goes in, word sequence comes out



深度學習好棒棒!

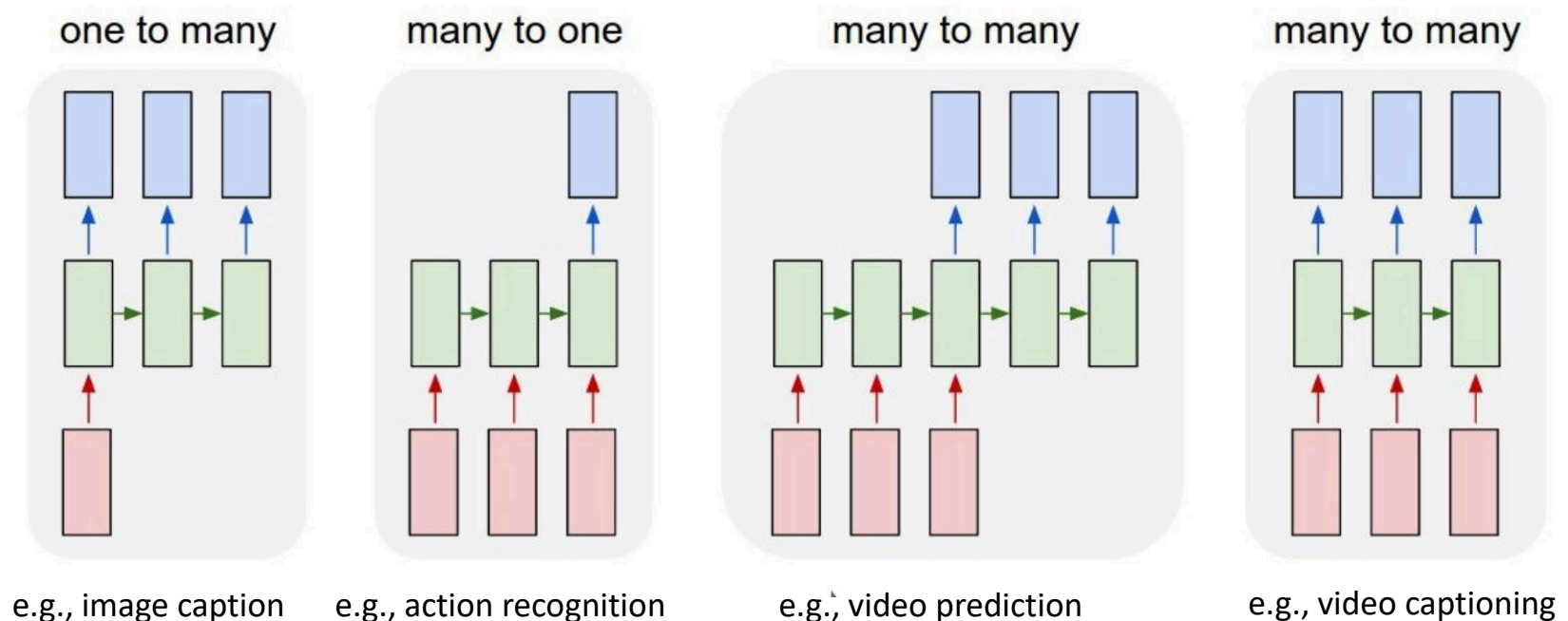


Deep learning rocks!



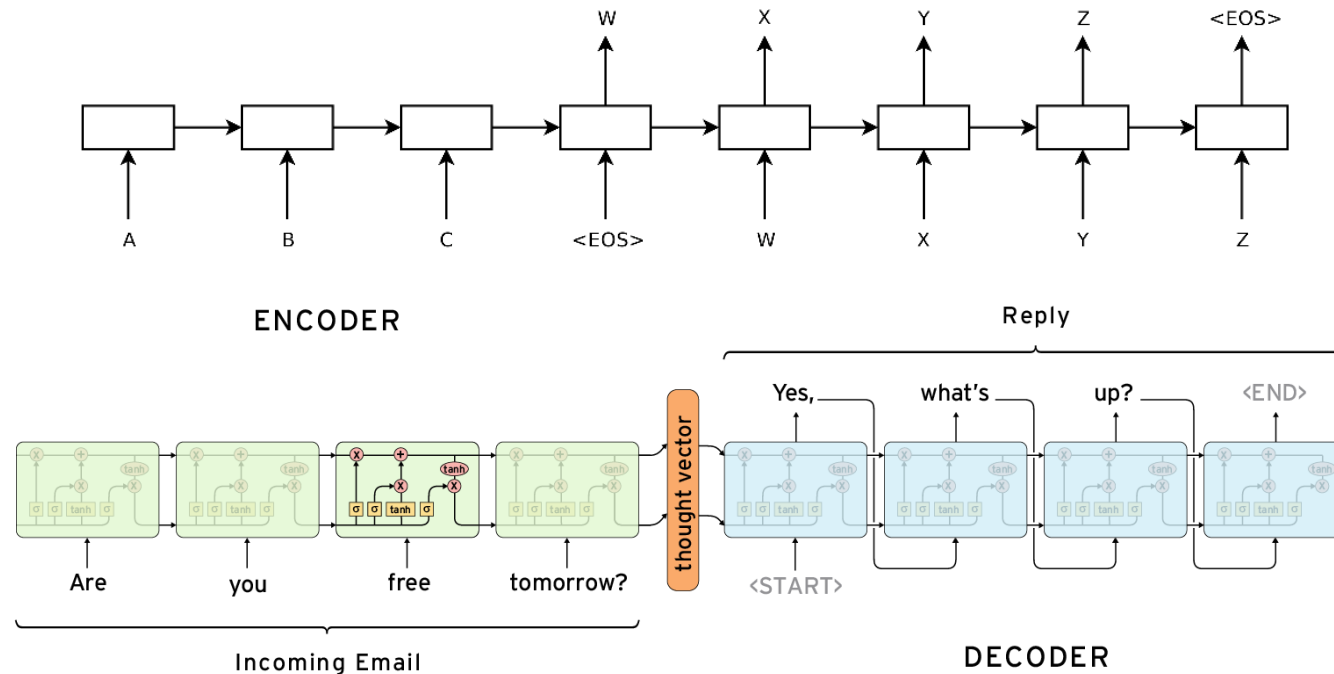
S-to-S Models with Alignment

- The input and output sequences happen in the same order
 - The input/output sequences may be asynchronous.
 - E.g., speech recognition or video captioning, in which the input sequence corresponds to the phoneme/caption sequence out.
 - Recall that...



Sequence-to-Sequence Modeling (cont'd)

- Original model proposed in NIPS 2014
 - An encoder-decoder model



Example of Seq-to-Seq Modeling: Image Captioning

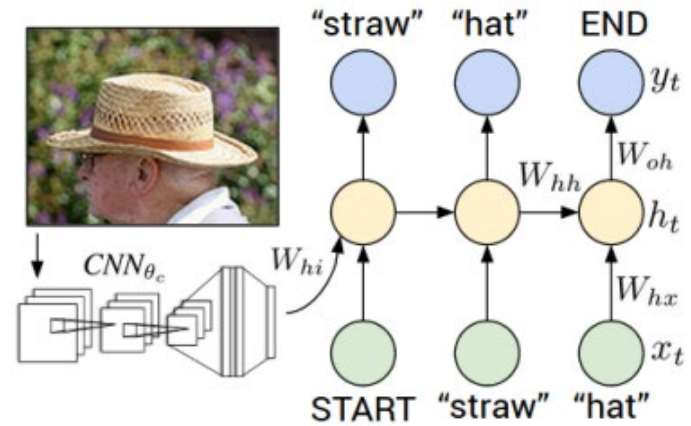
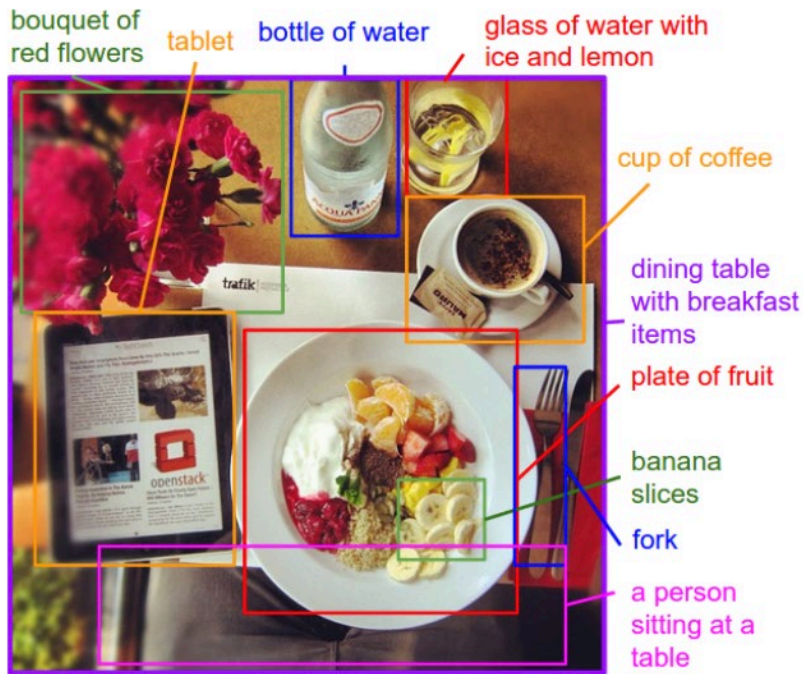


Figure from Karpathy et al, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

FC-4096

FC-1000

softmax

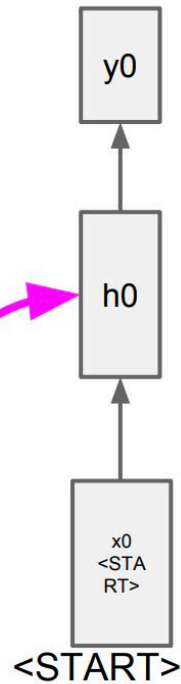
CNN



test image



V



test image

before:

$$h = \tanh(W_{xh} * x + W_{hh} * h)$$

now:

$$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$$

image

conv-64

conv-64

maxpool

conv-128

conv-128

maxpool

conv-256

conv-256

maxpool

conv-512

conv-512

maxpool

conv-512

conv-512

maxpool

FC-4096

FC-4096



test image

y0

h0

x0
<START>

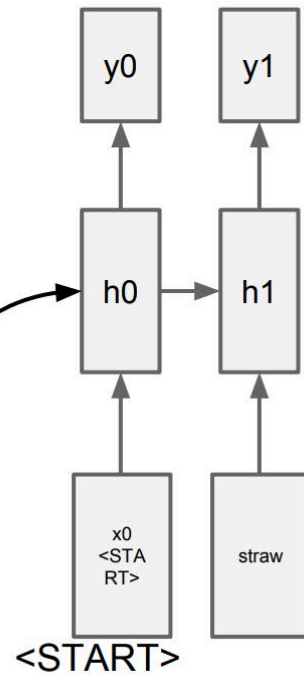
straw

sample!

<START>



test image



RNN

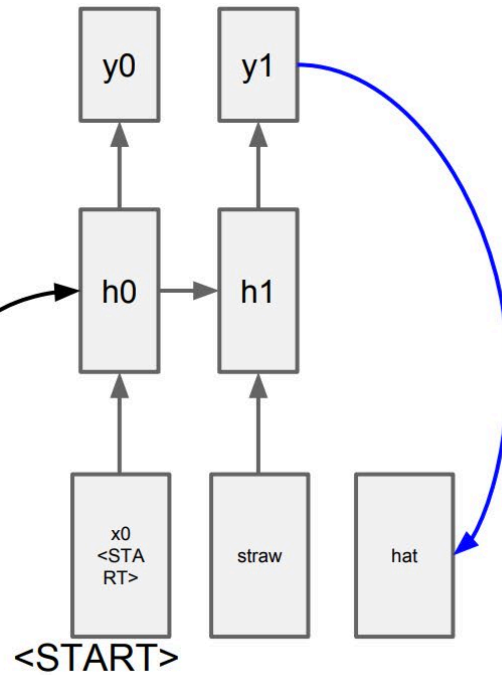
$$h_t = \tanh \left(W \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \right)$$

Output in time t

Input in time t



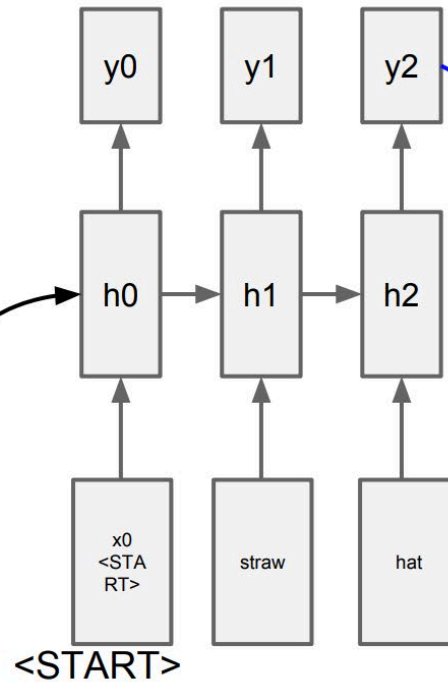
test image



sample!



test image



sample
<END> token
=> finish.

RNN

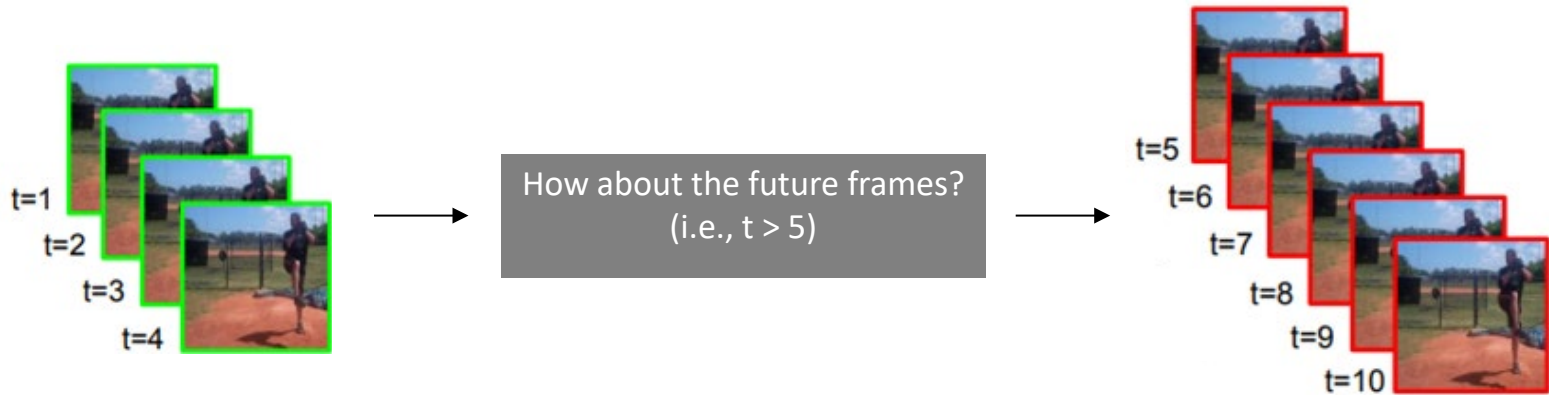
$$h_t = \tanh \left(W \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \right)$$

Output in time t

Input in time t

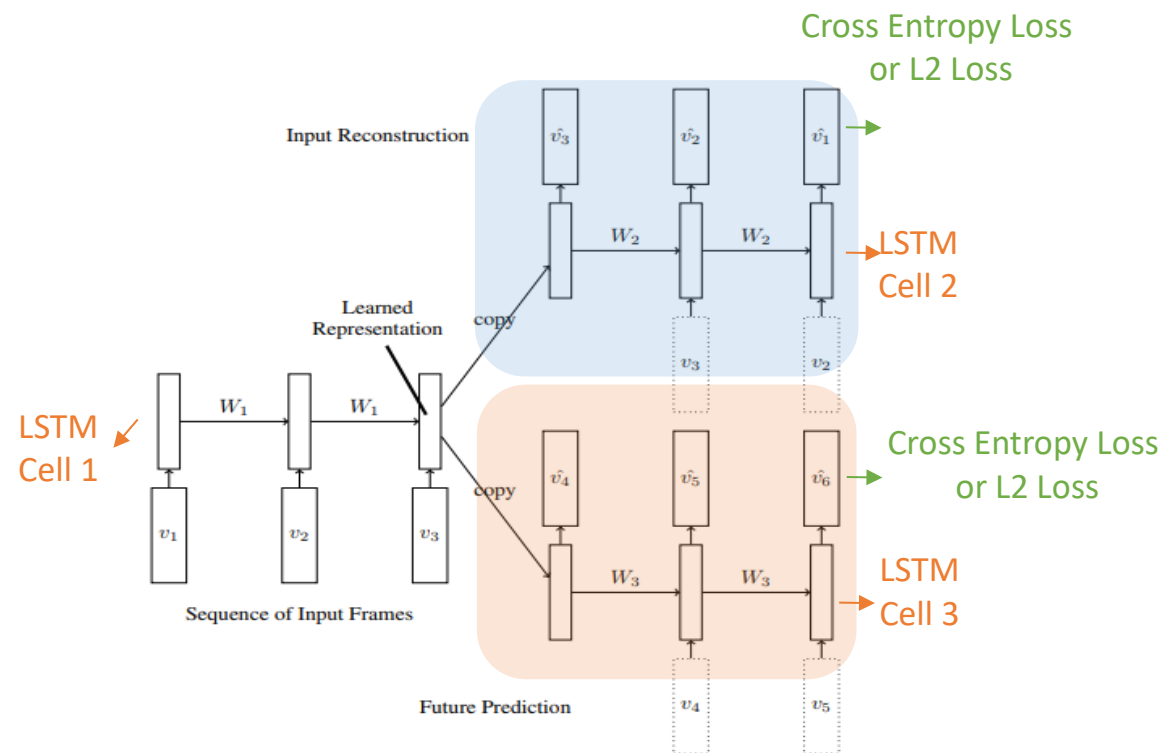
Example of Seq-to-Seq Modeling: Video Prediction

- Input: A few **known** frames
- Output: **Unknown** future frames

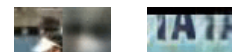


• Unsupervised Learning of Video Representations using LSTMs

(Srivastava et al., ICML'15)

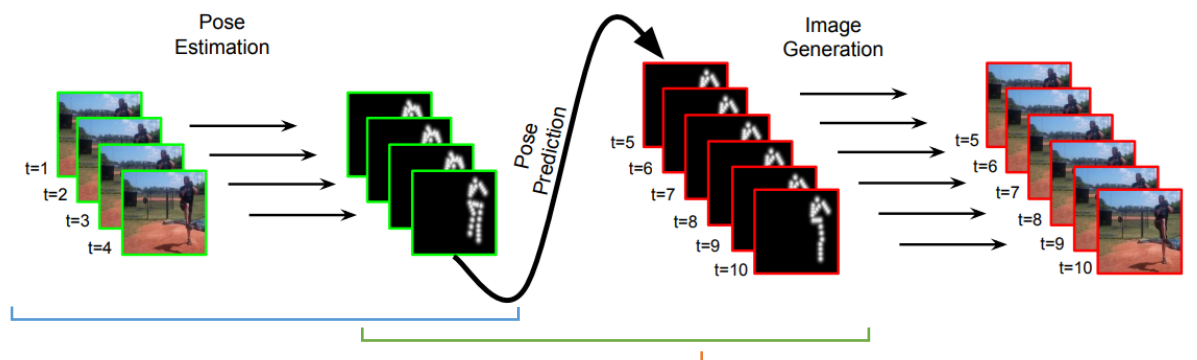


- LSTM Encoder-Decoder model
- Two tasks: Reconstruction & Prediction
- Results (L: ref video, R: output video)
 - Bouncing (Moving) MNIST
- Video patches of UCF-101



• Learning to Generate Long-Term Future via Hierarchical Prediction

(Villegas et al., ICML'17)



Stage 1:

Pose Estimation

Hourglass network (Newell et al., ECCV'16)

Stage 2:

Future Pose Prediction

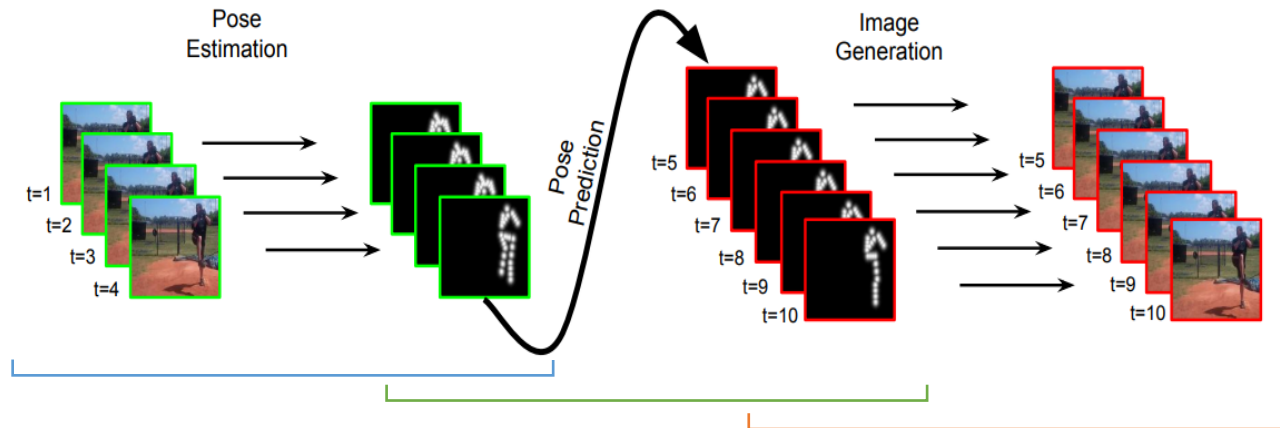
Sequence-to-Sequence model on high-level structure

Stage 3:

Image Generation

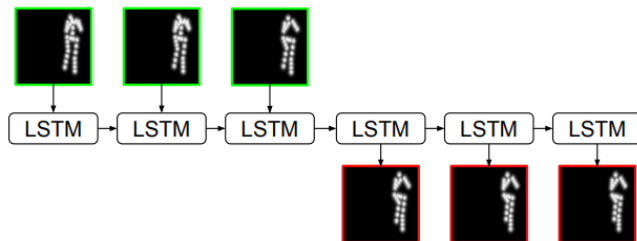
Visual-Structure Analogy

- Learning to generate long-term future via hierarchical prediction (Villegas et al., ICML'17)



Step 2:

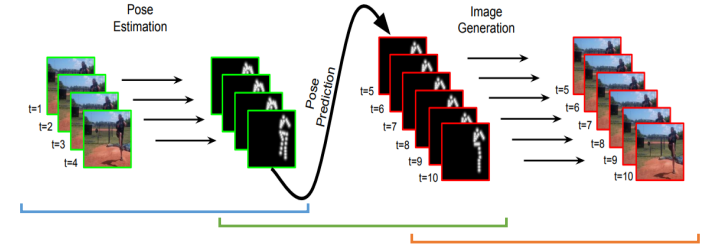
Future Pose Prediction



Sequence-to-Sequence model on high-level structure

Objective Function:

$$\mathcal{L}_{\text{pose}} = \frac{1}{TL} \sum_{t=1}^T \sum_{l=1}^L \mathbb{1}_{\{m_{k+t}^l=1\}} \|\hat{\mathbf{p}}_{k+t}^l - \mathbf{p}_{k+t}^l\|_2^2$$



- Learning to generate long-term future via hierarchical prediction (Villegas et al., ICML'17)

Step 3:

Image Generation



Visual-Structure Analogy

Objective Function:

Adversarial Training -> alternately minimize L & L_{Disc}

Update Image Generation Network (G)

$$\mathcal{L} = \mathcal{L}_{\text{img}} + \mathcal{L}_{\text{feat}} + \mathcal{L}_{\text{Gen}}$$

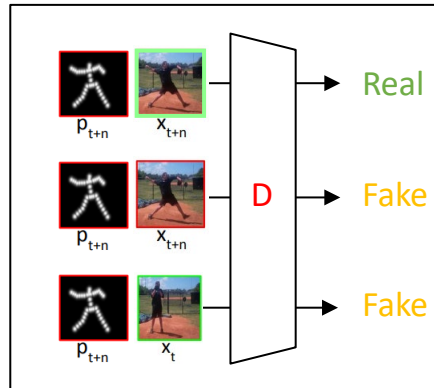
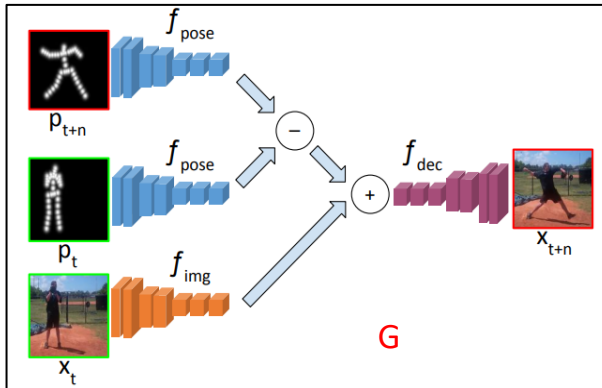
$$\mathcal{L}_{\text{img}} = \|\mathbf{x}_{t+n} - \hat{\mathbf{x}}_{t+n}\|_2^2$$

$$\mathcal{L}_{\text{feat}} = \|C_1(\mathbf{x}_{t+n}) - C_1(\hat{\mathbf{x}}_{t+n})\|_2^2 + \|C_2(\mathbf{x}_{t+n}) - C_2(\hat{\mathbf{x}}_{t+n})\|_2^2$$

$$\mathcal{L}_{\text{Gen}} = -\log D([\mathbf{p}_{t+n}, \hat{\mathbf{x}}_{t+n}])$$

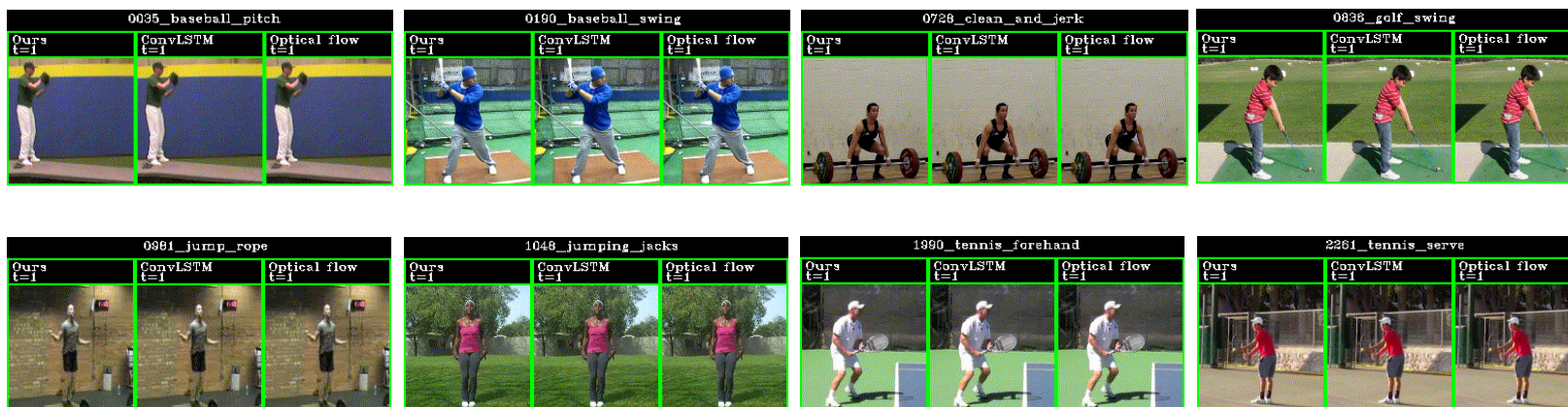
Update Discriminator (D)

$$\begin{aligned} \mathcal{L}_{\text{Disc}} = & -\log D([\mathbf{p}_{t+n}, \mathbf{x}_{t+n}]) \\ & - 0.5 \log(1 - D([\mathbf{p}_{t+n}, \hat{\mathbf{x}}_{t+n}])) \\ & - 0.5 \log(1 - D([\mathbf{p}_{t+n}, \mathbf{x}_t])), \end{aligned}$$



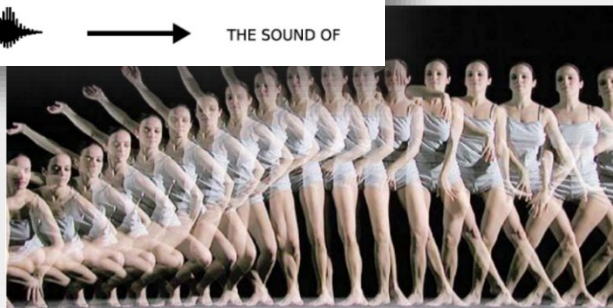
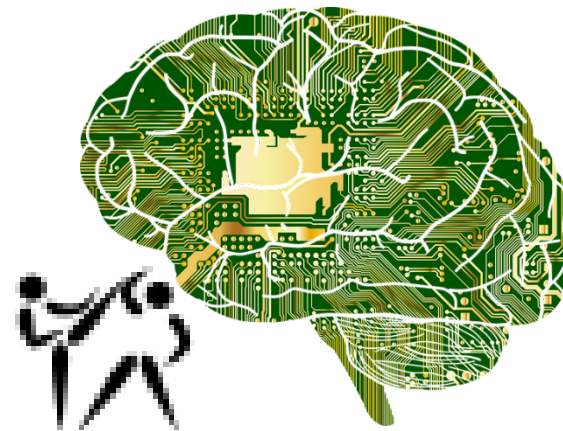
- Example Results

Results on Penn Action Dataset:



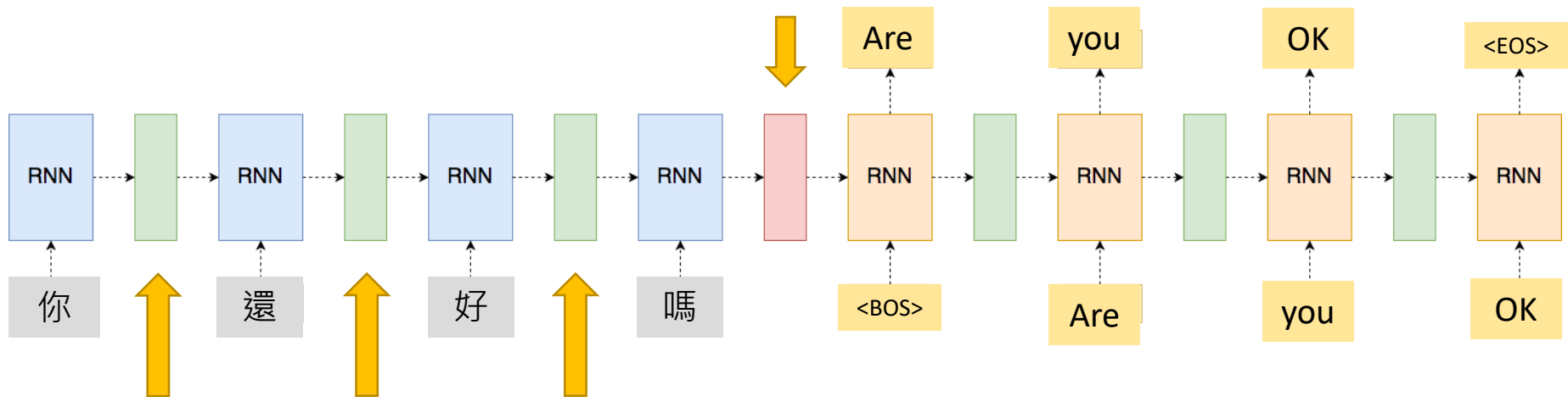
What to Be Covered Today...

- Adversarial Learning for Transfer Learning
- Recurrent Neural Networks
 - From RNN to LSTM & GRU
 - Sequence-to-Sequence Learning
 - Attention in RNN
- Transformer



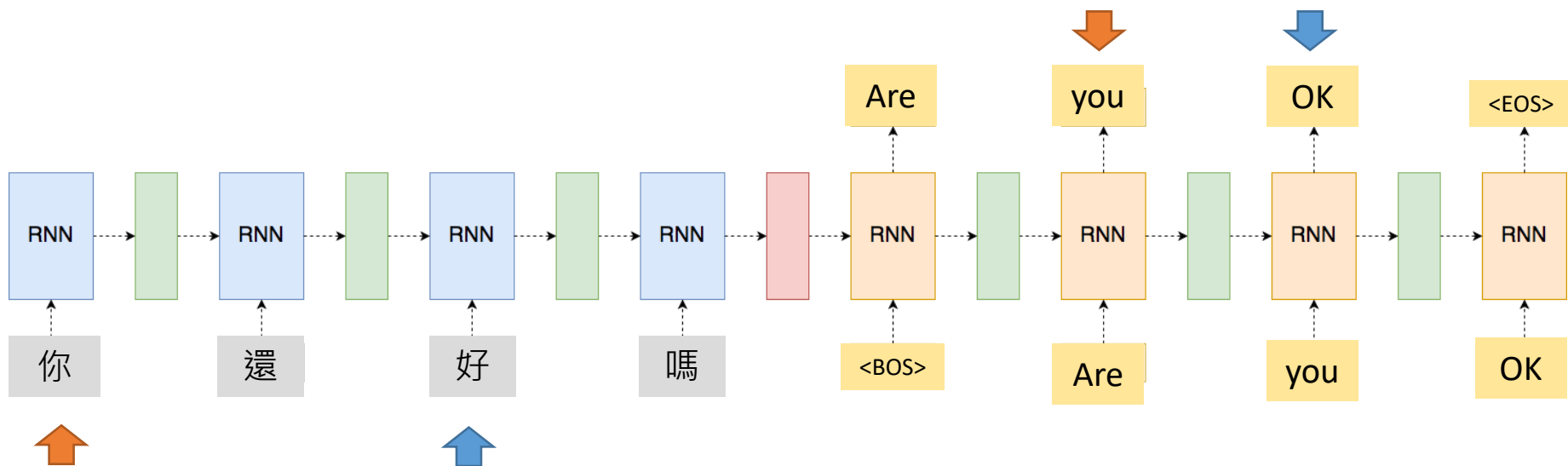
What's the Potential Problem in RNN?

- Each hidden state vector extracts/carries information across time steps (some might be diluted downstream).
- Information of the entire input sequence is embedded into a **single hidden state** vector.



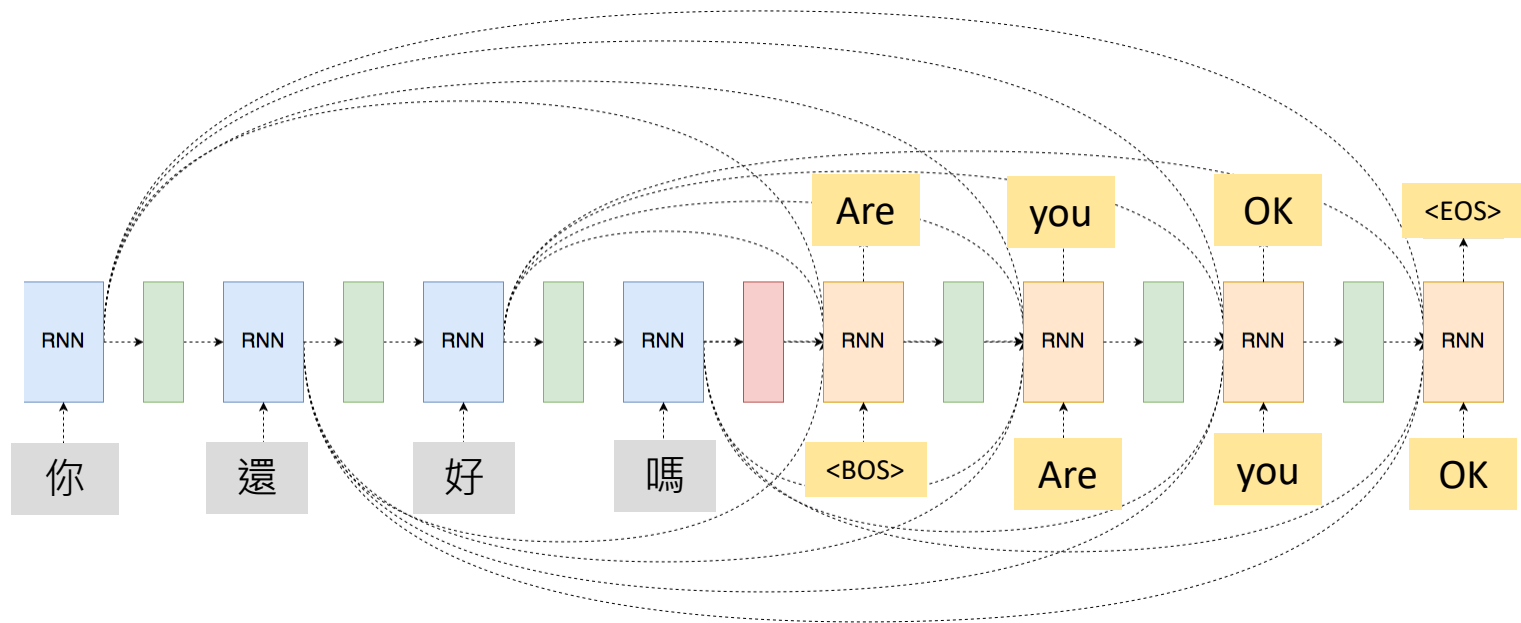
What's the Potential Problem? (cont'd)

- Outputs at different time steps have particular meanings.
- However, **synchrony** between **input** and **output seqs** is **not** required.



What's the Potential Problem? (cont'd)

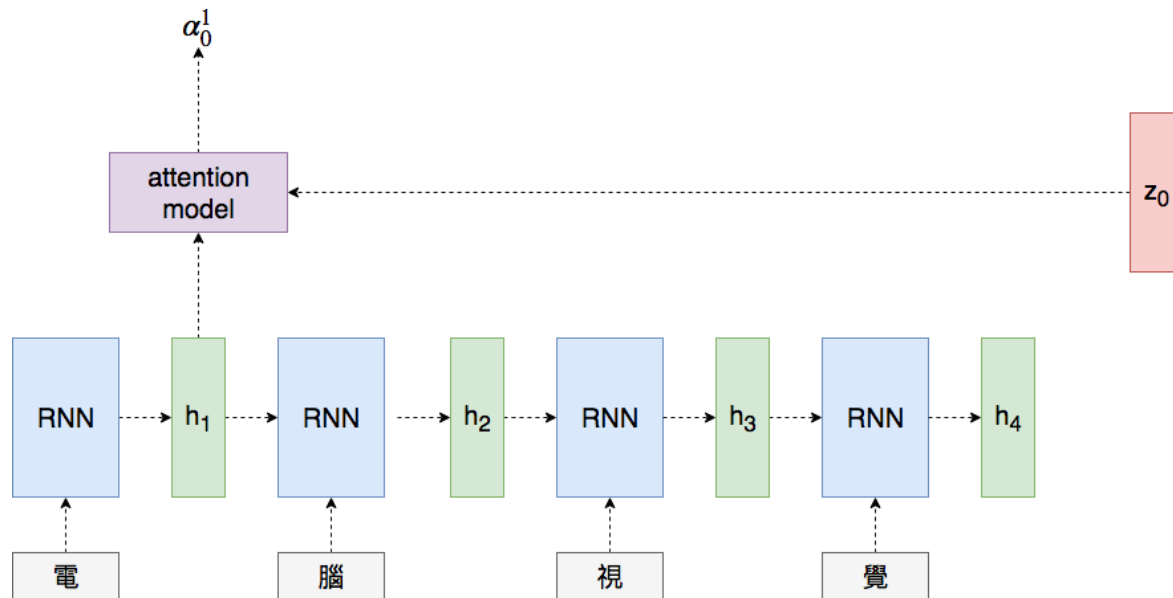
- Connecting every hidden state between encoder and decoder?



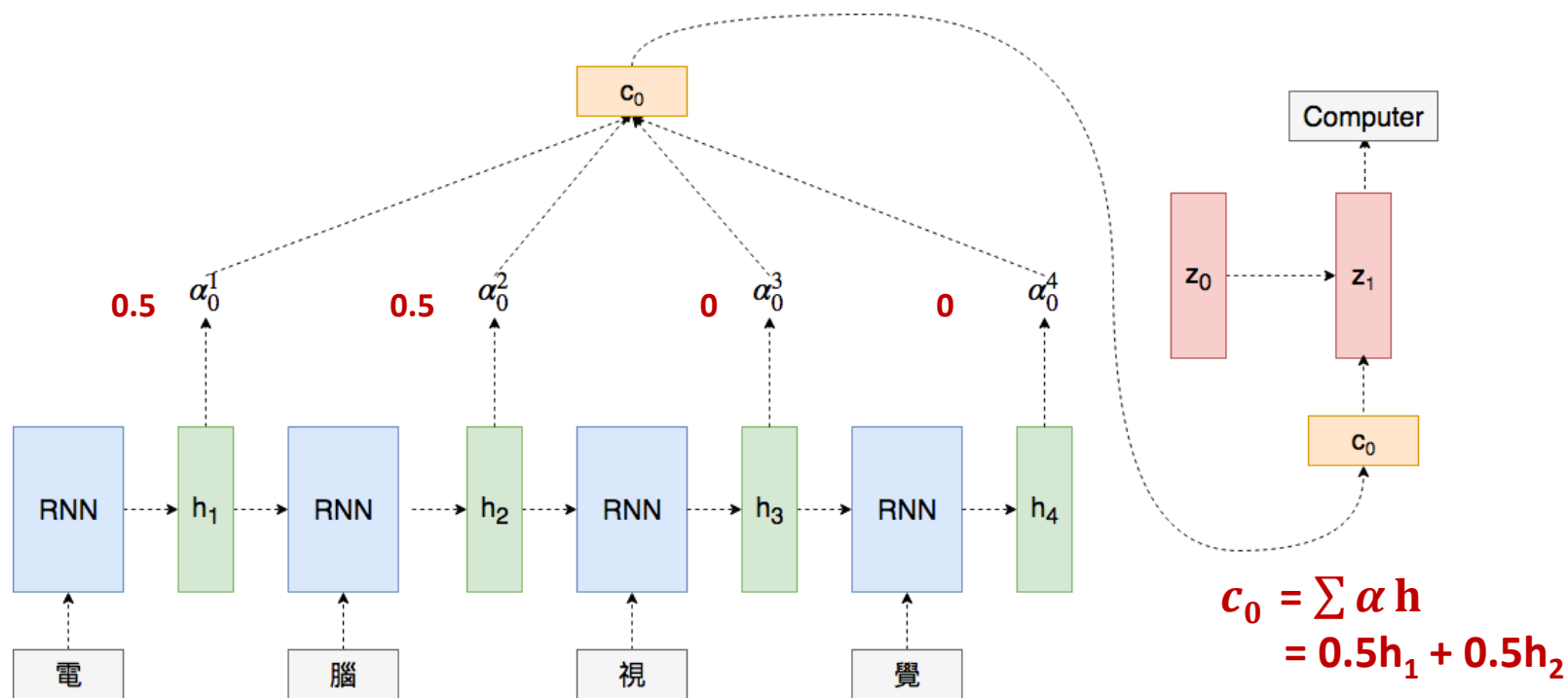
- Infeasible!
 - Both inputs and outputs are with varying sizes.
 - Overparameterized!
 - Possible solution: **attention**

Solution #1: Attention Model

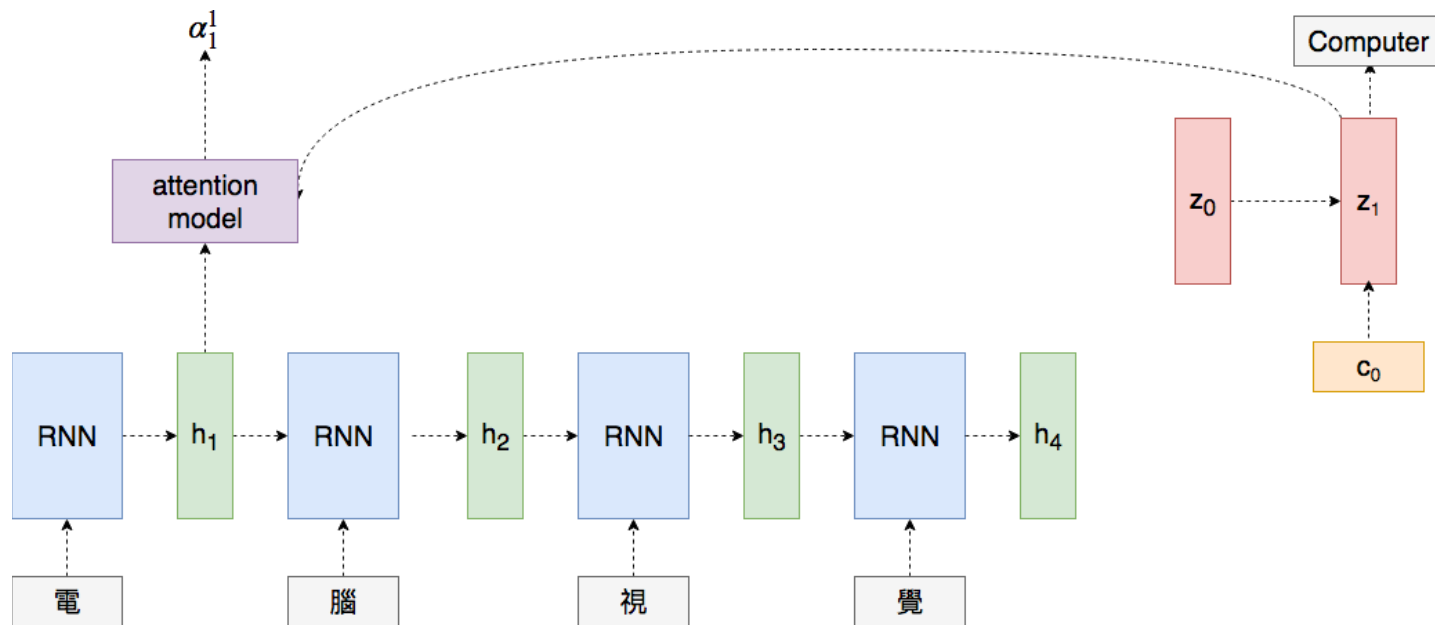
- What should the attention model be?
 - A NN whose inputs are $\mathbf{z}$ and $\mathbf{h}$ while output is a *scalar*, indicating the **similarity** between $\mathbf{z}$ and $\mathbf{h}$.
- Most attention models are jointly learned with other parts of network (e.g., recognition, etc.)



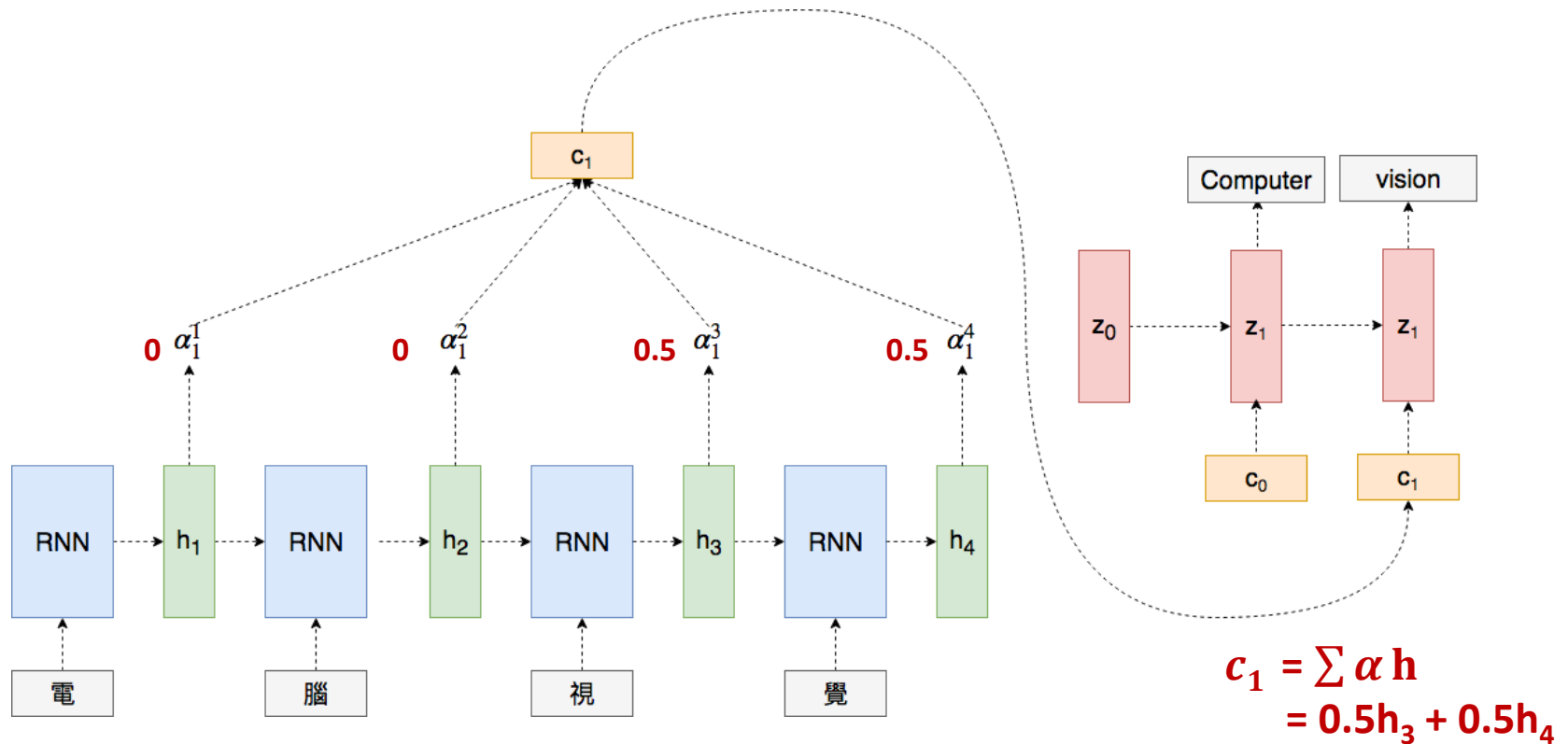
Solution: Attention Model



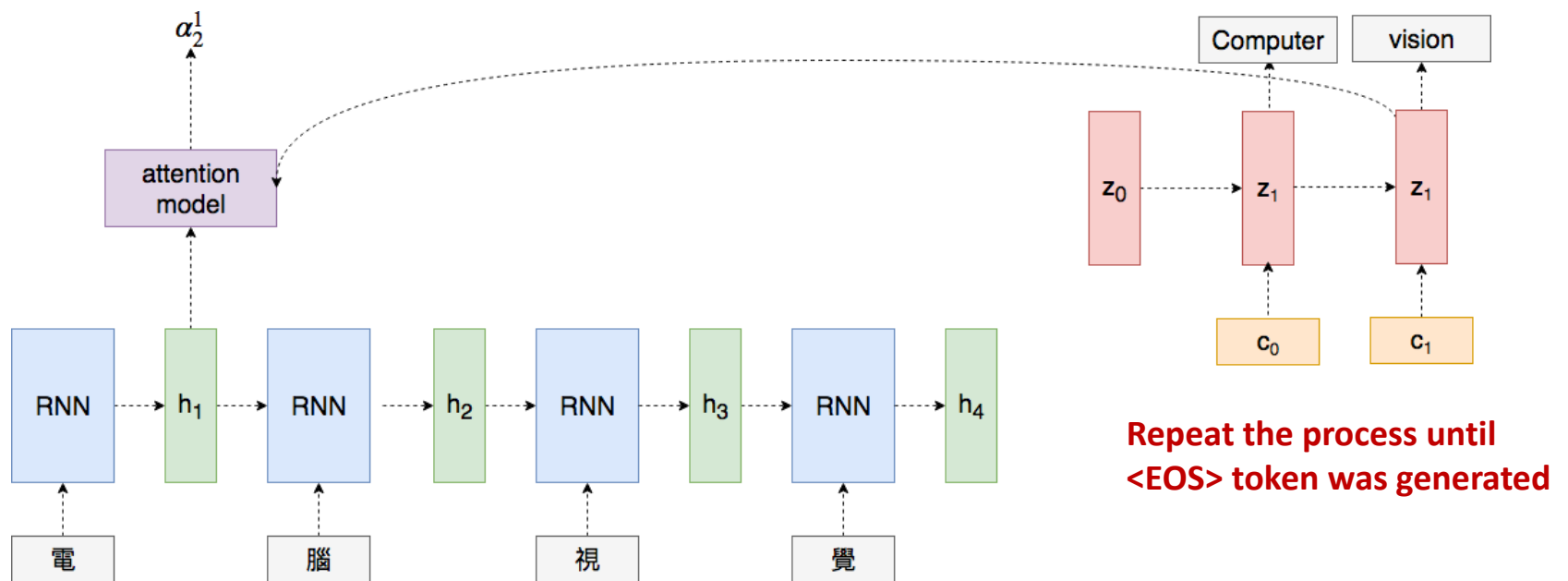
Solution: Attention Model



Solution: Attention Model



Solution: Attention Model



Why *Attention* is Preferable for Image Classification too?



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



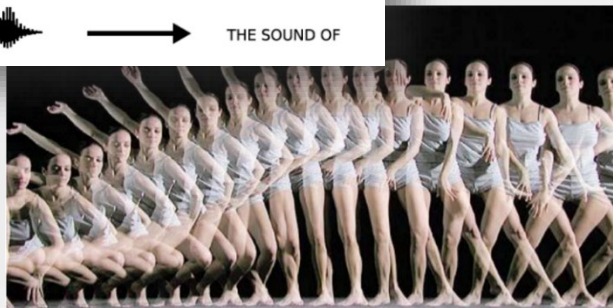
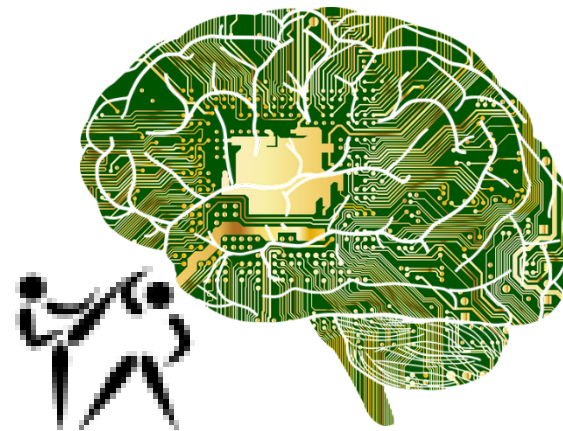
A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

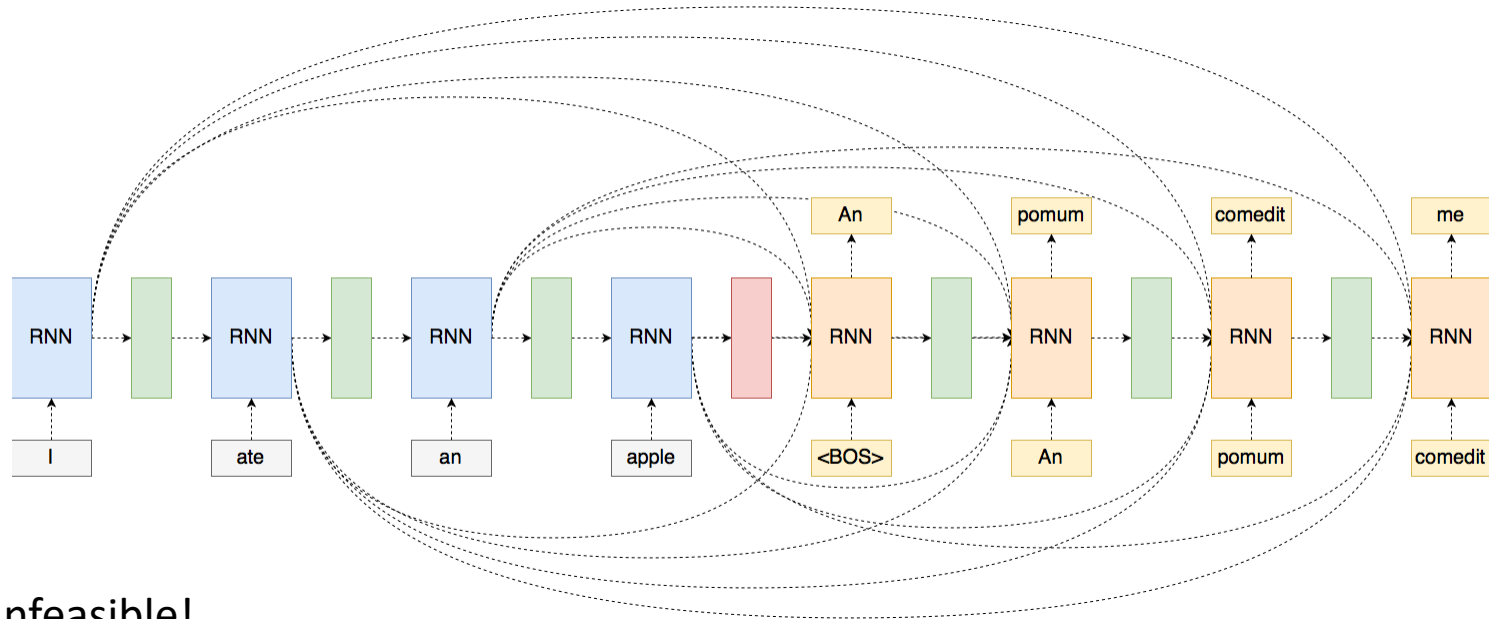
What to Be Covered Today...

- Adversarial Learning for Transfer Learning
- Recurrent Neural Networks
 - From RNN to LSTM & GRU
 - Sequence-to-Sequence Learning
 - Attention in RNN
- Transformer



RNN with Attention is Good, But..

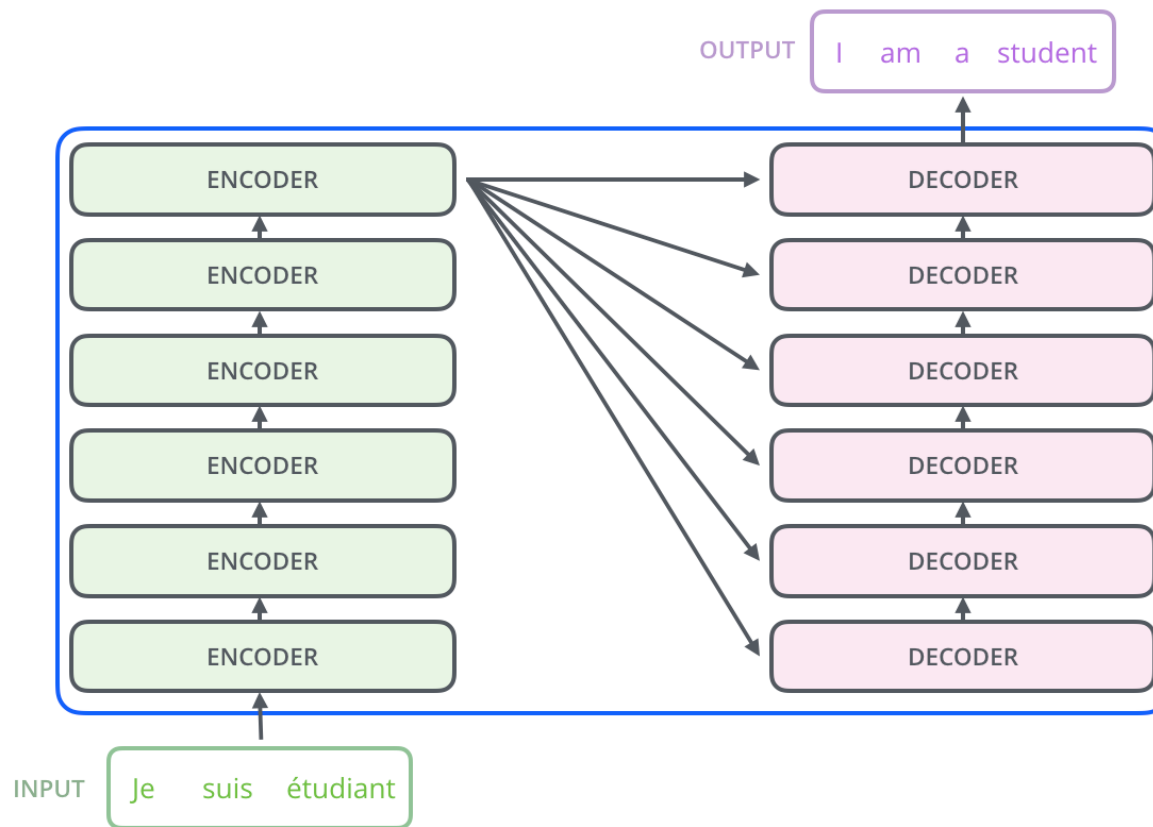
- Attention in a pre-defined sequential order
- Information loss due to long sequences...
- Connecting every hidden state between encoder and decoder?



- Infeasible!
 - Both inputs and outputs are with varying sizes.
 - Overparameterized

RNN with Attention is Good, But..

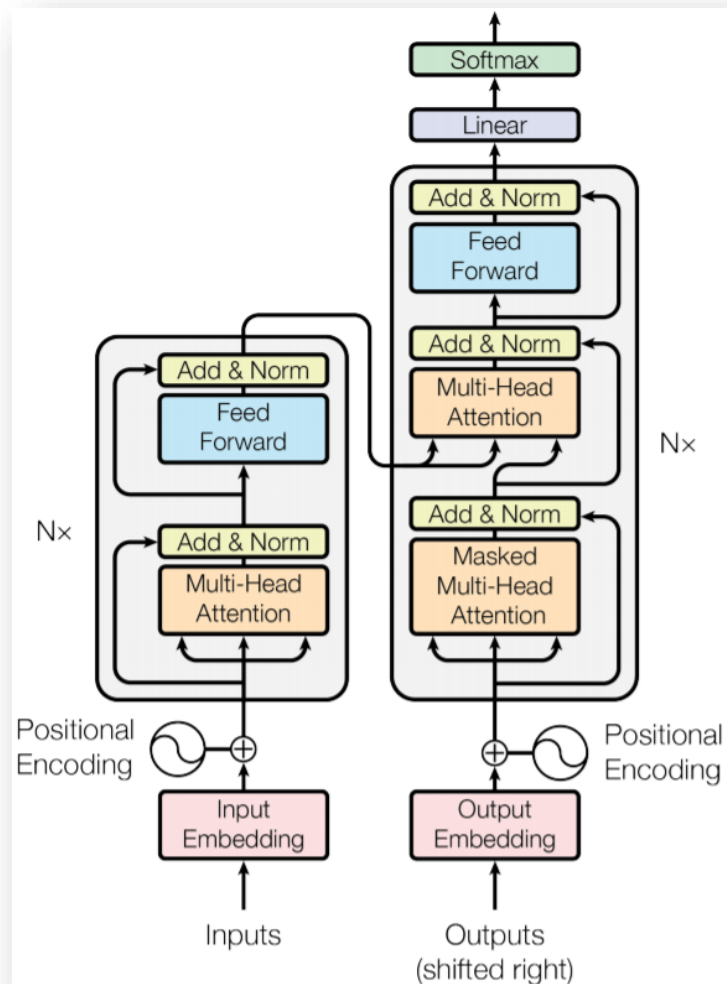
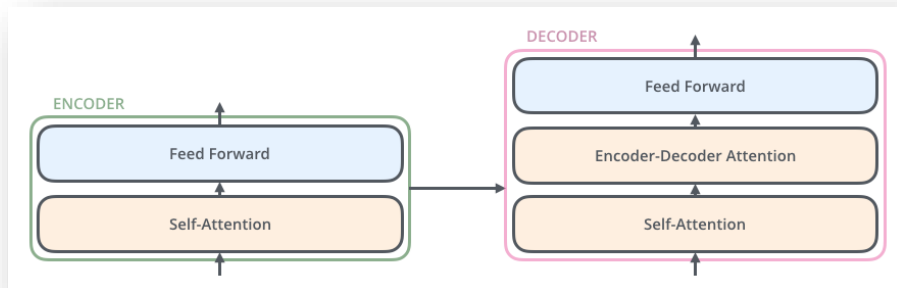
- Any better way to perform attention across features?



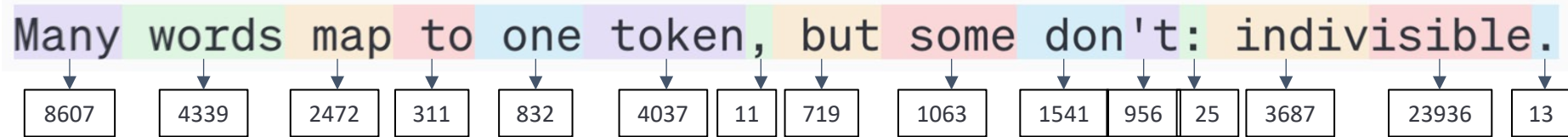


Solution #2: Transformer

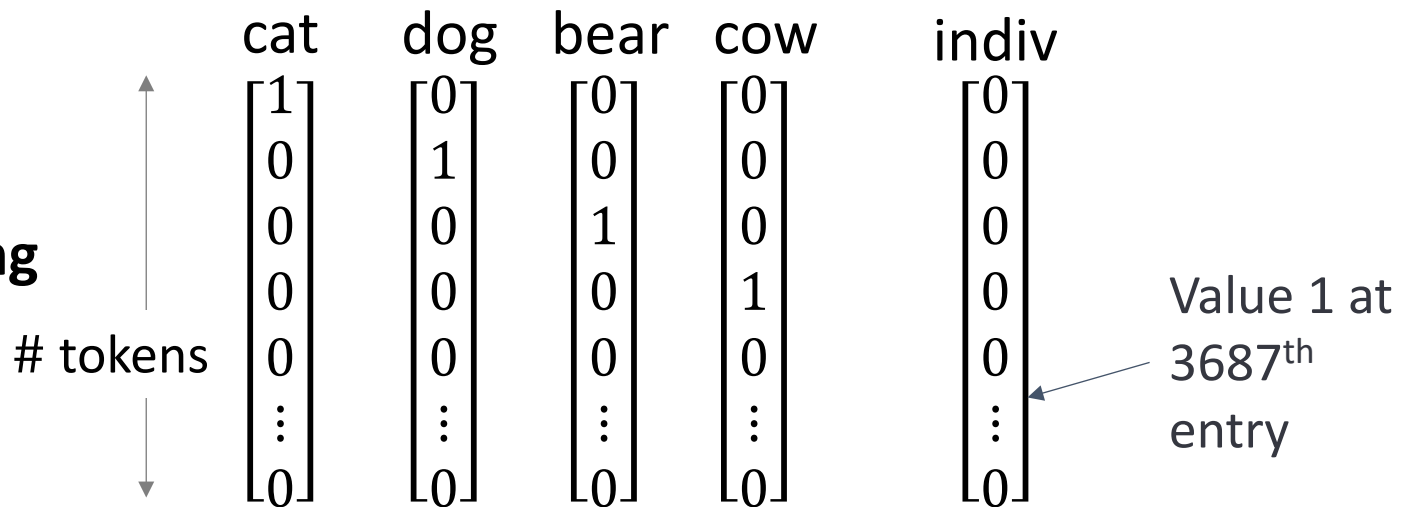
- “Attention is all you need”, NIPS/NeurIPS 2017
- Self-attention for text translation
- Say goodbye to CNN & RNN
- More details available at:
<https://www.youtube.com/watch?v=rcWMRA9E5RI>
<http://jalammar.github.io/illustrated-transformer/>



Tokenization



One-hot encoding

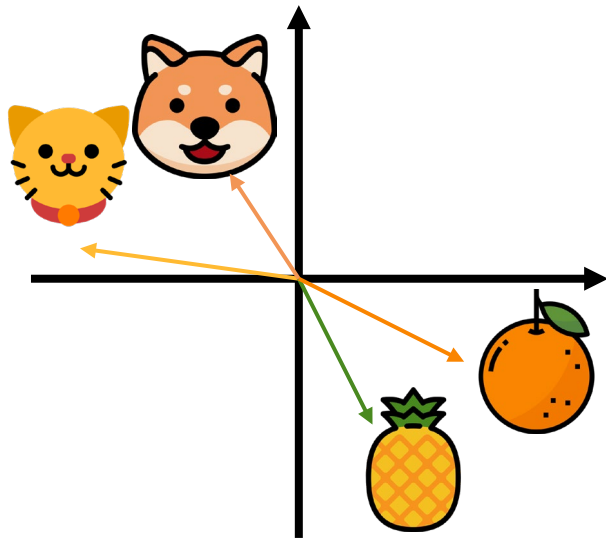


TOKEN EMBEDDING

One-hot encoding

		cat	dog	bear	cow	indiv	
		$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$	
							Value 1 at 3687 th entry

TOKEN EMBEDDING



Embedding Space

cat

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

dog

$$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

bear

$$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

cow

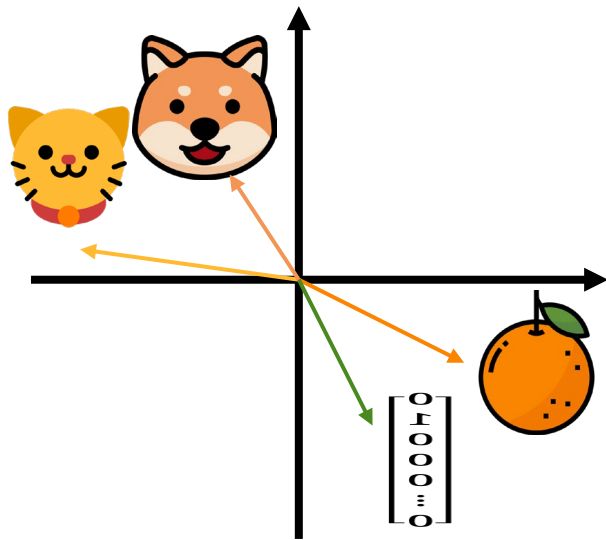
$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

indiv

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Value 1 at
3687th
entry

TOKEN EMBEDDING



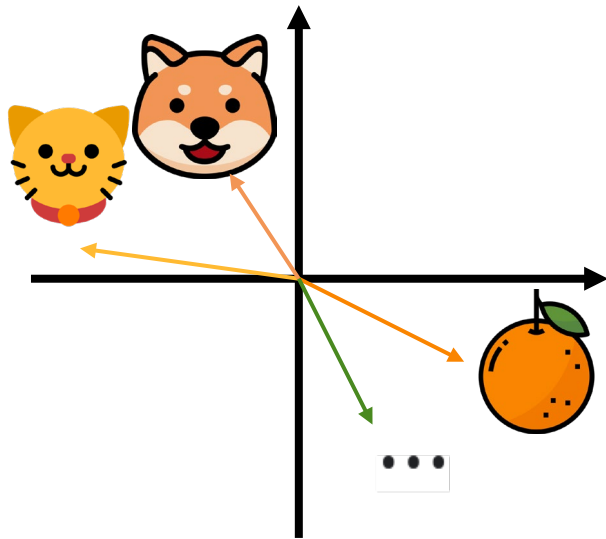
Embedding Space

$$\begin{array}{c} \updownarrow d \\ \begin{bmatrix} 0.5 \\ 2.7 \\ 1.2 \\ \vdots \\ 0.2 \end{bmatrix} \\ \updownarrow d \end{array} = \begin{array}{c} \begin{array}{c} \leftarrow \text{\# tokens} \rightarrow \\ \begin{bmatrix} W \\ E \end{bmatrix} \end{array} \begin{array}{c} \text{dog} \\ \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \end{array} \end{array}$$

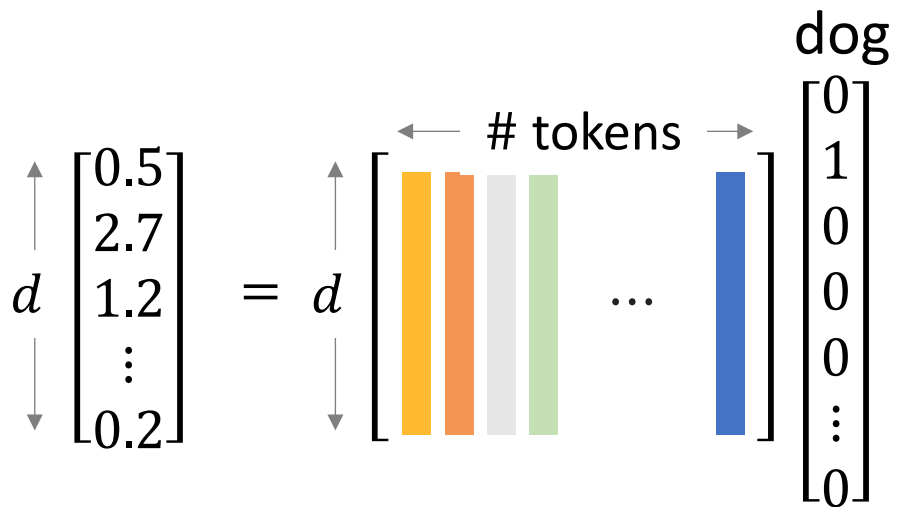
Embedded
token

Embedding Matrix

TOKEN EMBEDDING



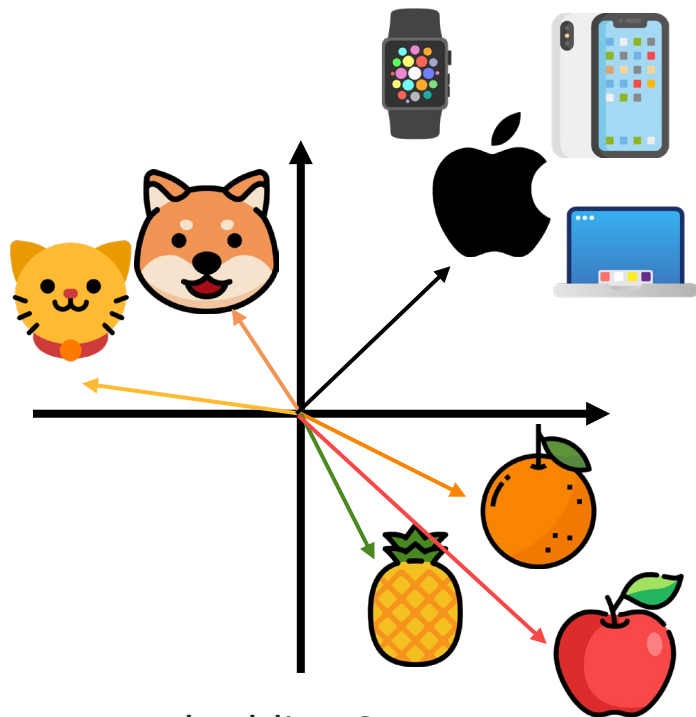
Embedding Space



Embedded token

Embedding Matrix

TOKEN EMBEDDING



Embedding Space

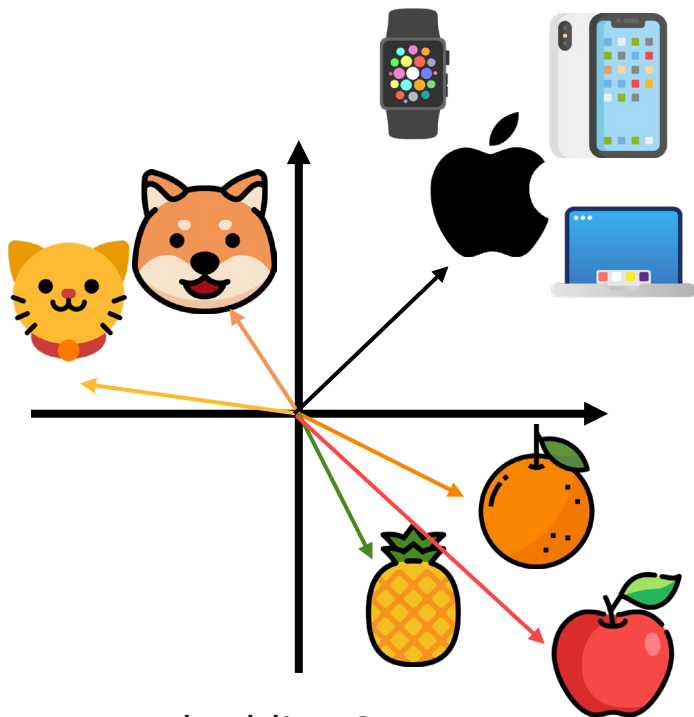
$$d \begin{bmatrix} 0.5 \\ 2.7 \\ 1.2 \\ \vdots \\ 0.2 \end{bmatrix}$$

Embedded token

$$= d \begin{bmatrix} \text{orange} & \text{apple} & \text{cat} & \text{dog} & \dots & \text{dog} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Embedding Matrix

TOKEN EMBEDDING



Embedding Space

Apple

I bought an **apple** and an orange.

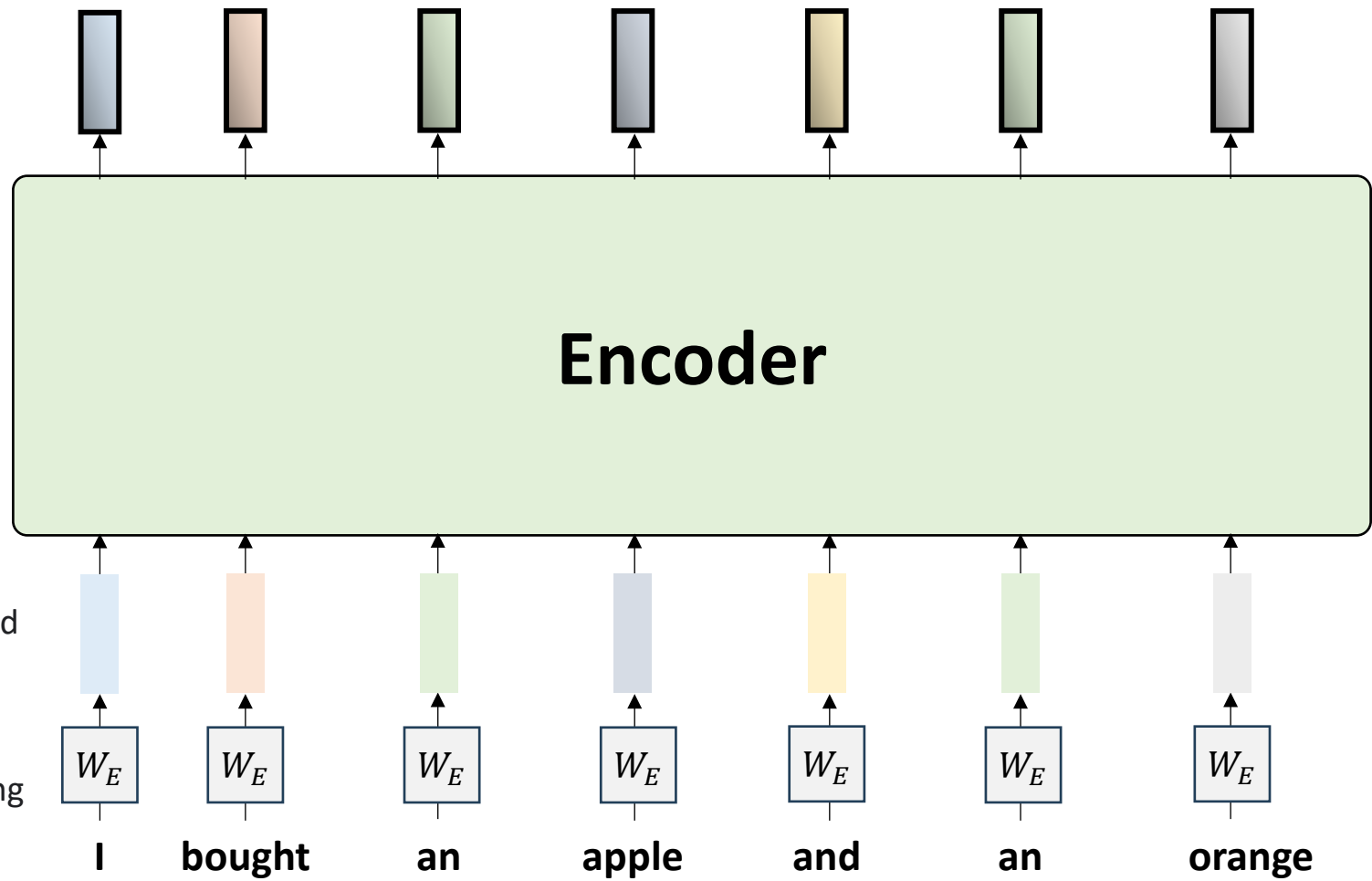
I bought an **apple** watch.

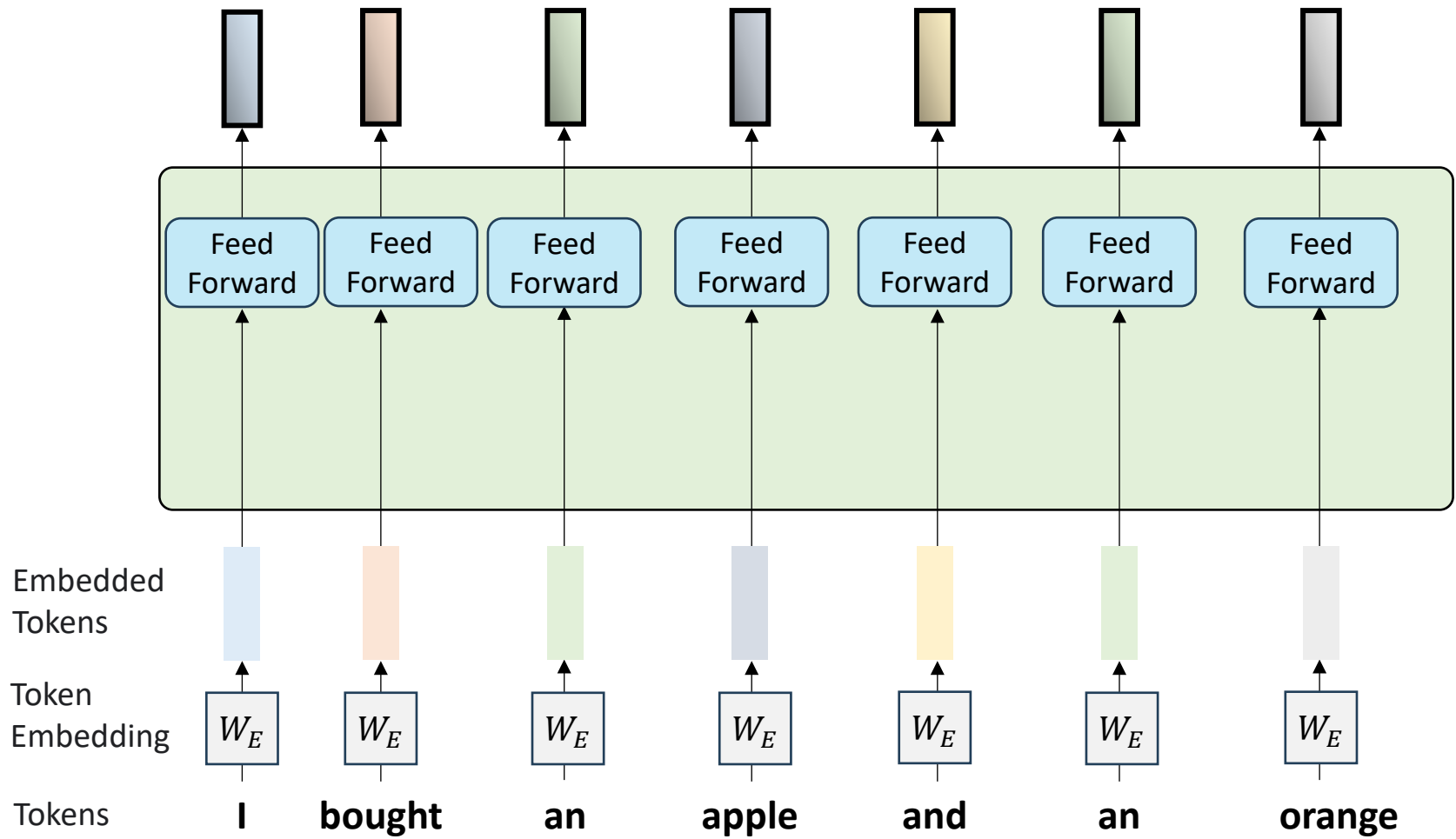
$$d \begin{bmatrix} 0.5 \\ 2.7 \\ 1.2 \\ \vdots \\ 0.2 \end{bmatrix}$$

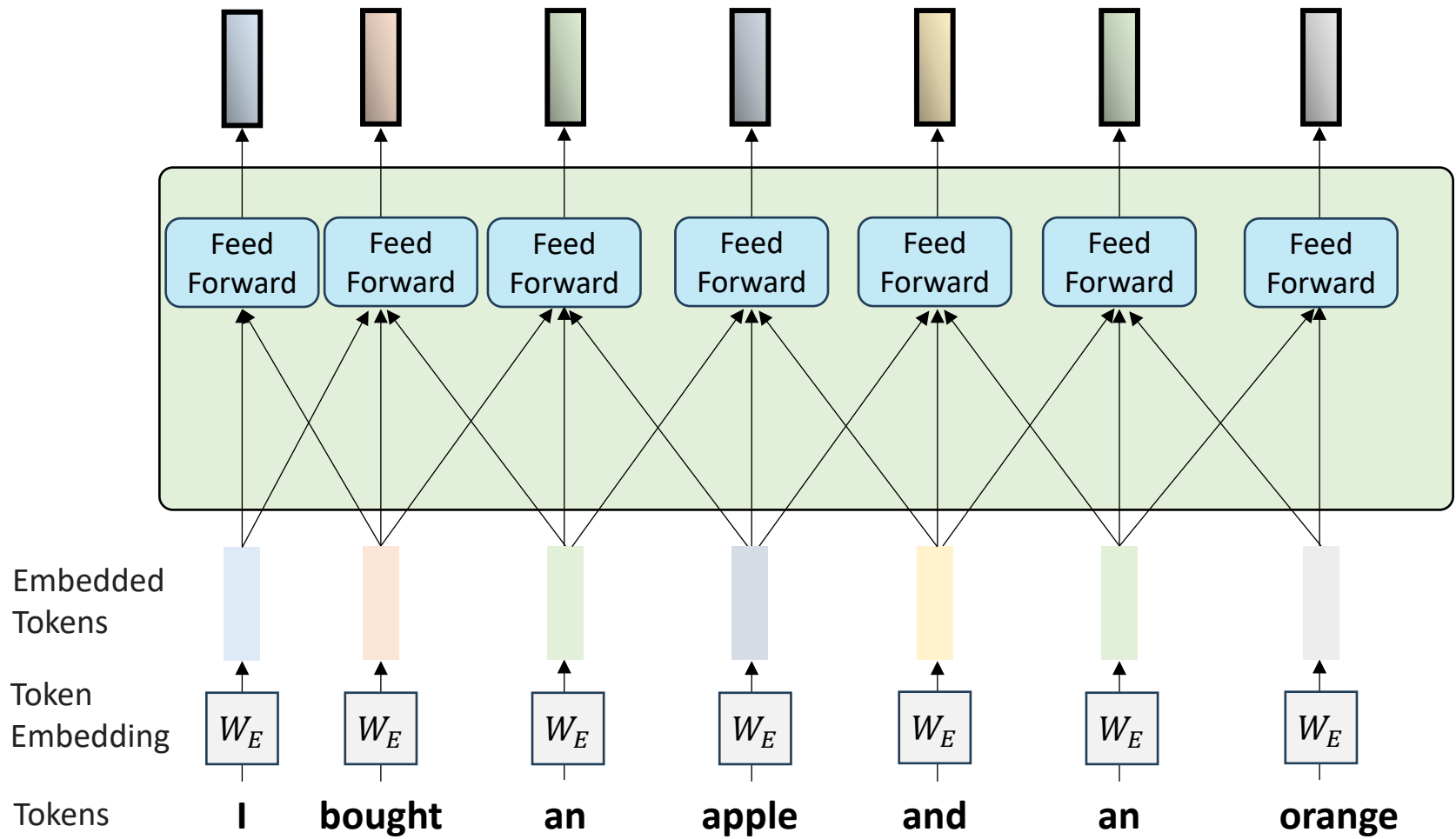
Embedded token

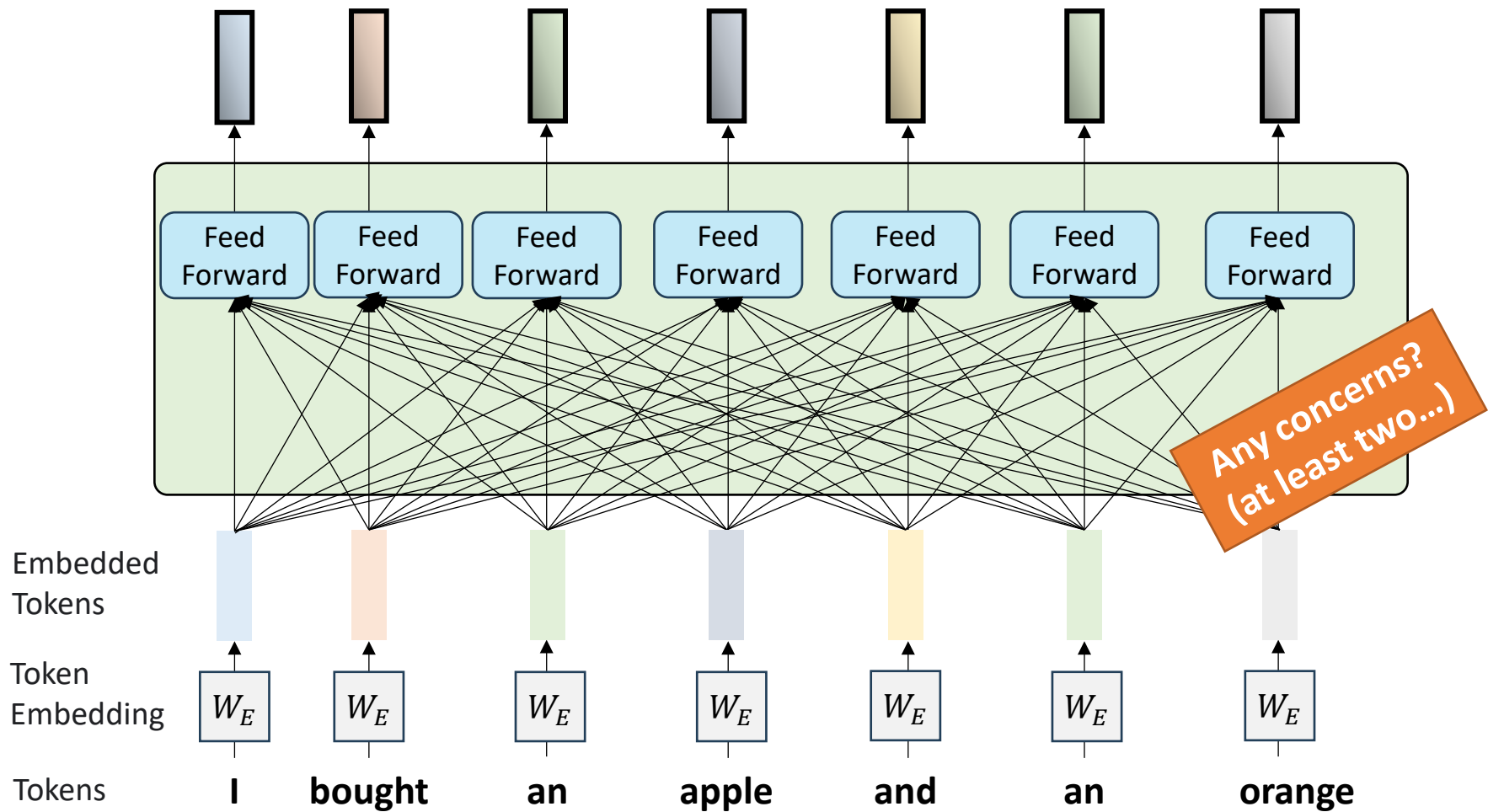
$$= d \begin{bmatrix} \text{orange} & \text{apple} & \text{watch} & \dots & \text{dog} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

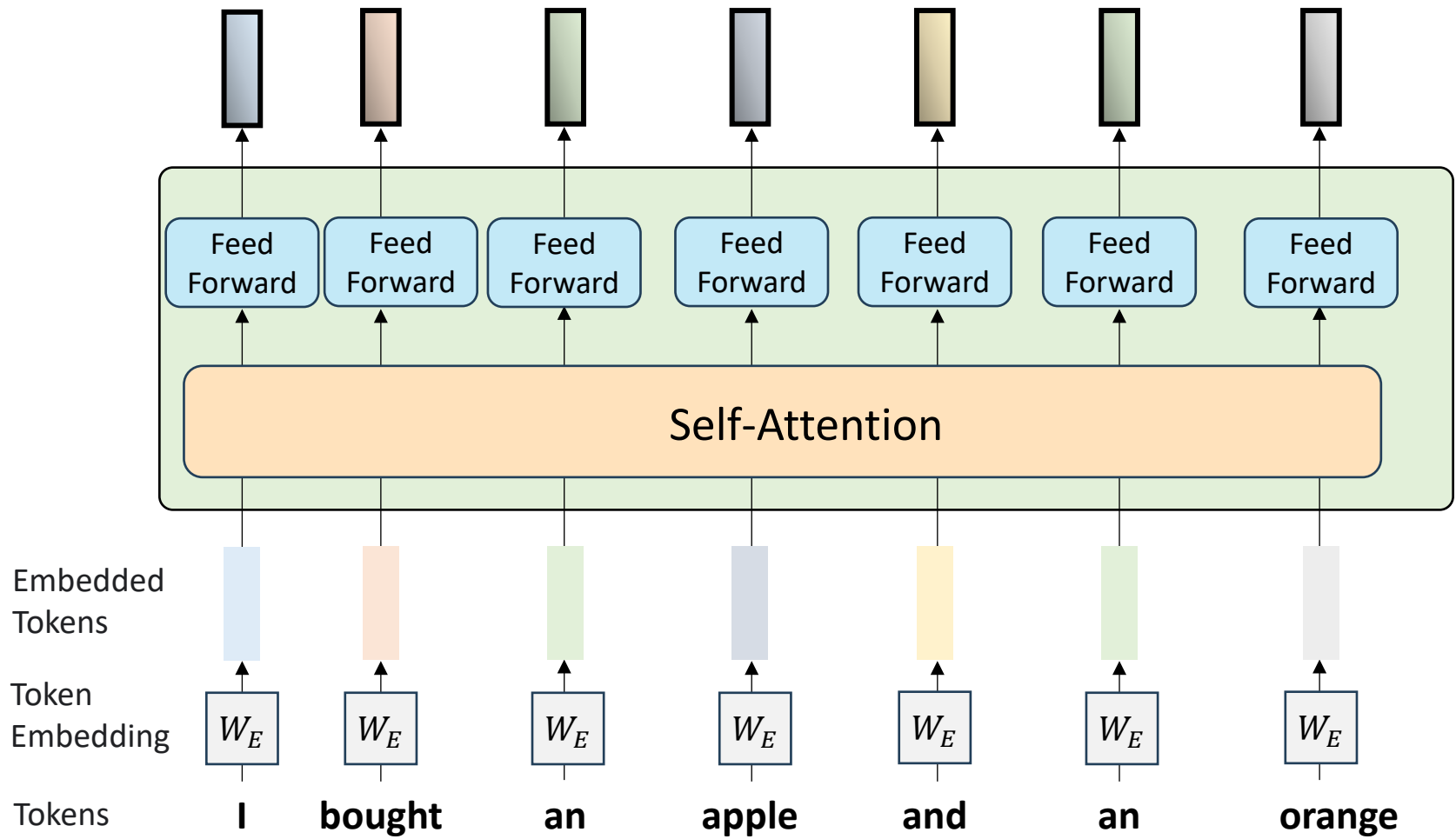
Embedding Matrix



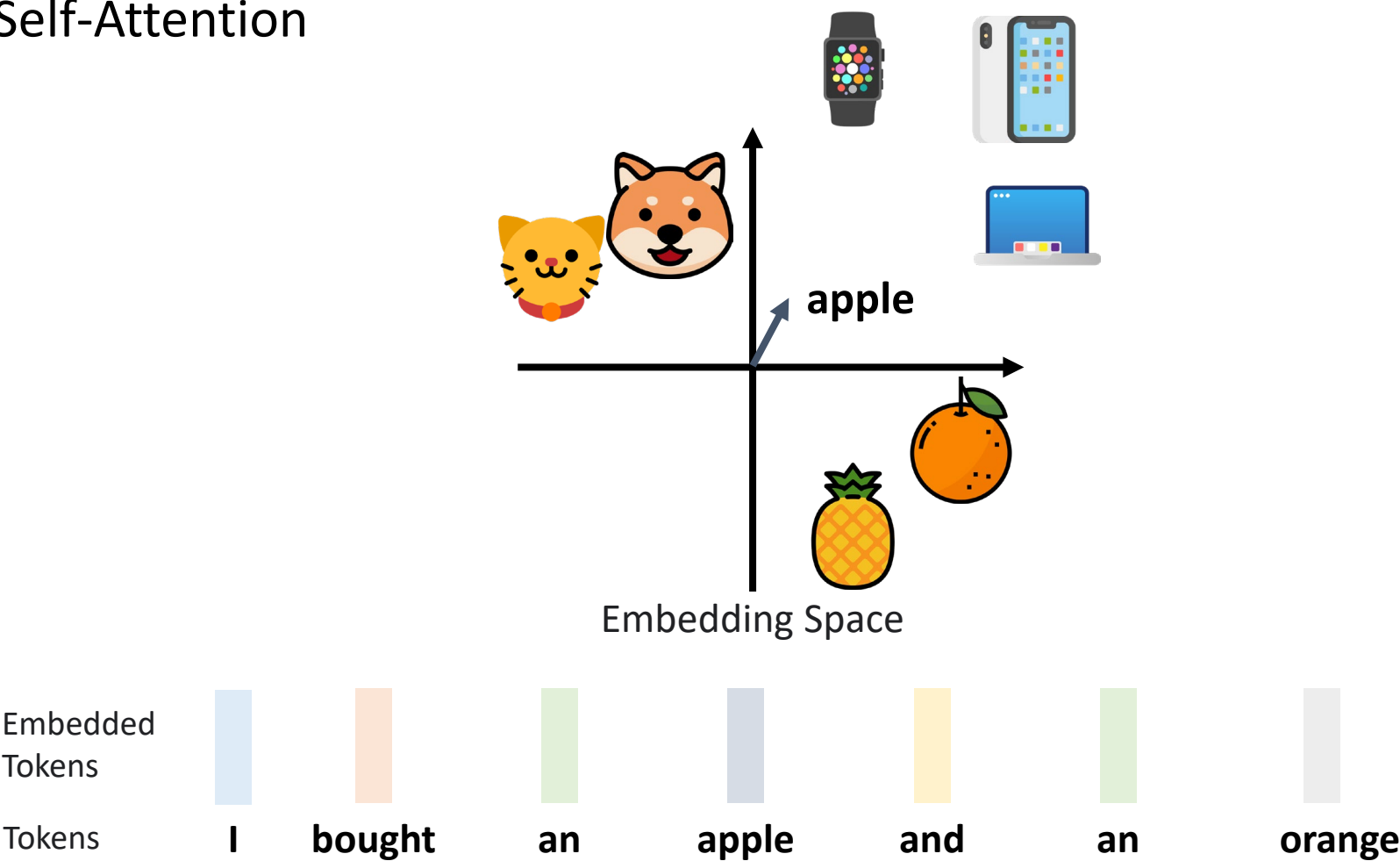




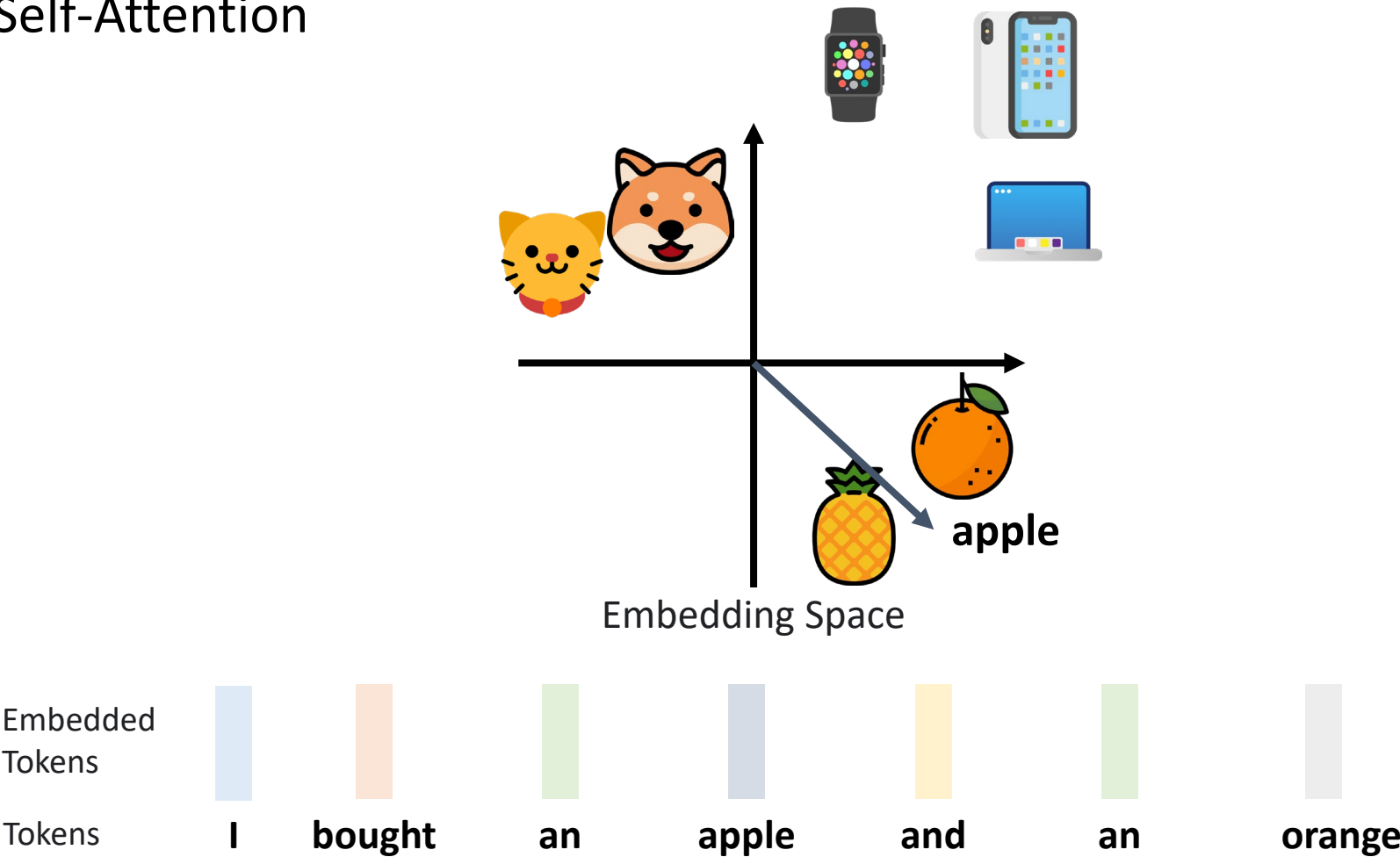




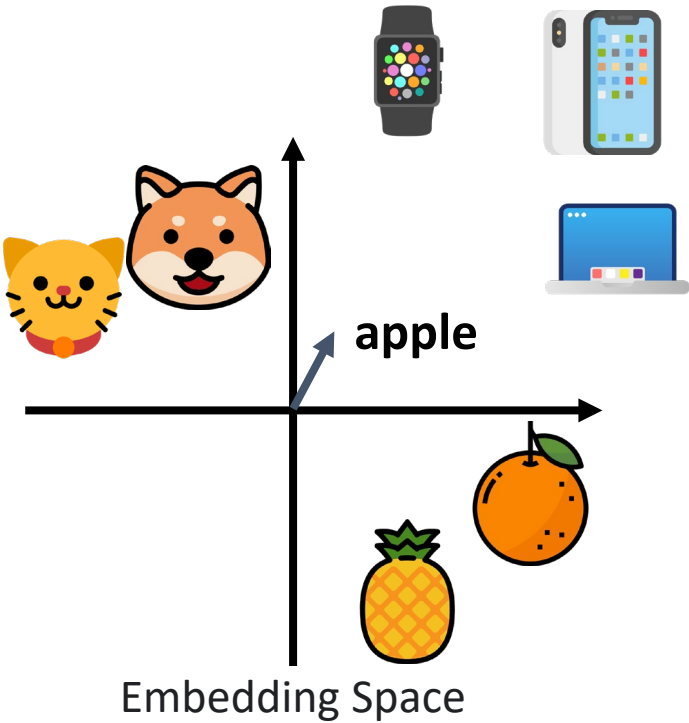
Self-Attention



Self-Attention

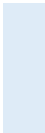


Self-Attention

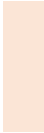


Embedded
Tokens

Tokens



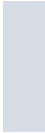
I



bought



an

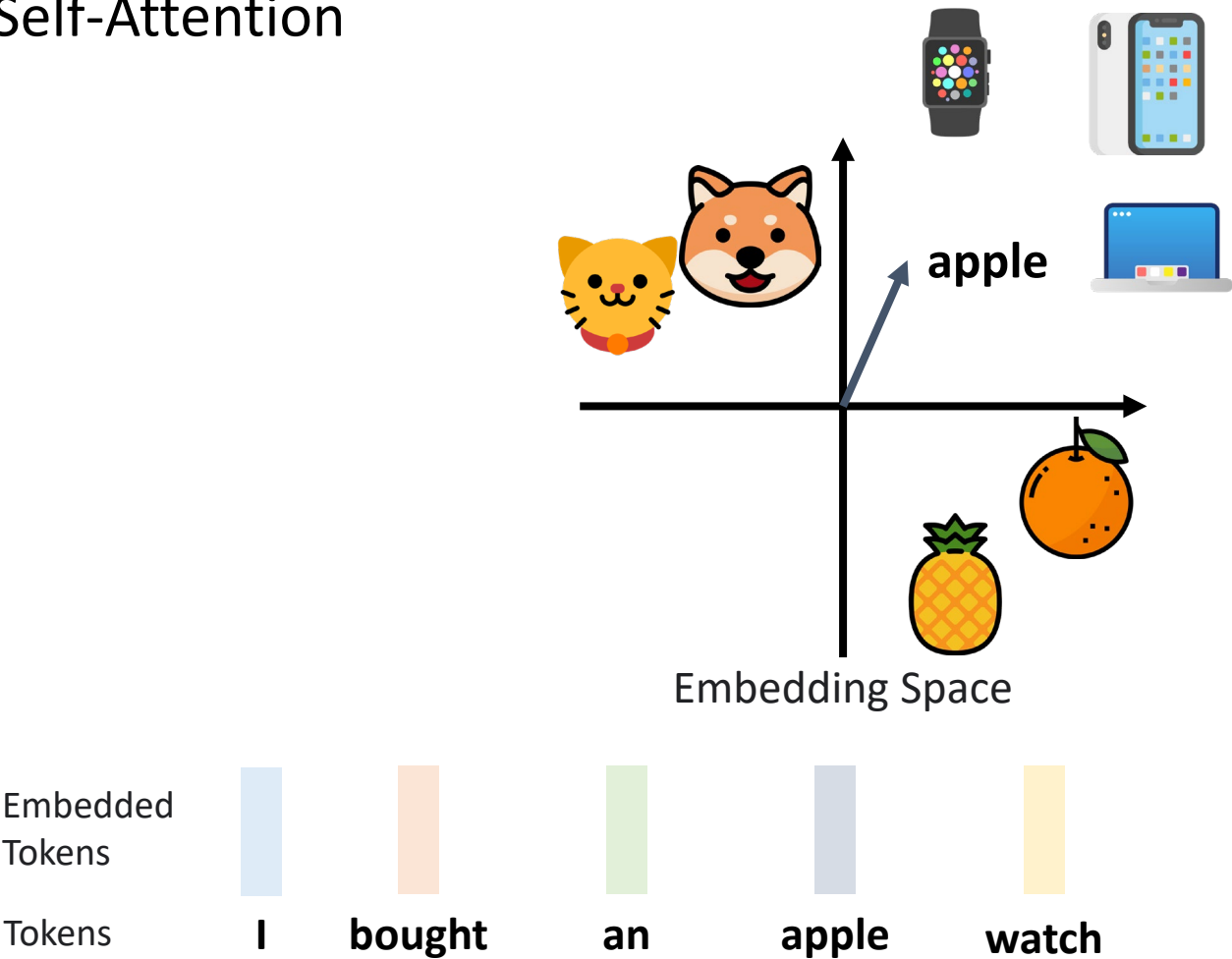


apple



watch

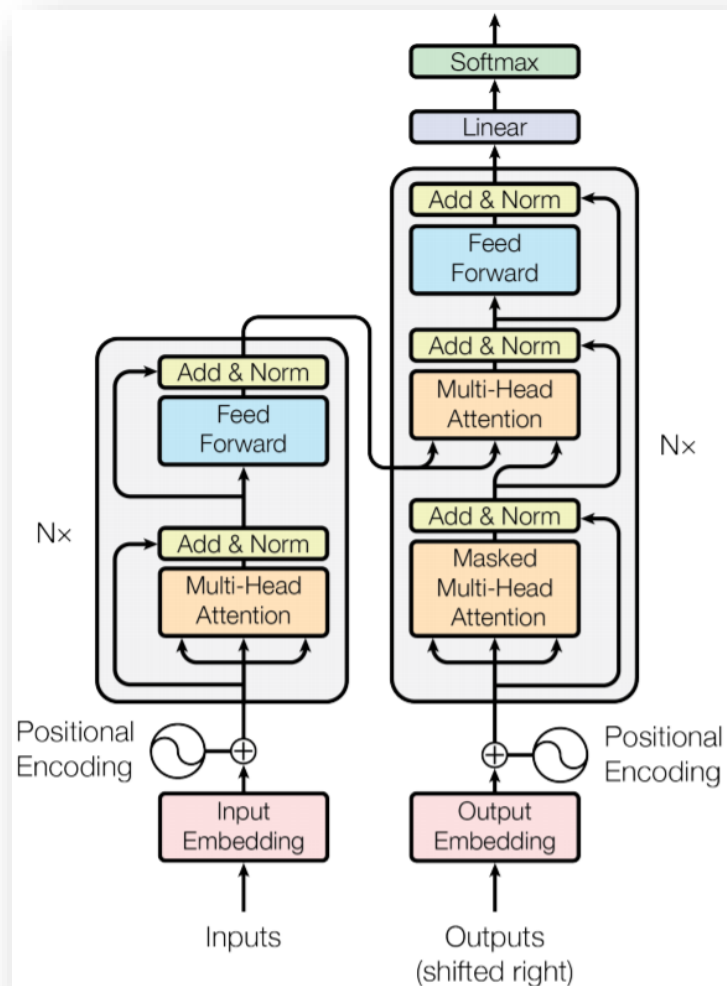
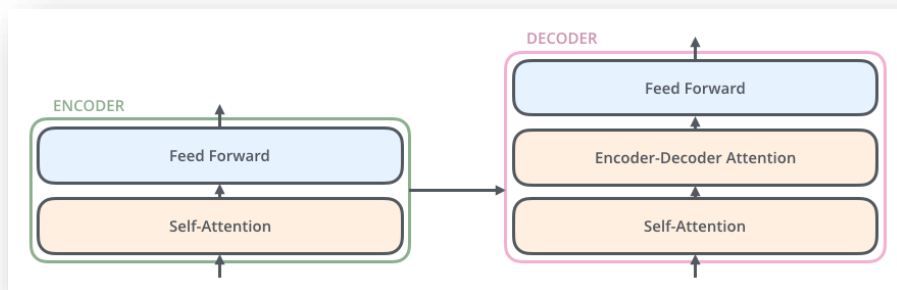
Self-Attention





Solution #2: Transformer

- “Attention is all you need”, NIPS/NeurIPS 2017
- Self-attention for text translation
- More details available at:
<https://www.youtube.com/watch?v=rcWMRA9E5RI>
<http://jalammr.github.io/illustrated-transformer/>



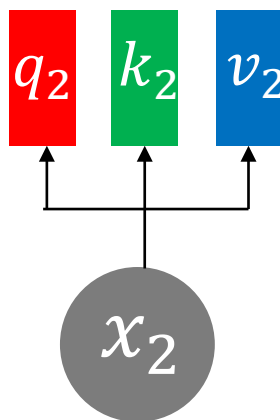
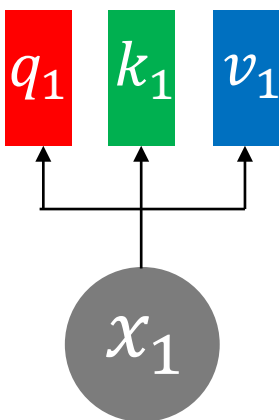
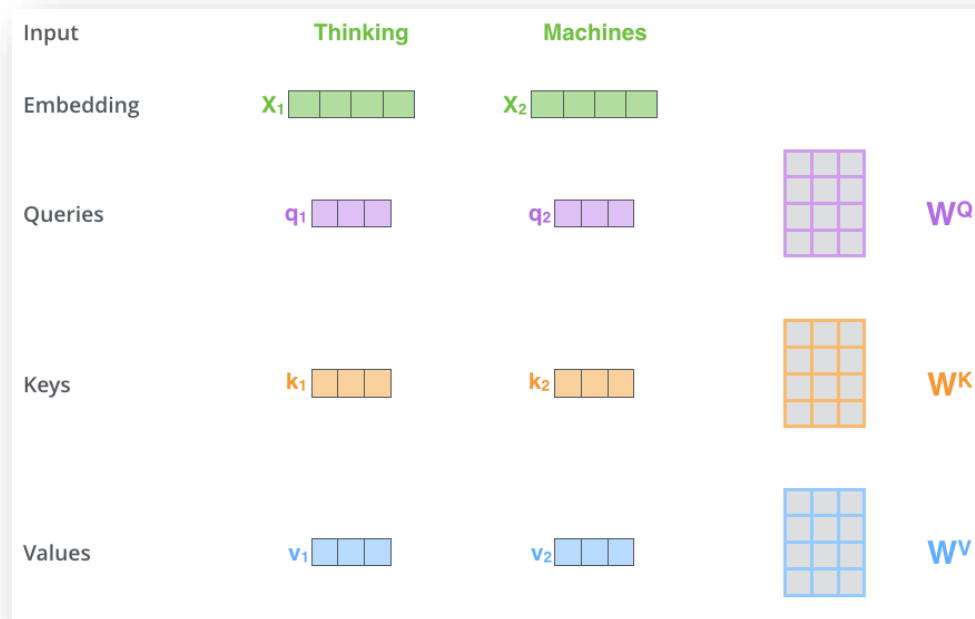
Self-Attention (1/5)

- Query q , key k , value v vectors are learned from each input x

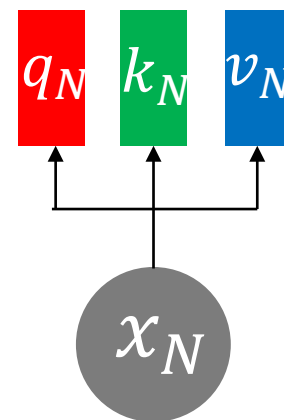
$$q_i = W^Q x_i$$

$$k_i = W^K x_i$$

$$v_i = W^V x_i$$



...

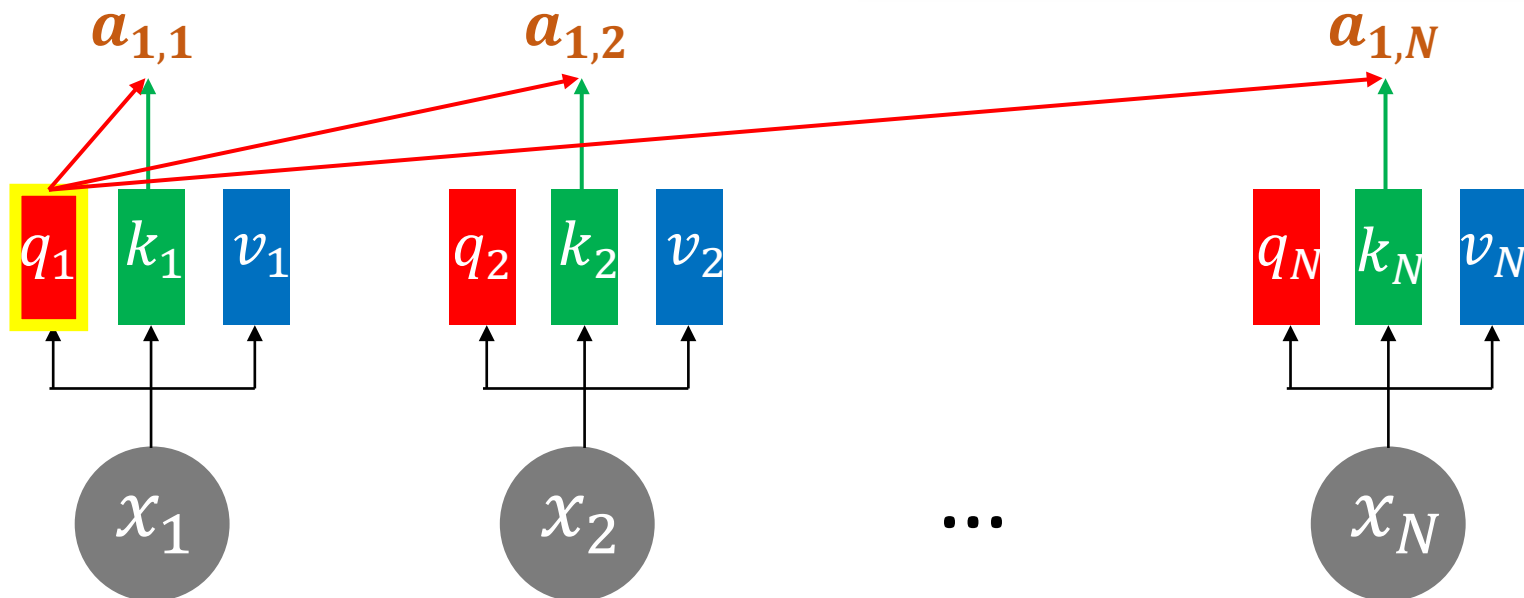


Self-Attention (2/5)

- Relation between each input is modeled by inner-product of **query** q and **key** k .

$$a_{1,i} = \frac{q_1 \cdot k_i}{\sqrt{d}}, \text{ where } a \in R, q, k \in R^d$$

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$

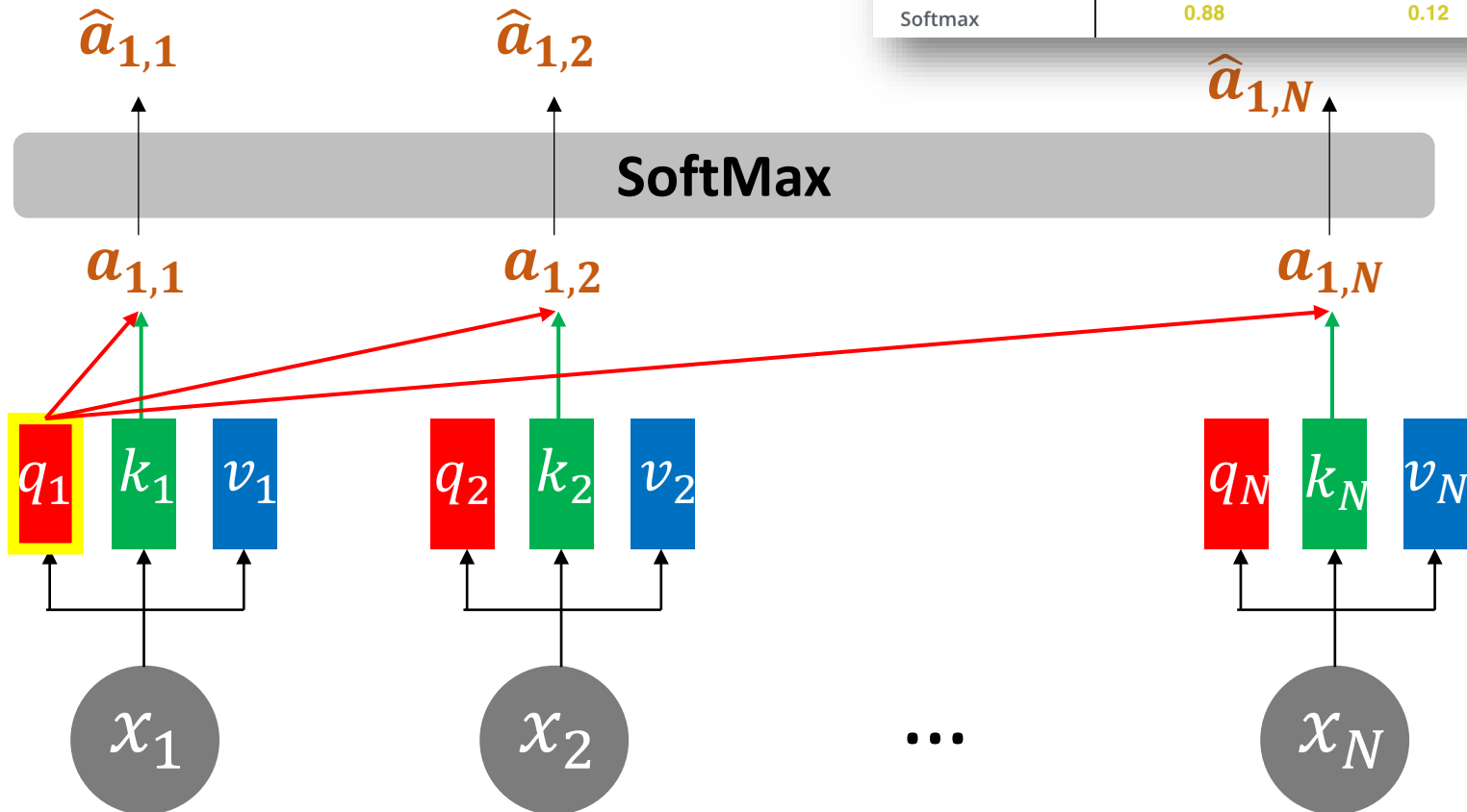


Self-Attention (3/5)

- SoftMax is applied:

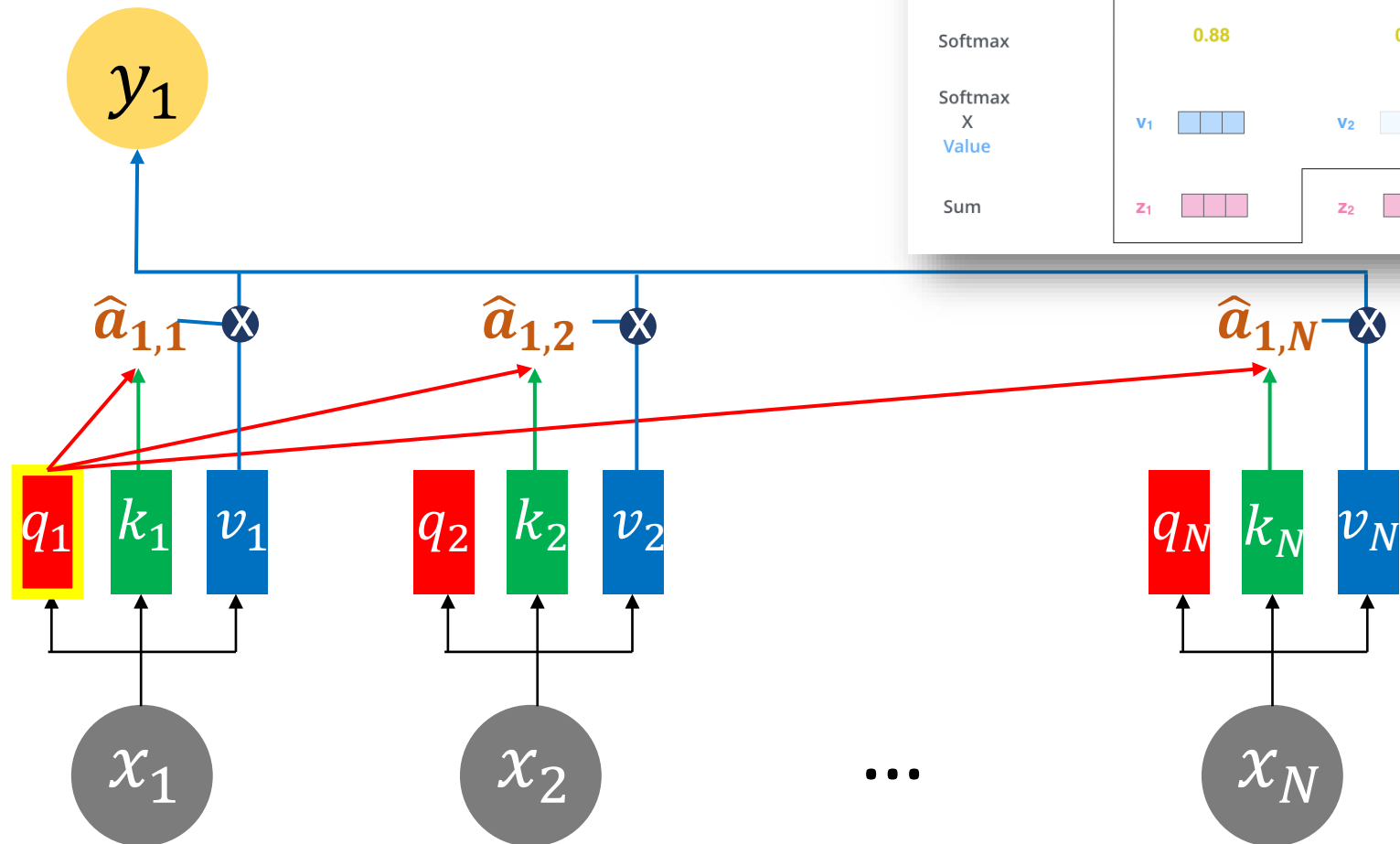
$$0 \leq \hat{a}_i = e^{a_i} / \sum_j^N e^{a_j} \leq 1, \text{ for } i=1, \dots, N$$

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12



Self-Attention (4/5)

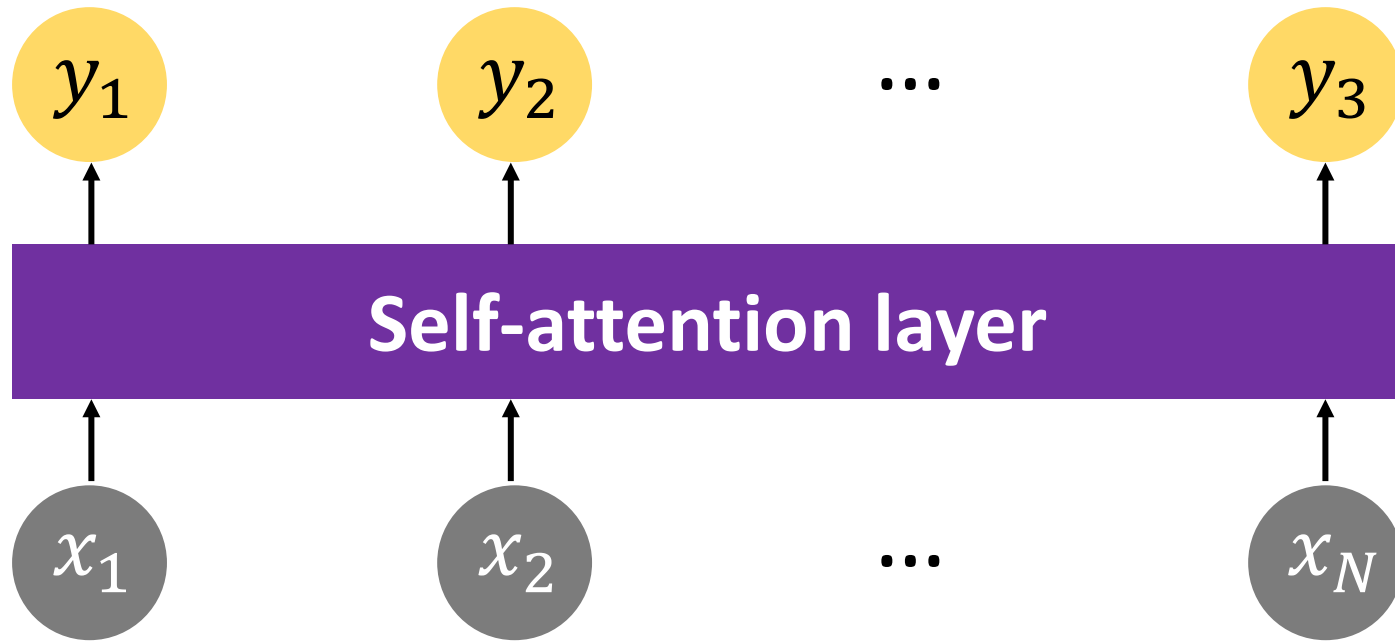
- Value vectors v are aggregated with attention weight $\hat{a}$, i.e., $y_1 = \sum_i^N \hat{a}_i \cdot v_i$



Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12
Softmax X Value	v_1	v_2
Sum	z_1	z_2

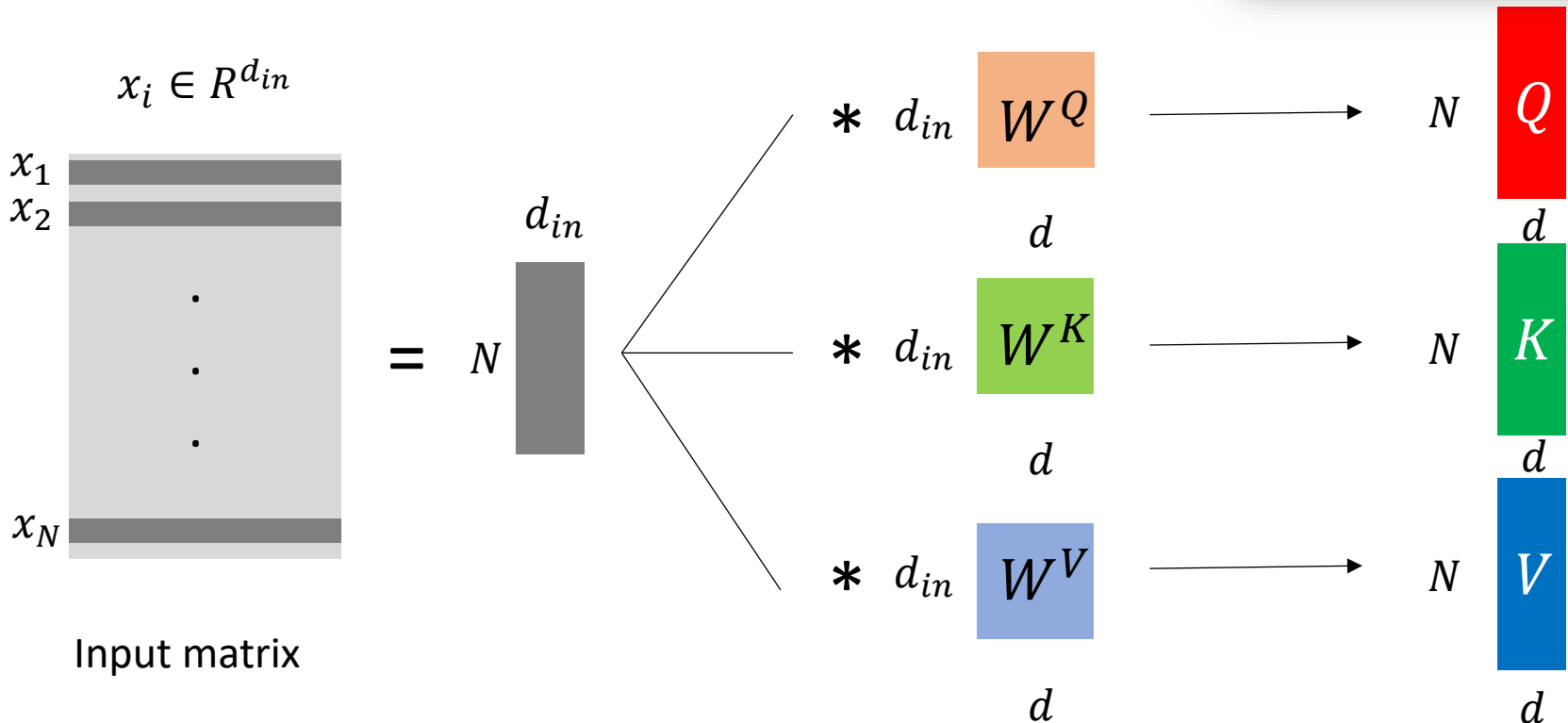
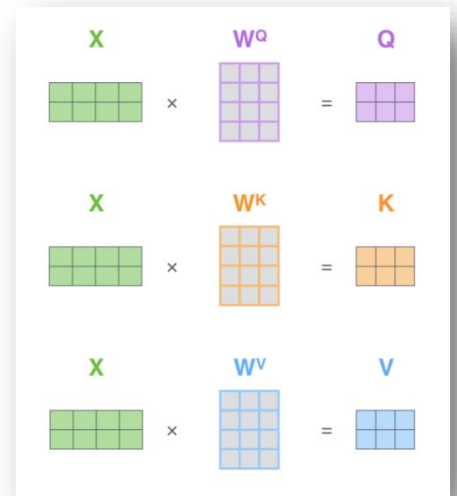
Self-Attention (5/5)

- All y_i can be computed **in parallel**
- Each y_i considers $x_1 \sim x_N$, modeling their **long-distance dependencies**.
- Global feature can be obtained by **average-pooling** over $y_1 \sim y_N$



Self-Attention: Implementation

- Input sequence can be represented as a $N \times d_{in}$ matrix
- * denotes matrix multiplication



Self-Attention: Implementation

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

- Output matrix Y
- All operations are **matrix multiplication**, can be parallelized on GPU.

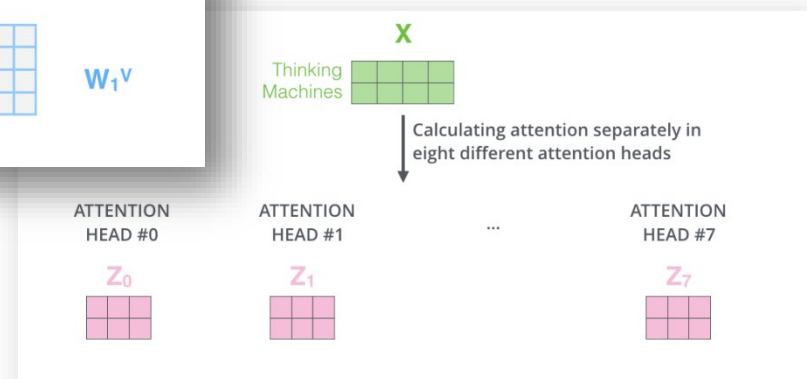
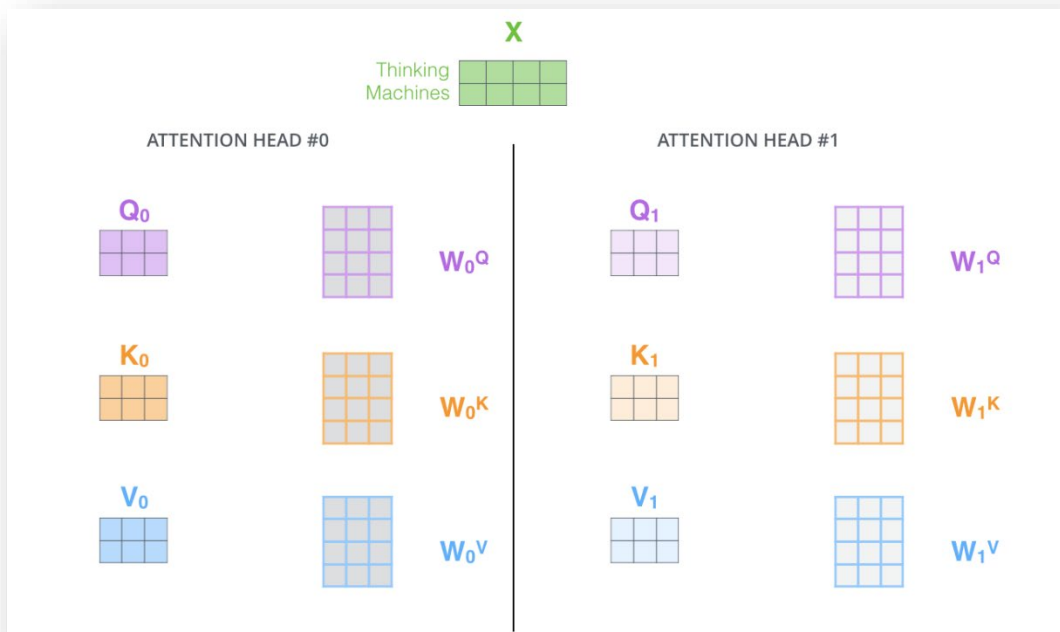
$$N \times d \text{ } Q * d \times N \text{ } K^T \xrightarrow{/\sqrt{d}} N \times N \text{ } A \xrightarrow{\text{SoftMax}} N \times N \text{ } \hat{A}$$

$$N \times N \text{ } \hat{A} * N \times d \text{ } V \longrightarrow N \times d \text{ } Y$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

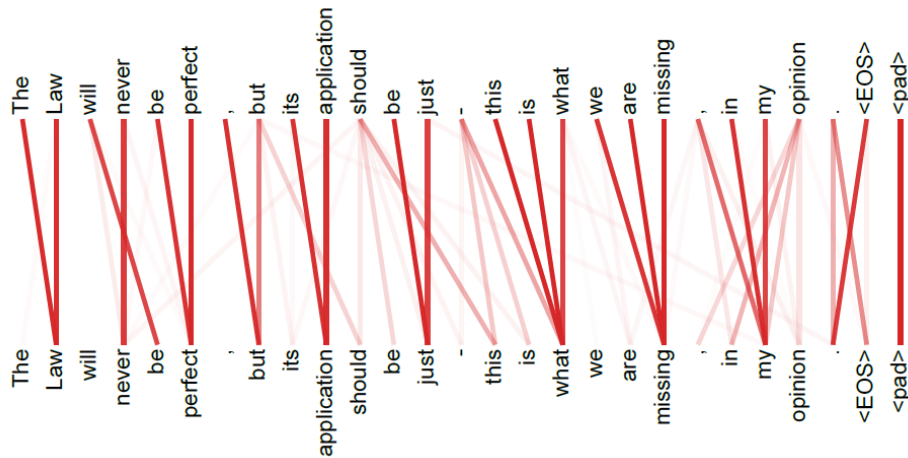
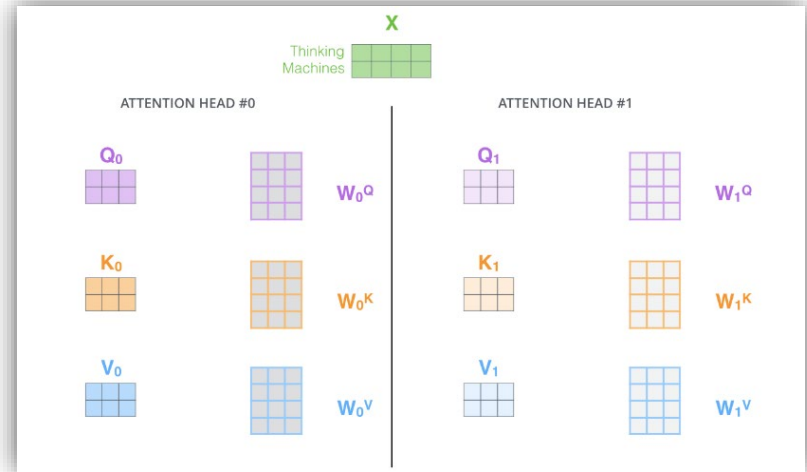
Multi-Head Self-Attention (1/4)

- Perform self-attention at **different subspaces**, implying performing attention over different input feature types (e.g., representations, modalities, positions, etc.)

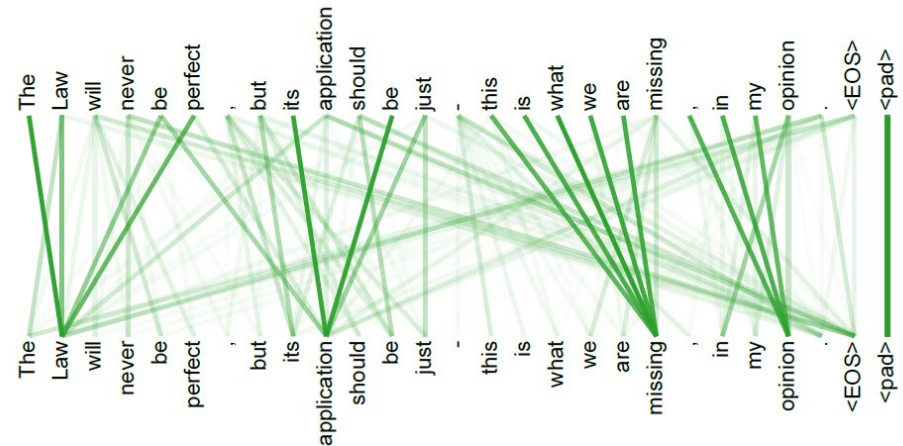


Multi-Head Self-Attention (2/4)

- Perform self-attention at **different subspaces**, implying performing attention over different input feature types
- See example below



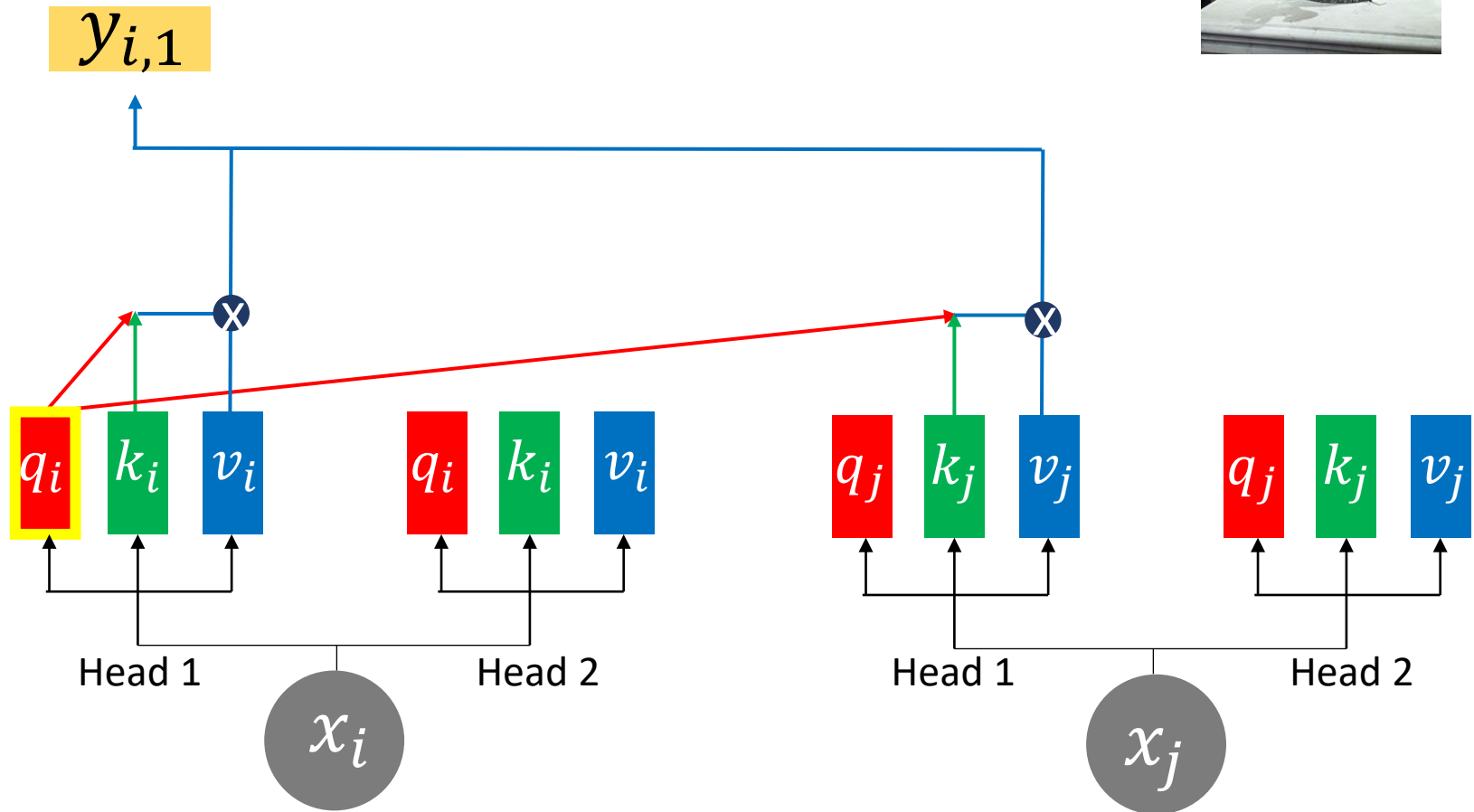
Attention weights
of Head 1



Attention weights
of Head 2

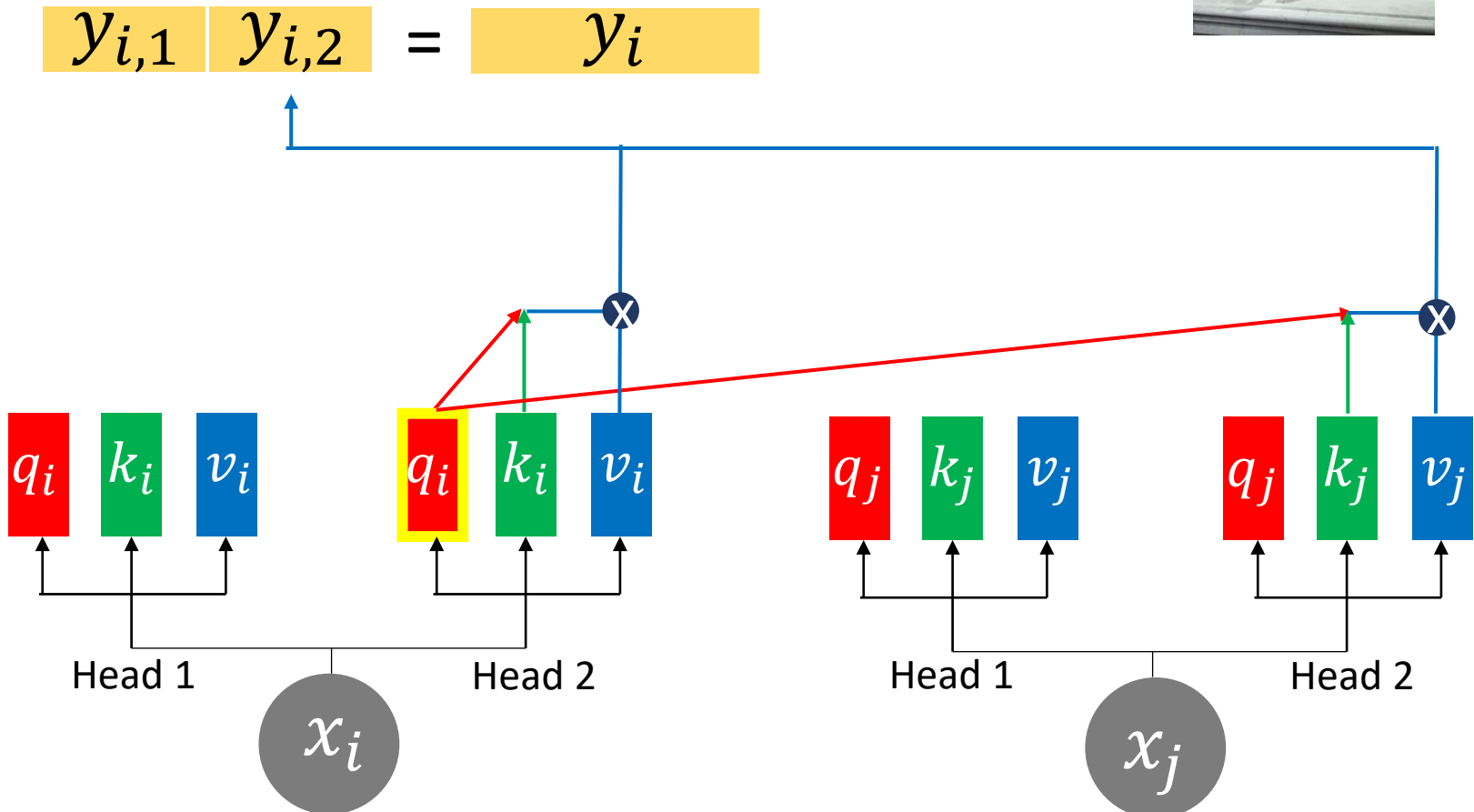
Multi-Head Self-Attention (3/4)

- A 2-head example, output of two heads are concatenated.

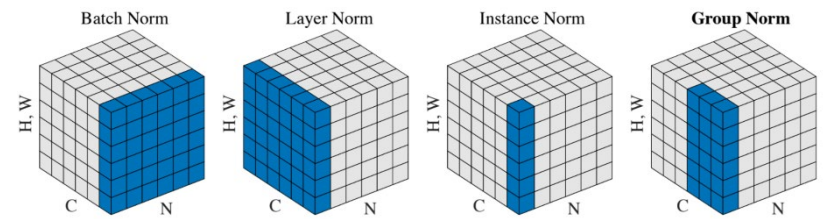


Multi-Head Self-Attention (4/4)

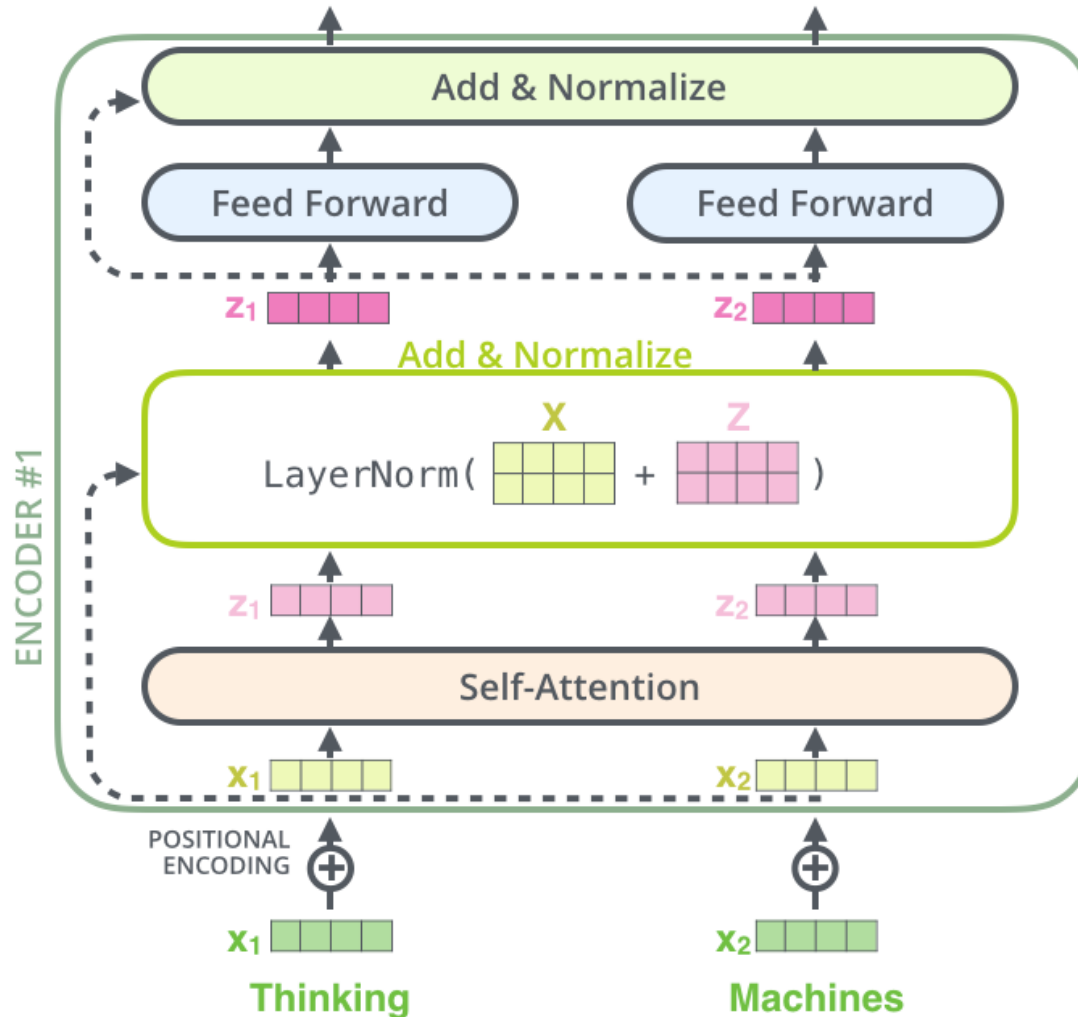
- A 2-head example, output of two heads are concatenated.



The Residuals

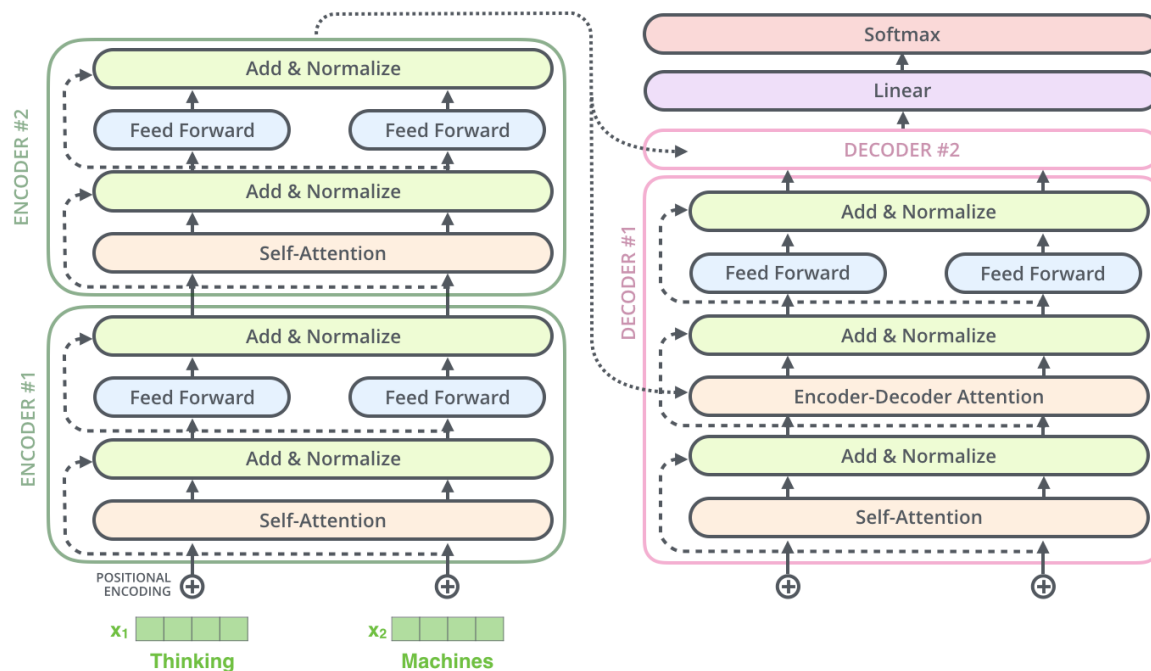


- A residual connection followed by layer normalization



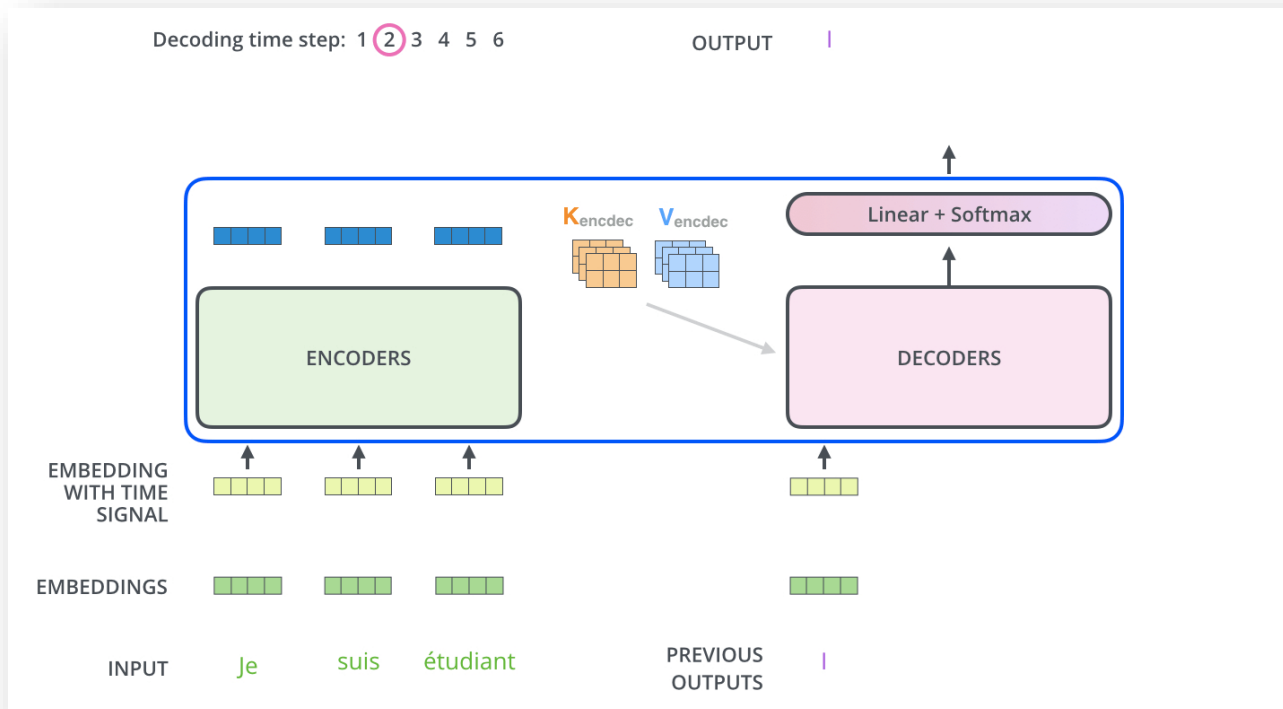
The Decoder in Transformer (if time permits today)

- Encoder-decoder attention
 - Q from self-attn in decoder, K & V from encoder outputs
- Masked multi-head attention
 - Design similar to that of encoder, except for decoder #1 which takes additional inputs (of GT/predicted word embeddings).
 - Mask unpredicted tokens during softmax: why?



The Decoder in Transformer (cont'd)

- Encoder-decoder attention
 - Q from self-attn in decoder, K & V from encoder outputs
- Masked multi-head attention
 - Design similar to that of encoder, except for decoder #1 which takes additional inputs (of GT/predicted word embeddings).
 - Mask unpredicted tokens during softmax: why?



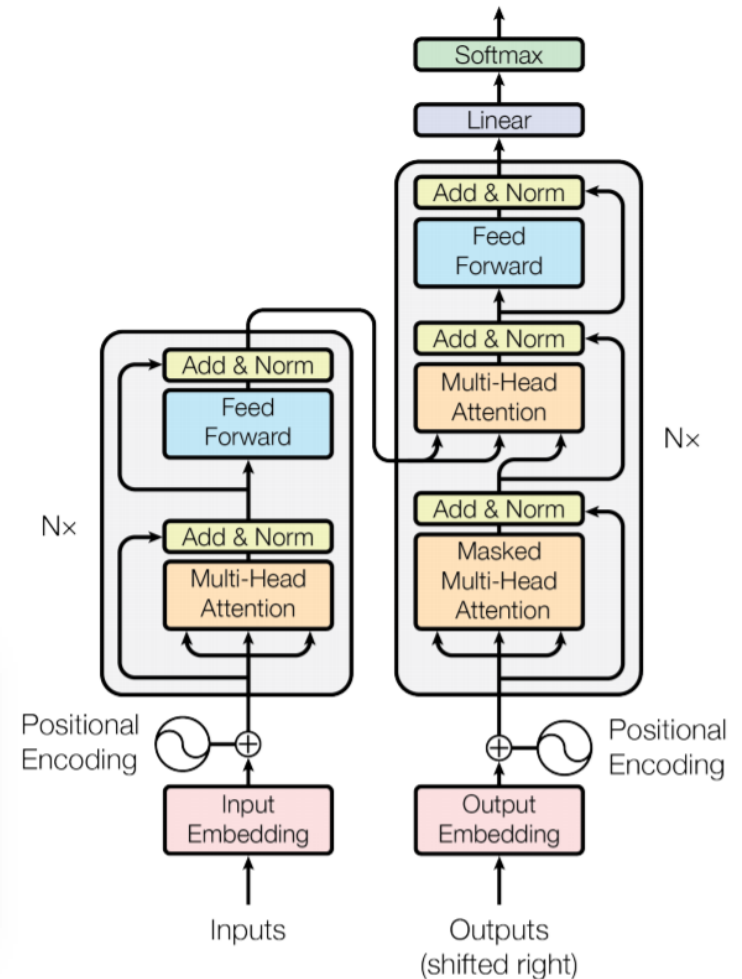
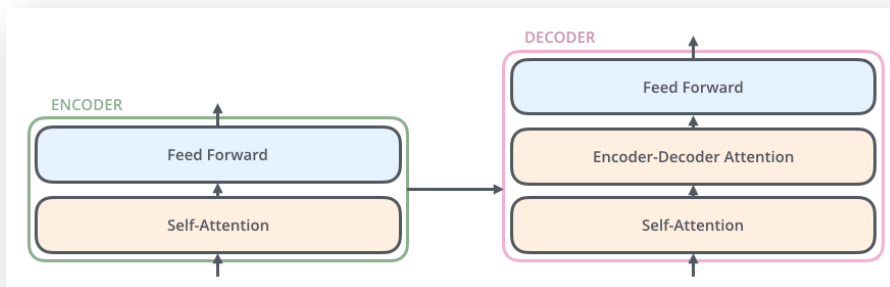
Overview of Decoding in Transformer

- Encoder/Decoder Cross-Attention + Decoder self-attention



Recap: Transformer

- “Attention is all you need”, NeurIPS 2017
- We didn’t cover **positional encoding** (particularly for language translation)
- Potential problems of Transformer?



What We've Covered Today...

- Adversarial Learning for Transfer Learning
- Recurrent Neural Networks
 - From RNN to LSTM & GRU
 - Sequence-to-Sequence Learning
 - Attention in RNN
- Transformer
 - Multi-head Attention

